



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
INFORMÁTICA**

TRABAJO FIN DE GRADO

**Desarrollo de un generador configurable
de modelos de características UVL
integrado en UVLHub**

Realizado por

Jiménez Ortega, Antonio

Para la obtención del título de

Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por

Benavides Cuevas, David

Romero Organvández, David

En el departamento de

Departamento de Lenguajes y Sistemas Informáticos

Convocatoria de octubre, curso 2025/26

DECLARACIÓN DE AUTORÍA

D./Dña. **Jiménez Ortega, Antonio**, autor/a del Trabajo Fin de Grado titulado «Desarrollo de un generador configurable de modelos de características UVL integrado en UVLHub», presentado en Sevilla, agosto 2026 para la obtención del título de Grado en Ingeniería Informática - Ingeniería del Software, declara bajo su responsabilidad que:

- El trabajo es original y ha sido realizado por el/la autor/a en colaboración con su/s tutor/es académicos.
- Se han citado adecuadamente todas las fuentes consultadas, tanto bibliográficas como digitales.
- No se ha plagiado contenido de otros autores ni se han utilizado materiales sin la correspondiente autorización.
- Autoriza a la Universidad a utilizar este trabajo con fines académicos y de difusión, citando siempre su autoría.

Sevilla, a 3 de agosto de 2026

Fdo.: Jiménez Ortega, Antonio

AGRADECIMIENTOS

Quiero expresar mi profundo agradecimiento a mis tutores, David Benavides y David Romero, por la orientación, supervisión y apoyo proporcionados durante el desarrollo de este Trabajo de Fin de Grado. Sus indicaciones y experiencia han sido fundamentales para comprender el contexto del proyecto, tomar las decisiones funcionales y técnicas adecuadas, y afrontar los distintos retos que han ido apareciendo durante su realización.

También quiero agradecer al grupo de DiversoLab y a todas las personas que han contribuido al desarrollo del entorno tecnológico sobre el que se ha construido este trabajo por facilitar un marco en el que aplicar los conocimientos adquiridos durante mis años en el grado.

Finalmente, dedico este trabajo a mi familia y a todo mi entorno cercano. Su acompañamiento y confianza han sido indudablemente esenciales para completar este proyecto y afrontar el esfuerzo que supone un trabajo de esta magnitud. Abuela, dondequiera que estés, ya estoy un paso más cerca de convertirme en ingeniero.

Mairena del Alcor, agosto 2026

ACKNOWLEDGEMENTS

I would like to express my deep gratitude to my mentors, David Benavides and David Romero, for the guidance, supervision, and support they provided throughout the development of this Bachelor's Thesis. Their advice and experience have been essential in helping me understand the context of the project, make the appropriate functional and technical decisions, and face the various challenges that arose during its completion.

I would also like to thank the DiversoLab team and everyone who contributed to the development of the technological environment on which this project was built for providing a framework in which to apply the knowledge I acquired during my undergraduate studies.

Finally, I dedicate this project to my family and all those close to me. Their support and trust have undoubtedly been essential to completing this project and tackling the effort required for a project of this magnitude. Grandma, wherever you are, I'm one step closer to becoming an engineer.

Mairena del Alcor, august 2026

RESUMEN

Este Trabajo de Fin de Grado aborda la evolución e integración de un generador configurable de modelos de características en el entorno UVLHub, una plataforma orientada a la gestión y manipulación de modelos escritos en Universal Variability Language (UVL). La necesidad de este componente surge de la escasez de modelos UVL expresivos y configurables que permitan evaluar herramientas de análisis de variabilidad software, especialmente aquellos que incorporan niveles avanzados del lenguaje como atributos y restricciones complejas. Partiendo de enfoques previos de generación de modelos UVL, se ha desarrollado e integrado un nuevo componente generador de modelos de características siguiendo la arquitectura ofrecida por Flamapy, separando la definición de los modelos, su configuración y las operaciones de generación. Además, se ha adaptado el módulo generador de UVLHub para delegar la lógica de generación en este nuevo componente independiente, mejorando la reutilización, mantenibilidad y capacidad de evolución del sistema. La solución ha sido implementada utilizando Flask (Python) y desplegada mediante contenedores Docker para garantizar un entorno reproducible. Como resultado, se ha conseguido desacoplar el generador de la aplicación principal manteniendo la funcionalidad existente y adaptando la infraestructura de pruebas asociada al módulo. El sistema cuenta además con una batería de 134 pruebas automatizadas superadas y pruebas de interfaz mediante Pytest y Selenium que validan el flujo completo de generación. La principal aportación del trabajo consiste en proporcionar una base más flexible y mantenible para la generación de modelos UVL destinados a la evaluación y desarrollo de herramientas de análisis de variabilidad.

Palabras clave: SPL, UVL, UVLHub, Flamapy, Modelos de Características, generación automática de modelos, Python, Docker, pruebas automatizadas.

ABSTRACT

This project addresses the development and integration of a configurable feature model generator within the UVLHub environment, a platform designed for the management and manipulation of models written in the Universal Variability Language (UVL). The need for this component arises from the scarcity of expressive and configurable UVL models that enable the evaluation of software variability analysis tools, especially those that incorporate advanced language features such as attributes and complex constraints. Building on previous approaches to UVL model generation, a new feature model generator component has been developed and integrated, following the architecture provided by Flamapy, which separates model definition, configuration, and generation operations. In addition, the UVLHub generator module has been adapted to delegate the generation logic to this new independent component, improving the system's reusability, maintainability, and scalability. The solution was implemented using Flask (Python) and deployed via Docker containers to ensure a reproducible environment. As a result, the generator has been decoupled from the main application while maintaining existing functionality and adapting the testing infrastructure associated with the module. Plus, the system includes a suite of 134 successfully passed automated tests and interface tests using Pytest and Selenium that validate the entire generation workflow. The main contribution of this work is to provide a more flexible and maintainable foundation for generating UVL models intended for the evaluation and development of variability analysis tools.

Keywords: SPL, UVL, UVLHub, Flamapy, Feature Models, automatic model generation, Python, Docker, automated testing.

ÍNDICE GENERAL

Declaración de autoría	ii
Agradecimientos	iv
Acknowledgements	v
Resumen	vii
Abstract	viii
Índice de tablas	xvi
Índice de figuras	xix
Lista de acrónimos	xxi
1. Introducción	1
1.1. Contexto	1
1.2. Problema	2
1.3. Motivación	3
1.4. Punto de partida y contribución propia	3
1.5. Objetivos	6
1.6. Alcance	9
1.7. Estructura de la memoria	10
2. Estado del arte	12
2.1. Ingeniería de Líneas de Productos Software (SPL)	12
2.1.1. Líneas de Productos Software	12
2.1.2. Modelos de características	14
2.1.3. Análisis automático y motores de razonamiento	15
2.2. Universal Variability Language (UVL)	17
2.2.1. Estructura básica de un modelo UVL	18
2.2.2. Sintaxis	18
2.2.3. Diseño del lenguaje	19
2.2.4. Niveles del lenguaje	22
2.3. Ecosistema de herramientas y aplicación en variabilidad software	27
2.3.1. Flamapy como framework de análisis de variabilidad	27
2.3.2. UVLHub como plataforma de gestión de modelos UVL	28

2.3.3. Generación automática de modelos de características	30
2.4. Trabajos relacionados	31
2.4.1. Generating Feature Models with UVL's Full Expressiveness . .	31
2.4.2. S.P.L.O.T.	32
2.4.3. FeatureIDE	32
2.4.4. BeTTy	32
2.4.5. ETHOM	33
2.4.6. Comparativa de trabajos relacionados	34
3. Planificación del proyecto	35
3.1. Acta de constitución del proyecto	36
3.1.1. Interesados del proyecto	37
3.2. Metodología	37
3.2.1. Metodología de desarrollo	37
3.2.2. Metodología de validación y evaluación	40
3.3. Estructura de descomposición del trabajo (EDT)	41
3.4. Cronograma e hitos (línea base)	42
3.5. Plan de comunicaciones y seguimiento	46
3.6. Gestión de riesgos	47
3.6.1. Seguimiento de riesgos al cierre	48
3.7. Control de cambios	51
3.8. Plan de calidad	54
3.9. Presupuesto	55
3.10. Cierre del proyecto y lecciones aprendidas	58
4. Análisis de requisitos	60
4.1. Descripción del sistema	60
4.2. Requisitos de información	61
4.3. Requisitos funcionales	68
4.4. Requisitos no funcionales	69
4.5. Reglas de negocio	70
4.6. Restricciones del proyecto	77
4.7. Casos de uso	77
4.7.1. Diagrama general de casos de uso	77
4.7.2. Diagrama general de casos de uso	77
4.7.3. Especificación de casos de uso principales	79
4.8. Matriz de trazabilidad	83
4.8.1. Requisitos funcionales $\leftrightarrow$ casos de uso	83
4.8.2. Requisitos $\leftrightarrow$ pruebas	83

5. Diseño de la solución	87
5.1. Modelo conceptual del dominio	87
5.1.1. Estructura del modelo	88
5.1.2. Restricciones del dominio	90
5.1.3. Espacio de soluciones / configuraciones	91
5.2. Arquitectura del sistema	91
5.2.1. Visión general	91
5.2.2. Diagrama de componentes	92
5.2.3. Diagrama de secuencia del flujo principal	93
5.3. Patrones de diseño aplicados	94
5.4. Diseño de la interfaz de usuario	95
5.4.1. Mapa de navegación	95
5.4.2. Wireframes y mockups	104
5.4.3. Decisiones de diseño visual	108
5.5. Esquema de base de datos	109
5.5.1. Modelo entidad-relación	109
5.5.2. Migraciones / versionado del esquema	110
5.6. Flujo de datos principal	110
5.6.1. Estructura de la representación intermedia	111
5.6.2. Mapeo entre niveles	111
5.6.3. Trazabilidad e identificadores	112
6. Implementación	113
6.1. Análisis de tecnologías	113
6.2. Tecnologías utilizadas	116
6.3. Módulo de validación / lógica principal	120
6.3.1. Validación sintáctica / comprobaciones básicas	120
6.3.2. Validación semántica y restricciones de dominio	122
6.4. Módulo de generación de artefactos	126
6.4.1. Cálculos o transformaciones específicas	126
6.4.2. Síntesis del artefacto final	129
6.4.3. Operaciones complementarias	130
6.5. API REST / capa de servicios	130
6.5.1. Endpoints implementados	130
6.5.2. Descarga / exportación de artefactos	131
6.6. Integración continua	131
7. Pruebas y validación	133
7.1. Estrategia de pruebas	133
7.2. Pruebas unitarias	133

7.3. Pruebas de integración	134
7.4. Pruebas de extremo a extremo	135
7.5. Cobertura y métricas de calidad	136
8. Resultados y discusión	139
8.1. Diseño de la evaluación	139
8.2. Resultados	139
8.2.1. Resultados sobre el objetivo O1	139
8.2.2. Resultados sobre el objetivo O2	141
8.2.3. Resultados sobre el objetivo O3	143
8.2.4. Resultados sobre el objetivo O4	144
8.2.5. Resultados sobre el objetivo O5	146
8.2.6. Resultados sobre los objetivos formativos O6–O10	147
8.3. Comparación con trabajos previos	149
8.4. Discusión	150
9. Conclusiones y trabajo futuro	151
9.1. Conclusiones	151
9.2. Limitaciones	153
9.3. Trabajo futuro	154
Bibliografía	157
A. Apéndice	160
A.1. Ejemplo de modelo de características en UVL	160
A.2. Modelos UVL generados para la evaluación de O1	161
B. Manual de instalación y despliegue	165
B.1. Requisitos previos	165
B.2. Obtención del código fuente	165
B.3. Instalación de dependencias y ejecución	166
B.4. Despliegue en producción	166
B.5. Verificación de la instalación	166
B.5.1. Ejecución de las pruebas automatizadas	167
B.5.2. Medición de cobertura	167
C. Manual de usuario	169
C.1. Acceso al sistema	169
C.2. Funcionalidad principal	170
C.3. Mensajes de error y resolución de incidencias	170
C.4. Contacto y soporte	171

ÍNDICE DE TABLAS

1.1. Procedencia de los principales elementos relacionados con el desarrollo.	5
1.2. Criterios de éxito de los objetivos y capítulos donde se verifican.	9
2.1. Comparativa entre SAT y BDD como técnicas de análisis automático de modelos de características.	17
2.2. Comparativa de trabajos relacionados con la generación automática de modelos de características.	34
3.1. Acta de constitución del proyecto.	36
3.2. Registro de interesados del proyecto.	37
3.3. Trazabilidad de los repositorios y recursos utilizados durante el desarrollo.	39
3.4. Estructura de descomposición del trabajo (EDT).	42
3.5. Cronograma de ejecución e hitos del proyecto.	43
3.6. Fases de desarrollo y esfuerzo asociado: línea base frente a valores reales.	44
3.7. Fases de redacción de la memoria (estimadas vs. reales).	45
3.8. Resumen global del proyecto (estimadas vs. reales).	46
3.9. Plan de comunicaciones y seguimiento del proyecto.	47
3.10. Riesgos identificados, exposición y planes de mitigación.	48
3.11. Seguimiento de riesgos al cierre: materialización, respuesta y residuales.	51
3.12. Registro de control de cambios sobre la línea base.	54
3.13. Costes de capital (CapEx).	56
3.14. Desglose de horas y costes de personal.	57
3.15. Costes de herramientas y servicios de desarrollo (OpEx).	57
3.16. Resumen del presupuesto del proyecto ($TCO = CapEx + OpEx$). . .	58
3.17. Lecciones aprendidas al cierre del proyecto.	59
4.1. Atributos del requisito de información RI-01.	62
4.2. Atributos del requisito de información RI-02.	63
4.3. Atributos del requisito de información RI-03.	63
4.4. Atributos del requisito de información RI-04.	64
4.5. Atributos del requisito de información RI-05.	65
4.6. Atributos del requisito de información RI-06.	66
4.7. Atributos del requisito de información RI-07.	67
4.8. Requisitos funcionales del sistema.	69
4.9. Requisitos no funcionales del sistema.	70
4.10. Reglas de negocio del dominio.	76

4.11. Restricciones del proyecto.	77
4.12. Especificación del caso de uso CU-01.	79
4.13. Especificación del caso de uso CU-02.	80
4.14. Especificación del caso de uso CU-03.	81
4.15. Especificación del caso de uso CU-04.	82
4.16. Especificación del caso de uso CU-05.	83
4.17. Matriz de trazabilidad entre requisitos funcionales y casos de uso. . .	83
4.18. Matriz de trazabilidad entre requisitos funcionales y no funcionales y sus verificaciones.	85
6.1. Bloques de configuración de <code>FmgeneratorModel</code>	123
7.1. Cobertura de líneas del código de producción del generador.	136
7.2. Resumen de la suite automatizada de pruebas.	137
8.1. Pruebas técnicas del efecto de los parámetros de configuración. . . .	142
8.2. Pruebas técnicas de integración con Flamapy y UVLHub.	144
8.3. Resultados de las diez repeticiones de la evaluación temporal. . . .	146
8.4. Evidencias asociadas a los objetivos formativos. Elaboración propia. .	148
9.1. Grado de cumplimiento de los objetivos y evidencia asociada. Elabo- ración propia.	152

ÍNDICE DE FIGURAS

2.1. Representación de una línea de productos software basada en activos comunes. Extraído de [9].	13
2.2. Ejemplo de modelo de características. Extraído de [13].	14
2.3. Ejemplo de modelo de características. Elaboración propia.	15
2.4. Funcionamiento de cada motor de razonamiento. Elaboración propia.	16
2.5. Arquitectura general de UVLHub y sus integraciones principales. Extraído de [15]	29
2.6. Arquitectura general de UVLHub y sus integraciones principales. Extraído de [15]	30
3.1. Arquitectura general de UVLHub y sus integraciones principales. Elaboración propia.	55
4.1. Diagrama general de casos de uso del sistema. Elaboración propia	78
5.1. Modelado conceptual del sistema. Elaboración propia	88
5.2. Diagrama de componentes del sistema. Elaboración propia	93
5.3. Interfaz del Step 1: Batch. Elaboración propia	96
5.4. Interfaz del Step 2: Language levels. Elaboración propia	97
5.5. Interfaz del Step 3: Feature tree. Elaboración propia	98
5.6. Interfaz del Step 3: Feature tree. Elaboración propia	99
5.7. Interfaz del Step 4: Constraints. Elaboración propia	100
5.8. Interfaz del Step 4: Constraints. Elaboración propia	100
5.9. Interfaz del Step 5: Attributes. Elaboración propia	101
5.10. Interfaz del Step 5: Attributes. Elaboración propia	102
5.11. Interfaz del Step 6: Output. Elaboración propia	103
5.12. Interfaz del Step 6: Output. Elaboración propia	103
5.13. Interfaz del Step 6: Output. Elaboración propia	104
5.14. Mapa de navegación del generador UVL. Elaboración propia	104
5.15. Wireframe inicial del Step 1 del generador. Elaboración propia	105
5.16. Wireframe inicial del Step 2 del generador. Elaboración propia	106
5.17. Wireframe inicial del Step 3 del generador. Elaboración propia	107
5.18. Wireframe inicial del Step 4 del generador. Elaboración propia	107
5.19. Wireframe inicial del Step 5 del generador. Elaboración propia	108
5.20. Componente selector de distribuciones. Elaboración propia	109
5.21. Flujo de datos principal. Elaboración propia	110

5.22. Ejemplo de uso de la <i>session</i> . Elaboración propia	111
6.1. Estructura del módulo generator de UVLHub. Elaboración propia . .	116
6.2. Estructura de la herramienta generadora basada en Flamapy. Elabo- ración propia	117
6.3. Plantillas HTML utilizadas para los pasos del <i>wizard</i> . Elaboración propia	118
6.4. Scripts y dependencias utilizadas para la ejecución con Pyodide. Ela- boración propia	118
6.5. Configuración de Docker en UVLHub. Elaboración propia	119
6.6. Suite de pruebas del módulo generator. Elaboración propia	119
8.1. Configuración usada para verificar O1. Elaboración propia	140
8.2. Configuración utilizada para la evaluación temporal. Elaboración propia.	145
8.3. Resultado de la ejecución de la suite de pruebas del módulo <i>generator</i> . Elaboración propia	147
8.4. Resultado de la ejecución de la suite de pruebas del <i>fm_generator</i> . Elaboración propia	147
B.1. Salida de consola tras el arranque correcto del contenedor web. Elabo- ración propia	167
C.1. Pantalla inicial de UVLHub. Elaboración propia	169

LISTA DE ACRÓNIMOS

SPL	Software Product Lines
UVL	Universal Variability Language
FLOSS	Free/Libre Open Source Software
TASOVA	Red en nuevas Tendencias en Arquitectura Software y Variabilidad
SAT	Boolean Satisfiability Problem
BDD	Binary Decision Diagrams
DOI	Digital Object Identifier

1 INTRODUCCIÓN

1.1 Contexto

El desarrollo de software actual debe dar respuesta a sistemas cada vez más configurables, capaces de adaptarse a distintas necesidades, plataformas y contextos de uso. Dentro de este escenario, las Líneas de Productos Software (Software Product Lines, SPL) [1] ofrecen un enfoque para gestionar familias de productos relacionados a partir de los elementos que comparten y de aquellos que pueden variar entre unas configuraciones y otras. Esto permite tratar la variabilidad de forma explícita y facilita tareas como la configuración de productos, la reutilización de componentes o el análisis automático de las distintas combinaciones posibles.

Una de las formas más extendidas de representar esta variabilidad son los modelos de características (Feature Models) [2]. Estos modelos describen las características que puede incluir una familia de productos y las relaciones que existen entre ellas, permitiendo establecer qué elementos son obligatorios u opcionales, cuáles pueden combinarse y qué restricciones deben respetarse para obtener una configuración válida.

A medida que los modelos han ganado en complejidad, también ha surgido la necesidad de disponer de formatos que permitan representarlos de manera común. Universal Variability Language (UVL) [3] nace con este propósito y proporciona un lenguaje para describir modelos de variabilidad con diferentes grados de expresividad. Además de las relaciones habituales entre características, UVL permite incorporar construcciones más avanzadas, como atributos y restricciones, ampliando así los escenarios que pueden representarse y analizarse.

En torno a estos modelos se han desarrollado distintas herramientas destinadas a facilitar su procesamiento y análisis. Entre ellas se encuentra Flamapy [4], un framework desarrollado en Python que proporciona soporte para el análisis automático de modelos de características. Su arquitectura extensible permite incorporar nuevos formatos, operaciones y técnicas de análisis sin necesidad de modificar el núcleo de la herramienta, lo que facilita su evolución y su uso en diferentes líneas de investigación.

El presente Trabajo Fin de Grado se desarrolla dentro de este ecosistema y se enmarca en la actividad investigadora de DiversoLab y en TASOVA Plus (Red en nuevas Tendencias en Arquitectura Software y Variabilidad) [5]. Este entorno promueve el desarrollo de herramientas relacionadas con la ingeniería de líneas de productos software, la arquitectura software y el análisis de variabilidad, siguiendo además principios asociados al software libre y a la ciencia abierta (Open Science).

Otro de los componentes de este ecosistema es UVLHub [6], una plataforma orientada a la publicación, gestión y exploración de modelos expresados en UVL. La plataforma ofrece un espacio en el que investigadores y desarrolladores pueden compartir modelos y reutilizarlos en distintos experimentos o herramientas. Flamapy, UVL y UVLHub conforman así buena parte del entorno tecnológico sobre el que se desarrolla este trabajo.

1.2 Problema

La evolución de los lenguajes de modelado y de los motores de razonamiento ha permitido trabajar con formas de variabilidad cada vez más complejas. Sin embargo, disponer de herramientas capaces de procesar esa complejidad no resulta suficiente por sí solo: para poder evaluarlas también es necesario contar con modelos que hagan uso de esas capacidades.

Buena parte de los modelos de características disponibles se concentra en construcciones relativamente básicas. Estos modelos siguen siendo útiles para numerosos análisis, pero resultan menos adecuados cuando se pretende experimentar con elementos más avanzados del lenguaje o estudiar cómo se comportan los motores de razonamiento ante distintos grados de complejidad. Se produce así una diferencia entre la expresividad que pueden ofrecer los lenguajes y herramientas actuales y la variedad de modelos disponibles para ponerlos a prueba.

Es por ello que se identifican de forma explícita las siguientes limitaciones:

1. **Expresividad limitada de los modelos de características disponibles:** Muchos de estos modelos se basan principalmente en relaciones estructurales básicas entre características. Aunque estos modelos permiten realizar análisis fundamentales, no siempre permiten representar escenarios más complejos donde intervienen elementos avanzados del lenguaje, lo que reduce la capacidad de explorar nuevas técnicas de razonamiento y análisis.
2. **Complicaciones a la hora de evaluar y mejorar motores de razonamiento avanzados sobre modelos más complejos:** Esto es consecuencia directa del problema anteriormente descrito. Sin estos modelos más avanzados, la capacidad de validar nuevas aproximaciones se ve limitada, por lo que es más complicado estudiar hasta qué punto estos motores pueden aprovechar construcciones más avanzadas del lenguaje de modelado.
3. **Necesidad de ampliar el conjunto de modelos disponibles para investigación en variabilidad software** La investigación en este ámbito requiere disponer de modelos que permitan experimentar con diferentes escenarios, niveles de complejidad y características del lenguaje. Sin una base amplia y diversa

de modelos, resulta mucho más difícil comparar técnicas, evaluar resultados y avanzar en el desarrollo de herramientas más potentes.

1.3 Motivación

La disponibilidad de modelos de características con distintos niveles de expresividad resulta fundamental para continuar avanzando en el desarrollo y evaluación de técnicas de análisis automático de variabilidad software. Como se ha comentado anteriormente, la evolución de los lenguajes de modelado permite representar escenarios cada vez más complejos, pero esta capacidad debe estar acompañada de modelos que permitan estudiar y validar el comportamiento de las herramientas encargadas de procesarlos.

Abordar esta necesidad dentro de un entorno de investigación permite trabajar sobre problemas que aparecen de forma directa durante el desarrollo de nuevas técnicas y herramientas. En este sentido, el contexto proporcionado por TASOVA Plus y DiversoLab ofrece un marco adecuado para contribuir al avance de soluciones relacionadas con la ingeniería de líneas de productos software, la arquitectura software y el análisis de variabilidad, lo que además permite seguir principios asociados al software libre y la reutilización del conocimiento científico.

Desde una perspectiva académica y profesional, este Trabajo Fin de Grado representa también una oportunidad para aplicar los conocimientos adquiridos durante el grado en un proyecto software real. Durante su desarrollo, he tenido la oportunidad de trabajar dentro de un ecosistema ya existente, enfrentándome a aspectos habituales en la ingeniería del software, como la comprensión de código desarrollado previamente, la integración de nuevas funcionalidades, la documentación o la validación mediante pruebas automatizadas.

Además, la participación en un proyecto vinculado a investigación me ha permitido adquirir experiencia en un contexto diferente al de los desarrollos software tradicionales. La necesidad de analizar información técnica, consultar documentación especializada y tomar decisiones fundamentadas ha favorecido el desarrollo de habilidades relacionadas con la búsqueda autónoma de conocimiento y la resolución de problemas abiertos.

1.4 Punto de partida y contribución propia

Este Trabajo Fin de Grado se desarrolla sobre un conjunto de componentes y avances previos que forman parte del ecosistema Flamapy-UVLHub. Por este motivo, resulta necesario delimitar qué elementos constituyen la base sobre la que se construye

el trabajo y cuáles corresponden a las contribuciones realizadas durante el desarrollo del presente proyecto.

En primer lugar, el diseño conceptual del generador de modelos de características no surge desde cero en este trabajo. La propuesta de generación de modelos UVL con diferentes niveles de expresividad está basada en el enfoque descrito por Sundermann et al. [7], donde se presenta un generador capaz de producir modelos de características aprovechando las construcciones avanzadas del lenguaje UVL. Este trabajo previo establece las bases conceptuales sobre las que se desarrolla la implementación realizada en este TFG.

Además, el proyecto parte de la infraestructura existente del ecosistema UVLHub y Flamapy. UVLHub ya proporcionaba una plataforma para la gestión y manipulación de modelos UVL, así como un módulo *generator* inicial sobre el que se realiza la integración de la nueva funcionalidad.

La siguiente tabla resume la procedencia de los principales elementos relacionados con el desarrollo:

Elemento	Procedencia
Diseño conceptual del Basado en el trabajo de Sundermann et al. [7], generador de modelos donde se define el enfoque general de generación UVL con expresividad de modelos de características con soporte para las avanzada	construcciones avanzadas del lenguaje UVL.
Ecosistema Flamapy y Infraestructura previa sobre la que se desarrolla el plataforma UVLHub	trabajo. En Flamapy participan diferentes grupos de investigación, entre ellos los pertenecientes a las Universidades de Sevilla, Málaga, Graz y la UNED. En cuanto a UVLHub, ha sido desarrollado principalmente por DiversoLab.
Generador de modelos Implementación desarrollada principalmente en es- de características co- te Trabajo Fin de Grado a partir del enfoque como herramienta com- ceptual diseñado por Sundermann et al [7]. Incluye patible con Flamapy	la definición del modelo de configuración, la generación de estructuras jerárquicas, atributos, restricciones y la integración con las operaciones de Flamapy.
Integración del genera- Desarrollo realizado durante este TFG, adaptando dor dentro del módulo la aplicación existente para utilizar el nuevo com- <i>generator</i> de UVLHub	ponente generador independiente.
Nueva interfaz del <i>wi- Aportación realizada por David Romero dentro zard</i> del generador	del módulo <i>generator</i> de UVLHub. Incluye la ampliación del <i>wizard</i> de cinco a seis pasos, el panel lateral de seguimiento de la configuración actual y el componente <i>slider</i> para facilitar la definición de distribuciones.
Refactorización de de- Trabajo realizado por David Romero dentro del terminados elementos proyecto global, incluyendo la reorganización de de UVLHub	algunos ficheros del módulo generador como <i>routes.py</i> , <i>wizard.py</i> y otros elementos relacionados. Posteriormente, durante este TFG, se han realizado nuevas modificaciones arquitectónicas necesarias para integrar el generador.
Batería de pruebas uni- Las pruebas relacionados con <i>test_engine.py</i> , tarias del módulo gene- <i>test_validators.py</i> , <i>test_wizard_outputs.py</i> y rador <i>test_wizard_flow.py</i> fueron desarrollados en su mayoría por David Romero. Durante el desarrollo del TFG algunas de estas pruebas han sido adaptados para ajustarse a la nueva arquitectura del generador.	
Generador basado en	Desarrollado por David Romero dentro del ecosis-

Por tanto, la contribución principal de este Trabajo Fin de Grado se centra en la implementación del generador configurable de modelos de características como componente independiente dentro de Flamapy y en su integración dentro de UVLHub. En concreto, las principales aportaciones realizadas durante este trabajo son:

- La implementación de la lógica de generación de modelos de características siguiendo el enfoque conceptual definido en trabajos previos, incorporando soporte para diferentes niveles de expresividad del lenguaje UVL.
- La creación de una arquitectura modular compatible con Flamapy, separando la configuración del generador, la representación del modelo y las operaciones necesarias para producir nuevos modelos.
- La implementación del sistema de configuración asociado al generador, permitiendo definir parámetros relacionados con características, jerarquía, atributos y restricciones.
- La integración del nuevo componente dentro del módulo *generator* de UVLHub y la adaptación de la infraestructura existente para trabajar con esta nueva arquitectura.
- La validación del funcionamiento del sistema mediante pruebas automatizadas adaptadas a la implementación.

1.5 Objetivos

El objetivo principal de este trabajo es dar apoyo a la mejora de los motores de razonamiento existentes para el análisis de modelos de características mediante la obtención de una cantidad significativa y variada de modelos con expresividad avanzada. Como propuesta para conseguirlo, se pretende desarrollar una herramienta de generación automática de modelos de características aleatoria y completamente configurable, compatible con el ecosistema Flamapy, capaz de producir modelos UVL con diferentes niveles de expresividad [7].

De forma más específica, podemos desgranar los objetivos de este trabajo de la siguiente forma:

- O1.** Desarrollar un generador automático de modelos de características capaz de producir modelos UVL con diferentes niveles de complejidad y expresividad, ampliando la variedad de modelos disponibles para el análisis de variabilidad.
- O2.** Desarrollar un sistema de configuración flexible que permita modificar los parámetros principales del proceso de generación y adaptar los modelos obtenidos a distintos escenarios.

- O3.** Desarrollar una herramienta compatible con el ecosistema Flamapy e integrarla en UVLHub como solución para la generación configurable de modelos de características.
- O4.** Evaluar el comportamiento temporal del generador bajo escenarios de generación con un volumen elevado de modelos, analizando el coste asociado a la inicialización del entorno y al proceso de generación.
- O5.** Validar el funcionamiento del generador mediante una batería de pruebas automatizadas que permita comprobar la corrección de los modelos producidos y detectar posibles errores durante el proceso de generación.

De forma complementaria, el trabajo persigue una serie de objetivos formativos relacionados con la aplicación de conocimientos técnicos y metodológicos durante el desarrollo. Su cumplimiento se evaluará de forma cualitativa a partir de las evidencias documentadas en la memoria, incluyendo las decisiones de diseño, la integración realizada, las pruebas desarrolladas y la planificación seguida.

- O6.** Reutilizar el conocimiento científico en el desarrollo de herramientas software (objetivo heredado de la red TASOVA Plus)
- O7.** Adquirir experiencia en el desarrollo y mantenimiento de proyectos software reales mediante la participación en un entorno de investigación y código abierto.
- O8.** Profundizar en el uso de tecnologías empleadas en el proyecto, especialmente Python, Flask y Docker, así como en la aplicación de pruebas automatizadas durante el desarrollo de un proyecto software integrado.
- O9.** Mejorar la capacidad de búsqueda, análisis y selección de información técnica relevante para resolver problemas dentro de un contexto de investigación.
- O10.** Aplicar buenas prácticas de planificación, gestión y documentación durante el desarrollo individual de un proyecto software de mayor duración y complejidad.

Obj.	Criterio de éxito (medible)	Cómo se verifica	Cap.
O1	El generador permite producir modelos UVL automáticamente incorporando diferentes niveles de complejidad y expresividad del lenguaje, ampliando la variedad de modelos disponibles para el análisis de variabilidad.	Generación de modelos con configuraciones variadas y validación de que los modelos producidos cumplen la sintaxis y estructura esperada de UVL.	6, 7
O2	El generador dispone de un sistema de configuración que permite modificar los parámetros principales del proceso de generación y adaptar los modelos obtenidos a distintos escenarios.	Revisión de las opciones de configuración disponibles y ejecución de pruebas con diferentes configuraciones para comprobar su efecto sobre los modelos generados.	6, 7
O3	La solución desarrollada proporciona una herramienta compatible con el ecosistema Flamapy y puede ser utilizada desde UVLHub como herramienta de generación de modelos de características.	Verificación de la compatibilidad con los componentes empleados de Flamapy y comprobación del flujo completo de generación desde UVLHub.	6, 7, 8
O4	Evaluar el comportamiento temporal del generador en escenarios de generación con un volumen elevado de modelos de características.	Medición del tiempo de inicialización del generador y del tiempo requerido para generar un conjunto elevado de modelos UVL bajo una configuración representativa, analizando el coste asociado a cada fase del proceso.	8
O5	El generador cuenta con una batería de pruebas automatizadas que permita validar su correcto funcionamiento y detectar errores en los modelos producidos.	Ejecución de la suite de pruebas automatizadas y análisis de los resultados obtenidos.	7

Obj.	Criterio de éxito (medible)	Cómo se verifica	Cap.
O6	El desarrollo reutiliza conocimiento científico relacionado con la generación de modelos de características y lo aplica en una herramienta software concreta.	Análisis del trabajo de Sundermann et al. y trazabilidad de su enfoque conceptual en el diseño e implementación del generador.	1, 2, 5, 6
O7	El desarrollo proporciona experiencia en la evolución de un proyecto software real, integrado en un ecosistema de investigación y código abierto.	Análisis de la integración del componente en Flamapy y UVLHub, del uso de Docker y de las pruebas ejecutadas sobre ambos proyectos.	3, 5, 6, 7, 8
O8	El desarrollo aplica Python, Flask, Docker y pruebas automatizadas en la construcción e integración del generador.	Revisión de la implementación del componente, de su integración en UVLHub y de la ejecución de las suites de pruebas.	5, 6, 7
O9	Las decisiones técnicas del proyecto se apoyan en la búsqueda, análisis y selección de información técnica y científica relevante.	Revisión del estado del arte, de las fuentes consultadas y de las decisiones justificadas en el diseño e implementación.	2, 5, 6
O10	El desarrollo individual del proyecto aplica prácticas de planificación, gestión de riesgos y documentación técnica.	Revisión de la planificación, de la gestión de riesgos, de la documentación elaborada y de la reflexión final sobre el desarrollo.	3, 9

Tabla 1.2: Criterios de éxito de los objetivos y capítulos donde se verifican.

1.6 Alcance

Este TFG comprende el desarrollo de un generador automático de modelos de características integrado dentro del ecosistema de herramientas asociado a Flamapy y orientado principalmente a su utilización desde UVLHub. La solución desarrollada

permite generar modelos expresados en UVL mediante una configuración definida por el usuario, incorporando diferentes niveles de complejidad y expresividad con el objetivo de ampliar la disponibilidad de modelos para tareas de análisis e investigación en variabilidad software.

Dentro del alcance del proyecto se incluye el diseño e implementación del generador como herramienta compatible con Flamapy, siguiendo la arquitectura extensible del framework. Asimismo, se contempla su integración dentro del módulo *generator* de UVLHub, permitiendo que los usuarios de la plataforma puedan interactuar con la funcionalidad desarrollada mediante una interfaz adaptada al flujo existente de la aplicación. Esta integración incluye la definición de los mecanismos necesarios para configurar el proceso de generación, la incorporación de soporte para diferentes construcciones del lenguaje UVL y la validación del funcionamiento del generador mediante pruebas automatizadas destinadas a comprobar la corrección de los modelos producidos.

En cuanto a lo que queda **fuera** del alcance del proyecto, se consideran los siguientes aspectos:

- La creación de una aplicación web independiente para el generador. Aunque la integración realizada en UVLHub incorpora una interfaz que permite configurar y ejecutar el proceso de generación, el generador como componente reutilizable no incluye una interfaz propia. En caso de utilizarse fuera de UVLHub, sería necesario desarrollar una capa de interacción adaptada al contexto concreto de uso.
- La modificación o ampliación de otros módulos de UVLHub distintos al módulo *generator*, ya que la intervención realizada se limita a la incorporación del generador dentro del módulo correspondiente.
- El desarrollo de nuevos motores de razonamiento o algoritmos de análisis de variabilidad. El objetivo del generador es proporcionar modelos con diferentes características y niveles de expresividad que puedan servir como base experimental para este tipo de herramientas, pero esto es algo que se pretende conseguir indirectamente con su uso.

1.7 Estructura de la memoria

El resto de la memoria se organiza como sigue:

- El **Capítulo 2** presenta el **estado del arte** relacionado con la ingeniería de líneas de productos software, los modelos de características, el lenguaje UVL y las herramientas que forman parte del ecosistema en el que se desarrolla el

trabajo. Además, se revisan trabajos relacionados con el objetivo de identificar las necesidades que justifican la propuesta desarrollada.

- El **Capítulo 3** describe la **planificación y gestión del proyecto**, incluyendo la metodología seguida durante el desarrollo, la organización de las tareas, la gestión de riesgos y los aspectos relacionados con la evolución del trabajo a lo largo del tiempo.
- El **Capítulo 4** define los **requisitos** que debe cumplir la solución desarrollada. En este capítulo se establecen las funcionalidades esperadas del generador, sus restricciones y la relación existente entre los requisitos y las pruebas destinadas a validarlos.
- El **Capítulo 5** presenta el **diseño de la solución** propuesta, describiendo la arquitectura general del sistema, la organización de sus componentes y las decisiones tomadas para integrar el generador dentro del ecosistema de Flamapy y UVLHub.
- El **Capítulo 6** detalla la **implementación** de la solución, incluyendo las tecnologías utilizadas, la estructura de la herramienta desarrollada, los mecanismos empleados para la generación de modelos y la integración realizada con la plataforma UVLHub.
- El **Capítulo 7** describe la estrategia de **pruebas** aplicada para verificar el correcto funcionamiento del sistema. En él se presentan las pruebas unitarias, de integración y de extremo a extremo utilizadas para validar la generación de modelos y la integración de la funcionalidad desarrollada.
- El **Capítulo 8** analiza los **resultados** obtenidos durante la evaluación del trabajo. Se contrastan los resultados con los criterios de éxito definidos para los objetivos y se discuten las aportaciones realizadas frente al contexto presentado en capítulos anteriores.
- El **Capítulo 9** recoge las **conclusiones** del trabajo, valorando el grado de cumplimiento de los objetivos planteados, las limitaciones encontradas y las posibles líneas de trabajo futuro derivadas de la solución desarrollada.

Definidos el contexto, el problema, los objetivos y el alcance del trabajo, el siguiente capítulo establece los fundamentos teóricos y tecnológicos necesarios para comprender el entorno de variabilidad software sobre el que se desarrolla la propuesta.

2 ESTADO DEL ARTE

2.1 Ingeniería de Líneas de Productos Software (SPL)

2.1.1 Líneas de Productos Software

A lo largo de la historia, el desarrollo de software se ha basado tradicionalmente en la construcción de sistemas individuales orientados a resolver necesidades concretas, donde cada nuevo producto software suele desarrollarse como un proyecto independiente, incluso cuando comparte una parte significativa de sus funcionalidades, arquitectura o componentes con otros sistemas existentes. Esta estrategia puede resultar adecuada cuando el número de productos es reducido; sin embargo, hoy en día esto presenta limitaciones importantes para la gran mayoría de organizaciones, ya que surge de forma paralela la necesidad de adaptar un determinado sistema base a una gran cantidad de clientes con problemas ligeros o enormemente diferentes.

Debido a este hecho surge la Ingeniería de Líneas de Productos Software, definidas por Clements y Northrop como "*a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way*"[8]. Gracias a ellas es posible identificar, gestionar y reutilizar aquellos elementos comunes entre diferentes productos, a la vez que se mantienen explícitamente aquellos aspectos que pueden variar entre las distintas configuraciones posibles, haciendo así factible y mucho más viable la gestión de dichos elementos en lugar de construir cada producto desde cero.

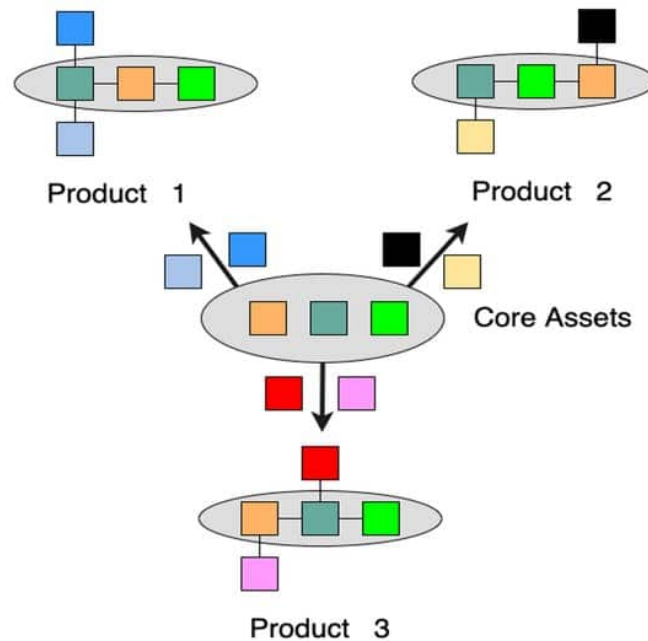


Figura 2.1: Representación de una línea de productos software basada en activos comunes. Extraído de [9].

La existencia de elementos comunes y diferencias entre los productos introduce el concepto de variabilidad, definido por van Gurp, Bosch y Svahnberg como «Variability is the ability to change or customize a system» [10]. Esta debe ser identificada, representada y gestionada durante el proceso de desarrollo, ya que permite definir qué elementos pueden formar parte de cada producto y bajo qué condiciones pueden combinarse.

La gestión de esta variabilidad requiere distinguir entre dos actividades complementarias. Por una parte, la ingeniería de dominio (*domain engineering*) se encarga de analizar un determinado dominio, identificar sus elementos comunes y variables, y construir los activos reutilizables que formarán la base de la línea de productos. Por otra parte, la ingeniería de aplicación (*application engineering*) utiliza dichos activos para derivar productos concretos mediante la selección de las características necesarias y la resolución de las variabilidades definidas.

No obstante, este paradigma introduce consigo retos relacionados con la representación y gestión de la complejidad. A medida que aumenta el número de características variables y configuraciones posibles, resulta necesario disponer de mecanismos que permitan gestionar esta variabilidad de forma íntegra y precisa, lo cual da surgimiento a técnicas como los modelos de características.

2.1.2 Modelos de características

Los modelos de características (*Feature Models*) son considerados uno de los mecanismos más utilizados para representar la variabilidad dentro de las líneas de productos software. Originalmente fueron introducidos como parte del método FODA (*Feature-Oriented Domain Analysis*) por Kang et al., con el objetivo de describir las características comunes y variables existentes dentro de una familia de sistemas software [11]. Benavides et al. los describen de la siguiente forma: "*a feature model represents the information of all possible products of a software product line in terms of features and relationships among them*"[12]. Estos modelos permiten representar el espacio de configuraciones de una línea de productos mediante las características que pueden formar parte de cada producto y las relaciones existentes entre ellas.

Para entenderlo mejor, estos modelos se pueden representar mediante una estructura jerárquica en forma de árbol, donde cada nodo representa una característica del sistema. Las relaciones existentes entre características permiten establecer dependencias entre ellas, indicando qué elementos son obligatorios, opcionales o alternativos dentro de una configuración concreta. Además, pueden incorporar restricciones adicionales que expresan condiciones que no pueden representarse únicamente mediante la jerarquía del modelo.

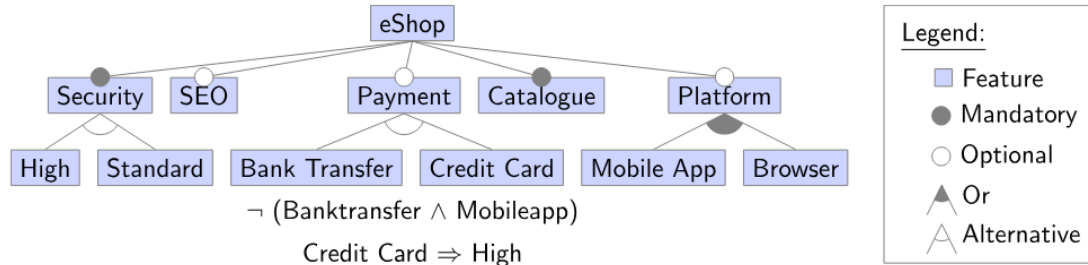


Figura 2.2: Ejemplo de modelo de características. Extraído de [13].

Aunque los modelos de características tradicionales permiten representar una gran cantidad de escenarios de variabilidad, inicialmente estaban orientados principalmente a expresar relaciones estructurales entre características. Con el aumento de la complejidad de los sistemas software surgió la necesidad de incorporar mecanismos adicionales, como atributos, restricciones aritméticas o diferentes tipos de dominios, que permitan representar escenarios más cercanos a los sistemas reales. A su vez, al incrementar la expresividad de los modelos, también se incrementa la complejidad de los mismos, lo cual hace necesaria la existencia de herramientas capaces de procesar estos modelos automáticamente y verificar propiedades sobre las configuraciones posibles.

2.1.3 Análisis automático y motores de razonamiento

Una vez definidos los modelos de características como mecanismo para representar la variabilidad de una SPL, surge la necesidad de disponer de técnicas capaces de analizar automáticamente la información contenida en dichos modelos. El análisis automático permite obtener información sobre las configuraciones representadas, comprobar propiedades del modelo y realizar operaciones de razonamiento sin necesidad de explorar manualmente cada una de las posibles variantes.

Los motores de razonamiento (*reasoning engine*) se encargan de realizar este análisis. Procesan los modelos de variabilidad y realizan operaciones como la comprobación de satisfacibilidad, la detección de inconsistencias o la identificación de características que no pueden formar parte de ninguna configuración válida. Por ejemplo, en un restaurante un cliente puede elegir en su primer plato Pasta o Arroz (obligatoriamente uno de ellos), y opcionalmente Salsa. No se puede establecer como restricción que se deba elegir Pasta y Arroz a la vez, ya que no tendría sentido si se ve desde un punto de vista natural. Los motores de razonamiento se encargan de comprobar que esto no ocurra, y lo categorizarían como no satisfacible.

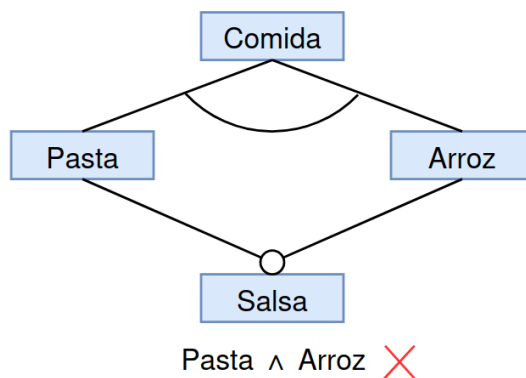


Figura 2.3: Ejemplo de modelo de características. Elaboración propia.

Una de las aproximaciones más utilizadas para realizar análisis automático sobre modelos de características consiste en transformar dichos modelos en fórmulas proposicionales que puedan ser procesadas mediante solucionadores SAT (*Boolean Satisfiability Problem*). El problema SAT consiste en determinar si existe una asignación de valores para un conjunto de variables booleanas que haga verdadera una fórmula lógica dada. En el contexto de las líneas de productos software, las características y restricciones de un modelo pueden representarse mediante variables booleanas y expresiones lógicas equivalentes, permitiendo utilizar solucionadores SAT para responder consultas relacionadas con la validez de configuraciones o la existencia de productos derivados [12].

Para aplicar esta técnica, el modelo se traduce normalmente a una fórmula lógica equivalente que conserva las restricciones definidas entre sus elementos. Por ejemplo, una relación obligatoria entre dos características puede expresarse mediante una implicación lógica, mientras que las restricciones de exclusión entre características pueden representarse mediante cláusulas que impidan determinadas combinaciones. Una vez realizada esta transformación, el solucionador SAT puede determinar si existe una configuración válida o identificar aquellas asignaciones que cumplen las restricciones del modelo.

Otra técnica ampliamente utilizada en el análisis automático de modelos de características son los Diagramas de Decisión Binarios (*Binary Decision Diagrams*, BDD). Introducidos originalmente por Bryant como una estructura eficiente para representar funciones booleanas, los BDD permiten representar dichas funciones mediante grafos dirigidos acíclicos cuyos nodos internos corresponden a variables de decisión y cuyos nodos terminales representan los valores posibles de la función [14]. Así es posible modelar de forma compacta y comprensible el espacio de configuraciones válidas de un modelo de características.

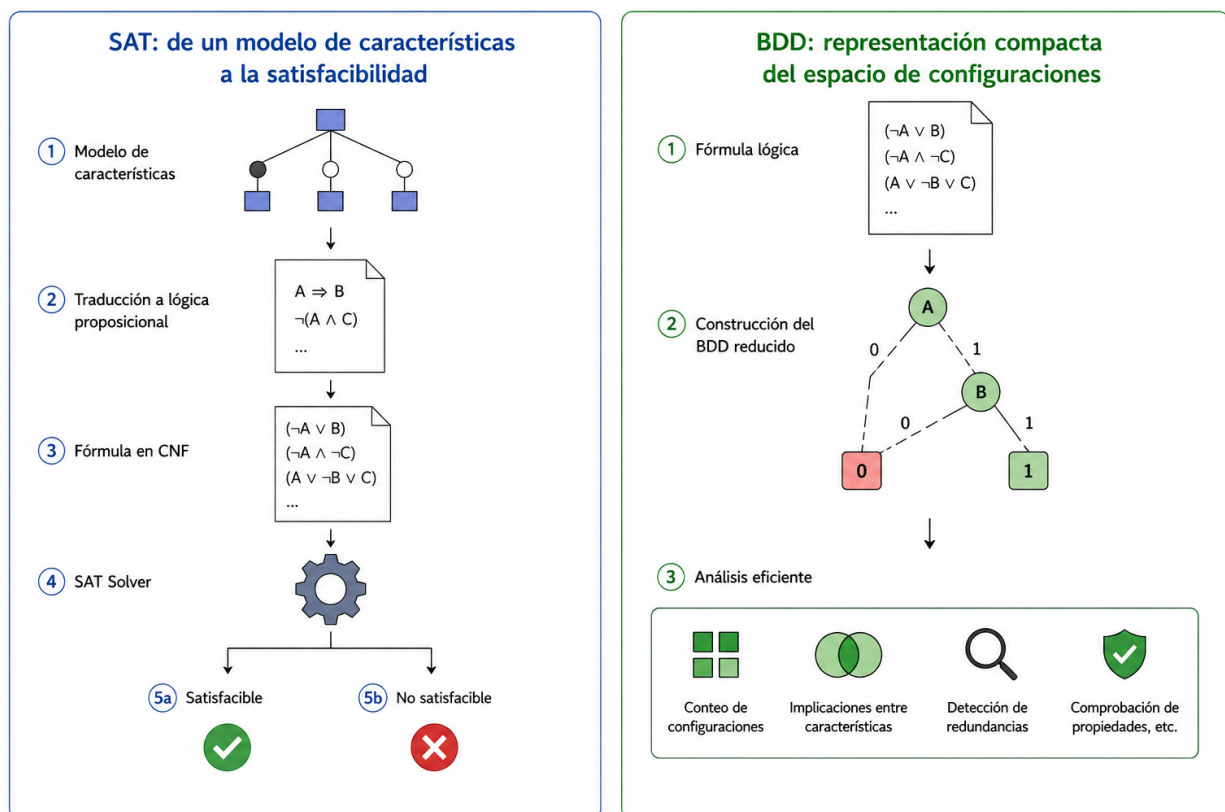


Figura 2.4: Funcionamiento de cada motor de razonamiento. Elaboración propia.

La siguiente tabla muestra una comparativa que describe los aspectos principales de ambos *solvers*:

Criterio	SAT	BDD
Representación del problema	Transformación del modelo de características a una fórmula lógica que debe ser evaluada por un solucionador SAT.	Construcción de una estructura gráfica que representa las posibles asignaciones de la función booleana.
Tipo de análisis	Orientado principalmente a responder consultas concretas sobre la existencia de configuraciones válidas.	Permite realizar análisis globales sobre el espacio completo de configuraciones representado.
Resultado obtenido	Determina si existe una asignación que satisface las restricciones y, en algunos casos, proporciona una configuración válida.	Proporciona una representación compacta del conjunto de configuraciones válidas.
Ventaja principal	Gran eficiencia para comprobar propiedades concretas del modelo y resolver problemas de satisfacibilidad.	Permite realizar múltiples operaciones de análisis sobre el modelo una vez construido el BDD.
Limitación principal	Requiere resolver consultas individuales y no mantiene necesariamente una representación completa del espacio de configuraciones.	La construcción del diagrama puede resultar costosa para modelos con elevada complejidad.

Tabla 2.1: Comparativa entre SAT y BDD como técnicas de análisis automático de modelos de características.

2.2 Universal Variability Language (UVL)

A continuación, se ofrecerán unas nociones básicas sobre UVL [3], un lenguaje diseñado para proporcionar una forma común de representar modelos de características y facilitar su procesamiento dentro del ecosistema de ingeniería de líneas de productos software. Entender la estructura y funcionamiento de este lenguaje es esencial, ya que supone la base teórica por la que funciona la solución propuesta.

El modelo completo utilizado como ejemplo se incluye en el Apéndice A.1.

2.2.1 Estructura básica de un modelo UVL

Un archivo UVL está compuesto principalmente por dos bloques:

- **features**: define la estructura jerárquica de las características del modelo, constituyendo el núcleo principal del documento.
- **constraints**: define las restricciones lógicas existentes entre características que no pueden expresarse mediante la estructura definida en **features**. Estas restricciones deben cumplirse para garantizar la consistencia del modelo descrito, ya sea debido a reglas de negocio del producto o a otros requisitos establecidos.

Además, UVL permite incorporar dos bloques opcionales en aquellos casos donde sea necesario:

- **include**: permite importar extensiones adicionales, como los *minor levels*.
- **imports**: permite importar submodelos de características situados en el mismo directorio, facilitando una organización más modular y limpia de los modelos y sus características.

Estos dos últimos bloques deben situarse al principio del archivo UVL.

2.2.2 Sintaxis

2.2.2.1. Imports

El mecanismo de importación de UVL permite dividir un modelo de características de gran tamaño en submodelos más pequeños y manejables. Mediante la palabra reservada **imports**, es posible incorporar otros archivos UVL, facilitando la organización de los modelos y favoreciendo su reutilización. Además, cada submodelo importado puede asociarse a un alias mediante la palabra reservada **as**, permitiendo acceder a sus características mediante la notación **alias.FeatureName**.

Los elementos importados pueden utilizarse tanto dentro de la estructura jerárquica de características como en la definición de restricciones, permitiendo mantener los modelos más limpios, modulares y facilitando su mantenimiento y evolución.

Por ejemplo:

```
1 imports
2     platform as pl
3     security
4
5 features
6     eShop
```

7
8
9
10

```

    mandatory
        security.Security
    optional
        pl.Platform

```

Código 2.1: Ejemplo de importación de submodelos en UVL.

En este caso, los modelos **platform** y **security** representan submodelos definidos en archivos UVL independientes situados en el mismo directorio.

2.2.2.2. Feature tree

En UVL, el árbol de características está compuesto por características y grupos de características. Cada modelo debe tener exactamente una característica raíz, a partir de la cual se define la estructura jerárquica del resto de elementos. Cada característica puede contener uno o varios grupos de características asociados.

La relación entre características y grupos se expresa mediante indentación: los grupos se escriben con respecto a su característica padre, mientras que las características se indentan respecto a su grupo correspondiente. Cada característica debe tener un nombre único que permita identificarla dentro del modelo, pudiendo utilizar comillas cuando contiene símbolos especiales o cuando su nombre está formado por varias palabras.

UVL define diferentes tipos de grupos para expresar las relaciones entre características, incluyendo **mandatory**, **optional**, **or** y **alternative**, pertenecientes al nivel booleano del lenguaje. También incorpora las cardinalidades de grupo (*group cardinality*), que permiten expresar restricciones más avanzadas sobre la selección de características.

Como detalle adicional, cuando una característica lleva mas de una palabra consigo, se suele rodear de comillas dobles "".

2.2.3 Diseño del lenguaje

UVL define modelos de variabilidad mediante una estructura jerárquica basada en árboles, donde cada nodo representa una característica y las relaciones entre dichos nodos permiten establecer las reglas de selección entre ellas. El núcleo del lenguaje se basa en diferentes construcciones que permiten especificar cómo pueden o deben seleccionarse las características dentro de un modelo.

2.2.3.1. mandatory

Una característica obligatoria debe ser seleccionada siempre que su característica padre forme parte de una configuración válida. Por ejemplo, en el siguiente modelo,

si se selecciona la característica *Sensors*, también debe seleccionarse obligatoriamente *Heart Rate Monitor*.

```

1 Sensors
2     mandatory
3     "Heart Rate Monitor"

```

Código 2.2: Ejemplo de característica obligatoria en UVL.

En este caso, la inclusión de *Sensors* implica necesariamente la selección de *Heart Rate Monitor*.

2.2.3.2. optional

Una característica opcional puede ser seleccionada o no dependiendo de las necesidades de una configuración concreta. Por ejemplo:

```

1 Sensors
2     optional
3     GPS
4     "Blood Oxygen Sensor" {Accuracy 95}
5     Accelerometer
6     Gyroscope

```

Código 2.3: Ejemplo de características opcionales en UVL.

En este caso, si la característica *Sensors* es seleccionada, el usuario puede decidir si incorporar o no características como *GPS*, *Blood Oxygen Sensor*, *Accelerometer* o *Gyroscope*.

2.2.3.3. alternative

Un grupo alternativo indica que, si la característica padre es seleccionada, exactamente una de las características hijas debe formar parte de la configuración. Por ejemplo:

```

1 Display
2     alternative
3     LCD
4     OLED
5     "E-Ink"

```

Código 2.4: Ejemplo de grupo alternativo en UVL.

En este caso, al seleccionar *Display*, debe escogerse exactamente una de las alternativas disponibles: *LCD*, *OLED* o *E-Ink*.

2.2.3.4. or

Un grupo **or** permite seleccionar una o más características hijas cuando su característica padre está incluida en una configuración. Por ejemplo:

```

1 Connectivity
2     or
3         Bluetooth
4         "Wi-Fi"
5         LTE

```

Código 2.5: Ejemplo de grupo OR en UVL.

En este caso, pueden seleccionarse varias opciones simultáneamente, siempre que al menos una de ellas sea seleccionada.

2.2.3.5. Restricciones entre características (*Cross-tree Constraints*)

Además de las relaciones jerárquicas definidas mediante la estructura del árbol, UVL permite establecer restricciones lógicas entre características situadas en diferentes ramas del modelo. Estas restricciones, denominadas *cross-tree constraints*, permiten expresar condiciones que no pueden representarse únicamente mediante las relaciones padre-hijo.

Estas restricciones se definen dentro del bloque **constraints** del archivo UVL y permiten utilizar diferentes operadores lógicos:

- Negación mediante **!A**: expresa que una característica no puede estar seleccionada.
- Conjunción mediante **A & B**: indica que ambas características deben estar seleccionadas.
- Disyunción mediante **A | B**: indica que al menos una de las características debe estar seleccionada.
- Implicación mediante **A =>B**: indica que si una característica A está seleccionada, entonces la característica B también debe estarlo.
- Equivalencia mediante **A <=>B**: indica que ambas características deben seleccionarse conjuntamente o ninguna de ellas debe seleccionarse.

Por ejemplo, la siguiente restricción establece que si se selecciona la característica *Fall Detection*, también deben seleccionarse *Accelerometer* y *Gyroscope*:

```
1 constraints
2   "Fall Detection" => (Accelerometer & Gyroscope)
```

Código 2.6: Ejemplo de restricción lógica en UVL.

Otro ejemplo sería:

```
1 constraints
2   "Voice Assistant" <=> "Wi-Fi"
```

Código 2.7: Ejemplo de restricción de equivalencia en UVL.

En este caso, ambas características deben aparecer juntas en una configuración válida, de forma que la selección de una de ellas implica necesariamente la selección de la otra.

2.2.4 Niveles del lenguaje

En UVL, los niveles del lenguaje permiten separar la expresividad del mismo en diferentes capas. Esta organización permite que usuarios y herramientas puedan decidir qué grado de complejidad desean soportar, además de facilitar la extensión del lenguaje sin incrementar la complejidad para aquellos usuarios que únicamente necesitan modelos sencillos.

Un usuario o herramienta que requiera modelos UVL básicos puede trabajar únicamente con el nivel booleano. Sin embargo, cuando sea necesario representar escenarios más complejos, como atributos numéricos, valores textuales u operaciones sobre ellos, pueden activarse niveles superiores.

UVL define tres niveles jerárquicos de expresividad:

- **Boolean level:** incluye características booleanas y restricciones lógicas. También incluye la incorporación de atributos de diferentes tipos en cada característica.
- **Arithmetic level:** incorpora atributos numéricos y operaciones aritméticas.
- **Type level:** incorpora características tipadas, que pueden tomar valores enteros, reales o textuales, y permite establecer restricciones sobre dichos valores.

Cada uno de estos niveles incluye completamente las capacidades proporcionadas por los niveles anteriores. Además, UVL incorpora niveles menores (*minor levels*), que representan extensiones opcionales que añaden funcionalidades adicionales sin ser necesarias para avanzar entre los diferentes niveles principales.

2.2.4.1. Boolean level

El nivel booleano constituye el nivel principal más básico y representa el núcleo del lenguaje UVL. En él se define la estructura del árbol de características, las relaciones entre ellas (**mandatory**, **optional**, **alternative** y **or**) y las restricciones lógicas básicas entre características mediante operadores como **!**, **&**, **|**, **=>** y **<=>**.

Por ejemplo:

```
1 include
2     Boolean.group-cardinality
3
4 features
5     Smartwatch
6         optional
7             GPS
8             Display
9             "Wi-Fi"
10            Bluetooth
11            "Water Resistance"
12            Accessories
13                [0..2]
14                    "Leather Strap"
15                    "Metal Strap"
16                    "Silicone Strap"
17
18 constraints
19     "Wi-Fi" | Bluetooth
20     "Water Resistance" => Display
```

Código 2.8: Ejemplo de modelo UVL utilizando el nivel booleano.

En este nivel únicamente se representa la presencia o ausencia de características dentro de una configuración.

2.2.4.2. Group Cardinality

La cardinalidad de grupo (*Group Cardinality*) es un nivel menor perteneciente al nivel booleano. Permite establecer que, dentro de un grupo de características, únicamente pueda seleccionarse un número determinado de elementos comprendido entre un límite inferior y superior.

Por ejemplo, la expresión:

```
1 [0..2]
```

Código 2.9: Ejemplo de cardinalidad de grupo en UVL.

indica que, cuando se selecciona la característica Accessories, pueden seleccionarse entre cero y dos de sus características hijas: Leather Strap, Metal Strap y Silicone Strap.

2.2.4.3. Arithmetic level

El nivel aritmético incluye las capacidades del nivel booleano y permite expresar restricciones sobre atributos numéricos asociados a las características. Para ello, se utilizan operadores aritméticos como +, -, * y /, junto con operadores de comparación como ==, !=, <, >, <= y >=.

Además, este nivel incorpora dos niveles menores: *Feature Cardinality* y *Aggregate Functions*.

Por ejemplo:

```
1 include
2     Arithmetic.feature-cardinality
3     Arithmetic.aggregate-function
4
5 features
6     Smartwatch
7         mandatory
8             Connectivity
9             or
10                Bluetooth
11                "Wi-Fi"
12        Battery
13            alternative
14                Standard {Capacity 300}
15                "Long Life" {Capacity 450}
16    optional
17        "Music Player" {Storage 2}
18        "Film Player" {Storage 10}
19        "Watch Faces" cardinality [5..20]
20
21 constraints
22     ("Music Player".Storage + "Film Player".Storage) == 12
23     avg(Capacity) >= 300
```

Código 2.10: Ejemplo de atributos y restricciones aritméticas en UVL.

2.2.4.4. Feature Cardinality

La cardinalidad de característica (*Feature Cardinality*) es un nivel menor del nivel aritmético que permite indicar un número de instancias que pueden seleccionarse de dicha característica.

Para utilizar esta funcionalidad debe incluirse explícitamente mediante:

```
1 Arithmetic.feature-cardinality
```

En el siguiente fragmento con partes omitidas se puede ver la forma en la que se emplea:

```
1 include
2     Arithmetic.feature-cardinality
3
4 features
5     Smartwatch
6     ...
7     optional
8         "Watch Faces" cardinality [5..20]
9
10 constraints
11     ...
```

Código 2.11: Ejemplo de cardinalidad de característica en UVL

2.2.4.5. Aggregate Functions

El nivel menor denominado *Aggregate Functions* permite utilizar funciones agregadas como `sum()` y `avg()` dentro del bloque de restricciones.

El siguiente fragmento con partes omitidas muestra dos restricciones agregadas de un modelo cuyas características contienen los atributos numéricos `PriceEuros` y `Calification`. La declaración de dichas características se omite para centrar el ejemplo en las restricciones.

```
1 include
2     Arithmetic.aggregate-function
3
4 features
5     ...
6
7 constraints
8     sum(PriceEuros) > 200
9     avg(Calification) > 5
```

En este ejemplo, `sum(PriceEuros)` suma los valores del atributo `PriceEuros` de las características seleccionadas que lo contienen, y la restricción exige que el resultado sea mayor que 200. Por su parte, `avg(Calification)` calcula la media de los valores del atributo `Calification` de las características seleccionadas que lo contienen, y exige que dicha media sea mayor que 5. Estas funciones agregan los valores de los atributos; no cuentan cuántas veces aparecen en el modelo.

2.2.4.6. Type level

El nivel de tipos amplía las capacidades de los niveles booleano y aritmético mediante características tipadas. Además de las características booleanas, permite declarar características de tipo `Integer`, `Real` y `String`, cuyos valores pueden utilizarse en restricciones. Estos tipos se indican delante del nombre de la característica y no deben confundirse con los atributos declarados entre llaves.

Este nivel también incorpora el nivel menor `String Constraints`, que permite expresar restricciones sobre cadenas mediante funciones como `len()`, utilizada para obtener su longitud.

Por ejemplo:

```
1 include
2     Type.string-constraints
3
4 features
5     Smartwatch
6         mandatory
7             String Color
8             Real Weight
9             String "Serial Number"
10
11 constraints
12     len("Serial Number") == 10
13     Weight > 30.0
14     Color != "Pink"
```

Código 2.12: Ejemplo de características tipadas y restricciones sobre sus valores en UVL.

En este ejemplo, `Color` y `Serial Number` son características de tipo `String`, mientras que `Weight` es una característica de tipo `Real`. La primera restricción exige que el valor de `Serial Number` tenga exactamente diez caracteres. Las otras dos establecen que el valor de `Weight` sea mayor que 30.0 y que el valor de `Color` sea distinto de "Pink".

2.3 Ecosistema de herramientas y aplicación en variabilidad software

A continuación se procederá a hablar sobre herramientas y entornos que permitan trabajar con los modelos de características más allá de su simple representación, las cuales proporcionan capacidades para la creación, gestión y análisis automático de modelos, integrando diferentes técnicas de razonamiento y facilitando su adaptación a nuevos escenarios. Dentro de este ecosistema destaca Flamapy como framework orientado al análisis automático de modelos de características mediante una arquitectura extensible. Asimismo, destaca UVLHub como plataforma destinada a la gestión y compartición de modelos definidos en UVL.

2.3.1 Flamapy como framework de análisis de variabilidad

Flamapy es un framework desarrollado en Python orientado al análisis automático de modelos de características dentro del ámbito de la ingeniería de líneas de productos software [4]. Su objetivo principal es proporcionar un entorno común para trabajar con modelos de variabilidad, facilitando tanto su procesamiento como la aplicación de diferentes técnicas de análisis automático. Para ello, Flamapy puede entenderse desde dos perspectivas complementarias: como una herramienta que permite ejecutar análisis sobre modelos de características de forma directa, y como un framework extensible que proporciona una arquitectura modular para incorporar nuevos modelos, operaciones y técnicas de análisis.

Desde la perspectiva de herramienta, Flamapy proporciona diferentes mecanismos para facilitar su utilización en distintos escenarios. Entre ellos se encuentra una interfaz de línea de comandos (*Command Line Interface*) que permite ejecutar análisis directamente sobre modelos de variabilidad, así como una fachada de Python (*Python Facade*) que simplifica su integración dentro de otros desarrollos software. Además, ofrece interfaces adicionales como servicios REST mediante Swagger y soporte para ejecución en entornos web mediante WebAssembly (WASM), permitiendo utilizar sus capacidades de análisis desde diferentes aplicaciones y contextos.

Como framework, Flamapy sigue una arquitectura modular basada en *plugins*, los cuales permiten extender sus capacidades sin modificar el núcleo principal del sistema. Esta arquitectura hace mucho más sencilla la incorporación de nuevos lenguajes de modelado, metamodelos o técnicas de análisis mediante componentes independientes que pueden integrarse dentro del ecosistema existente. Gracias a este enfoque, es posible adaptar Flamapy a diferentes necesidades relacionadas con la ingeniería de variabilidad, incorporando soporte para nuevos tipos de modelos o herramientas de razonamiento.

Otro de los elementos principales que ofrece Flamapy son las operaciones (*operations*), que representan las diferentes tareas de análisis que pueden ejecutarse sobre modelos de características. Cada operación define una funcionalidad concreta, como la comprobación de satisfacibilidad, la obtención de configuraciones válidas o el cálculo de propiedades del modelo, independientemente del formato en el que este haya sido definido. Esta separación permite reutilizar las mismas operaciones sobre diferentes tipos de modelos siempre que exista una representación compatible dentro del framework.

Por otra parte, las transformaciones (*transformations*) permiten convertir modelos entre diferentes representaciones dentro de Flamapy. Estas transformaciones actúan como mecanismo de conexión entre los distintos metamodelos soportados, permitiendo que los modelos puedan ser procesados por diferentes componentes del framework (o incluso no del framework) independientemente de su representación original. Por ejemplo, el componente *UVLWriter*, que es el que se ha utilizado para descargar los modelos creados por el generador diseñado, permite transformar un modelo de características almacenado internamente en Flamapy a su representación textual en formato UVL. Otras de las transformaciones existentes, por nombrar algunas, son *AFM Reader*, *AFM Writer*, *XML Reader*, *JSON Writer*, *SPLIT Writer* o *FM to BDD*. Todo esto facilita la interoperabilidad entre lenguajes de variabilidad, metamodelos y herramientas de análisis.

2.3.2 UVLHub como plataforma de gestión de modelos UVL

UVLHub es una plataforma orientada a la gestión de modelos de características definidos mediante el lenguaje UVL (*Universal Variability Language*) dentro del ámbito de la ingeniería de líneas de productos software [6]. Tiene como objetivo proporcionar un repositorio especializado que permita almacenar, organizar, compartir y analizar modelos de variabilidad, facilitando tanto la reutilización de estos artefactos tanto en entornos de investigación como de desarrollo software como la colaboración entre usuarios.

La razón principal de su existencia se debe a la complicada localización de los modelos de características empleados por los usuarios, ya que estos los almacenaban en diferentes repositorios, páginas personales o proyectos independientes, utilizando además diferentes formatos de representación, dificultando su búsqueda, reutilización y análisis. UVLHub sigue los principios de ciencia abierta (*Open Science*), como accesibilidad, transparencia y colaboración, con la finalidad de compartir estos modelos de forma homogénea, haciendo más sencilla la colaboración entre usuarios.

Una de las características más relevantes de UVLHub es su capacidad de integración con otras herramientas y servicios relacionados con la gestión y análisis de modelos de variabilidad. Por una parte, la plataforma se integra con Zenodo, un repo-

itorio abierto de investigación que permite almacenar los conjuntos de datos de forma permanente mediante la generación de identificadores DOI (*Digital Object Identifier*), garantizando que los modelos publicados puedan seguir estando disponibles aunque la plataforma dejase de mantenerse en el futuro. Por otra parte, UVLHub incorpora las capacidades de análisis automático proporcionadas por Flamapy, permitiendo obtener información relevante sobre los modelos almacenados, como el número de características, la validez del modelo o el número de configuraciones posibles que pueden derivarse de ellos.

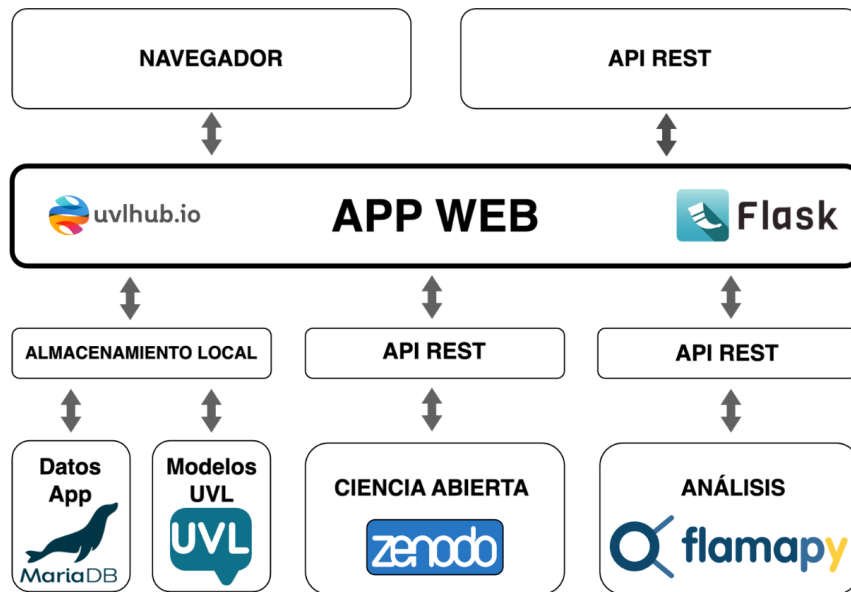


Figura 2.5: Arquitectura general de UVLHub y sus integraciones principales.
Extraído de [15]

Además de sus integraciones externas, UVLHub proporciona diferentes funcionalidades (*features*) en forma de módulos orientadas a cubrir las principales necesidades relacionadas con la gestión de modelos UVL. Algunas de las más relevantes son:

- **dataset**: representa el núcleo principal de UVLHub, permitiendo gestionar el ciclo de vida de los conjuntos de modelos UVL. Esta funcionalidad permite realizar operaciones como la subida de datasets, su almacenamiento junto con los metadatos asociados, la descarga de los modelos y la resolución de identificadores DOI generados mediante la integración con Zenodo.
- **featuremodel**: permite gestionar la información asociada a los modelos de características almacenados dentro de un dataset. Esta *feature* define elementos como el modelo de características, sus metadatos y las métricas asociadas, estableciendo la relación entre los datasets y los archivos UVL almacenados.

- **explore**: proporciona mecanismos para explorar los datasets disponibles dentro de la plataforma mediante diferentes criterios de búsqueda, permitiendo localizar modelos de características publicados previamente.
- **flamapy**: integra las capacidades de análisis automático proporcionadas por Flamapy dentro de UVLHub. Esta integración permite realizar operaciones sobre los modelos UVL, como la validación del modelo o transformaciones hacia otros formatos soportados.
- **zenodo**: permite la comunicación con la API REST de Zenodo para realizar la publicación y almacenamiento permanente de los datasets, proporcionando identificadores DOI asociados a los recursos compartidos.

La siguiente imagen representa, a grandes rasgos, los módulos actuales de UVLHub y cómo se complementan entre sí:

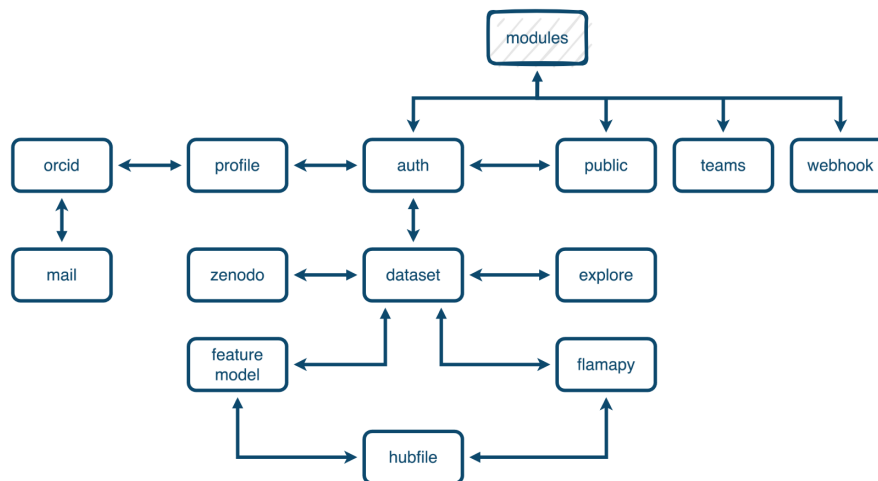


Figura 2.6: Arquitectura general de UVLHub y sus integraciones principales.
Extraído de [15]

Estos módulos son originales de UVLHub. A ellos hay que añadirle también el módulo **generator**, objetivo principal de este TFG.

2.3.3 Generación automática de modelos de características

Ya es bien sabido que los modelos de características permiten representar la variabilidad existente dentro de una línea de productos software. Sin embargo, a medida que aumenta el número de características, restricciones y posibles configuraciones, la creación manual de estos modelos puede convertirse en una tarea costosa y difícil de gestionar.

Por este motivo, la generación automática de modelos de características se ha utilizado como mecanismo para producir modelos sintéticos destinados, entre otros usos, a la evaluación y prueba de herramientas de análisis de variabilidad [16, 17].

Estas técnicas permiten obtener conjuntos de modelos con diferentes tamaños, estructuras y grados de complejidad, facilitando la realización de experimentos sobre distintos escenarios de evaluación.

2.4 Trabajos relacionados

Dentro del ámbito de la ingeniería de líneas de productos software existen diferentes propuestas orientadas a la generación automática de modelos de características. Estas aproximaciones han sido utilizadas principalmente para crear modelos sintéticos destinados a la evaluación de herramientas de análisis de variabilidad, la realización de experimentos controlados o el estudio de la complejidad de diferentes técnicas de razonamiento. Para analizar el estado actual de estas soluciones, se ha tomado como punto de partida el apartado *Related Work* del paper proporcionado por los tutores de este TFG para el desarrollo de un nuevo generador de modelos de características basado en UVL[7]. En dicho apartado se identifican diferentes propuestas existentes dentro de este ámbito, entre las que se encuentran S.P.L.O.T., FeatureIDE, BeTTY y ETHOM. A partir de esta información, se han realizado búsquedas individuales de cada generador, localizando las páginas web donde se encuentran disponibles sus respectivas publicaciones científicas, como ResearchGate para los trabajos asociados a BeTTY y S.P.L.O.T., ScienceDirect para FeatureIDE y ETHOM, así como las páginas oficiales de los propios proyectos cuando estas proporcionaban información adicional. A partir de estas fuentes se ha extraído la información necesaria para analizar las principales características de cada propuesta, prestando especial atención a sus capacidades de personalización, la expresividad de los modelos generados y su integración dentro de otros ecosistemas de variabilidad software.

2.4.1 Generating Feature Models with UVL's Full Expressiveness

El trabajo desarrollado por Sundermann et al. [7] constituye el principal punto de partida del presente TFG. En dicho trabajo se presenta un generador automático de modelos de características basado en Universal Variability Language (UVL), capaz de aprovechar los diferentes niveles de expresividad definidos por el lenguaje.

A diferencia de aproximaciones anteriores centradas principalmente en modelos booleanos, esta propuesta permite generar modelos que incorporan construcciones más avanzadas, incluyendo atributos, restricciones aritméticas y elementos dependientes de tipos. Para ello, los autores definen un sistema configurable que permite controlar diferentes aspectos relacionados con la estructura y complejidad de los modelos generados.

El presente TFG toma esta propuesta como base para desarrollar una implementación integrada dentro del ecosistema Flamapy-UVLHub. Mientras que el trabajo

de Sundermann et al. establece el enfoque de generación y la capacidad de producir modelos UVL expresivos, este proyecto pretende reflejarlo dentro de una plataforma existente, incorporando mecanismos de configuración, integración y validación mediante pruebas automatizadas.

2.4.2 S.P.L.O.T.

S.P.L.O.T. (*Software Product Lines Online Tools*) es una plataforma web orientada al desarrollo y análisis de líneas de productos software, proporcionando diferentes herramientas relacionadas con la gestión de modelos de características [18]. Entre sus funcionalidades se encuentra un generador automático de modelos de características, diseñado para crear modelos sintéticos que pueden ser utilizados posteriormente en tareas de experimentación y evaluación de técnicas de análisis de variabilidad.

Aunque S.P.L.O.T. supuso una de las primeras aproximaciones ampliamente utilizadas para la generación automática de modelos de características, sus capacidades están principalmente orientadas a construcciones pertenecientes al nivel booleano del lenguaje de variabilidad. Por tanto, permite representar estructuras jerárquicas y restricciones lógicas básicas entre características, pero no contempla elementos más avanzados como atributos numéricos, operaciones aritméticas o restricciones basadas en tipos.

2.4.3 FeatureIDE

FeatureIDE es un framework extensible orientado al desarrollo software basado en características que proporciona soporte para diferentes actividades relacionadas con la ingeniería de líneas de productos software [19]. La plataforma permite crear, editar y analizar modelos de características, integrando diferentes herramientas destinadas a facilitar el desarrollo de sistemas configurables.

Además de sus capacidades de gestión y análisis, FeatureIDE incorpora un generador automático de modelos de características que permite crear modelos sintéticos dentro de su propio ecosistema. Sin embargo, al igual que otras propuestas existentes, su generación está principalmente limitada a construcciones pertenecientes al nivel booleano de los lenguajes de variabilidad, por lo que no permite generar modelos que incorporen toda la expresividad proporcionada por lenguajes más recientes como UVL.

2.4.4 BeTTy

BeTTy (*Benchmarking and Testing on the Automated Analysis of Feature Models*) es un framework orientado al benchmarking y evaluación de herramientas destinadas al análisis automático de modelos de características [20]. A diferencia de aproxima-

ciones más simples, BeTTy permite configurar diferentes parámetros relacionados con la generación de modelos, facilitando la creación de escenarios experimentales adaptados a las necesidades de evaluación.

Una de sus principales aportaciones es la capacidad de generar modelos con diferentes niveles de complejidad estructural y restricciones, permitiendo estudiar el comportamiento de diferentes técnicas de razonamiento sobre escenarios controlados [20]. Sin embargo, aunque proporciona una mayor flexibilidad respecto a generadores anteriores, sus capacidades siguen centradas principalmente en el nivel booleano del lenguaje de variabilidad. Por ello, permite aumentar la dificultad de análisis de los modelos generados, pero no ampliar su expresividad mediante construcciones como atributos o restricciones aritméticas.

2.4.5 ETHOM

ETHOM (*Evolutionary algoRiTHm for Optimized feature Models*) es una propuesta basada en algoritmos evolutivos orientada a la generación automática de modelos de características optimizados [21]. A diferencia de los generadores puramente aleatorios, ETHOM formula la generación como un problema de optimización, utilizando una función de evaluación que permite buscar modelos con determinadas propiedades.

En particular, ETHOM fue diseñado para generar modelos de características computacionalmente difíciles de analizar, permitiendo evaluar el comportamiento de herramientas de análisis de variabilidad bajo escenarios exigentes. Para ello, el algoritmo recibe diferentes restricciones relacionadas con el modelo a generar y busca configuraciones que maximicen criterios como el tiempo de ejecución o el consumo de memoria de las herramientas evaluadas [21].

Esta propuesta supone una evolución respecto a la generación aleatoria de modelos, ya que permite crear casos de prueba adaptados a objetivos concretos de evaluación. Sin embargo, al estar orientado a modelos de características tradicionales, no contempla la generación de construcciones avanzadas propias de lenguajes con mayor expresividad como UVL.

2.4.6 Comparativa de trabajos relacionados

Trabajo	Generación automática	Personalización	Expresividad UVL	Integración UVLHub
S.P.L.O.T. [18]	Sí	Parcial	No	No
FeatureIDE [19]	Sí	Parcial	No	No
BeTTy [20]	Sí	Sí	No	No
ETHOM [21]	Sí	Sí	No	No
Sundermann et al. [7]	Sí	Sí	Sí	No
Este TFG	Sí	Sí	Sí	Sí

Tabla 2.2: Comparativa de trabajos relacionados con la generación automática de modelos de características.

El presente capítulo ha establecido los fundamentos teóricos y tecnológicos necesarios para comprender el contexto del trabajo, así como las principales soluciones existentes relacionadas con la generación automática de modelos de características. A partir de este análisis se define el marco sobre el que se desarrolla la propuesta presentada en este TFG. El siguiente capítulo describe la planificación seguida durante el proyecto, incluyendo la metodología, las fases de desarrollo, los recursos empleados y la gestión de riesgos.

3 PLANIFICACIÓN DEL PROYECTO

3.1 Acta de constitución del proyecto

Elemento	Descripción
Propósito	Desarrollar una herramienta que permita generar automáticamente modelos de características en formato UVL para facilitar la creación de modelos de variabilidad software, proporcionando a investigadores y desarrolladores un mecanismo configurable para definir las características, restricciones y atributos de los modelos generados.
Objetivos	Los objetivos del proyecto son los definidos en la Sección 1.5. De forma general, el proyecto busca diseñar e implementar un generador automático de modelos de características, integrarlo en UVLHub y proporcionar soporte para diferentes niveles de expresividad del lenguaje UVL.
Alcance	El proyecto comprende el diseño e implementación del generador automático de modelos de características, su integración dentro de la plataforma UVLHub y la incorporación progresiva de soporte para modelos booleanos, aritméticos y con tipos definidos por UVL. Quedan fuera del alcance la modificación del funcionamiento interno de Flamapy y el desarrollo de nuevos algoritmos de análisis o razonamiento.
Supuestos	Se asume la disponibilidad del ecosistema software necesario para el desarrollo, incluyendo acceso al código fuente de UVLHub y Flamapy, así como las herramientas necesarias para la generación y análisis de modelos de variabilidad. También se considera disponible el entorno de desarrollo necesario para implementar, probar y validar la solución.
Restricciones	El proyecto está condicionado por las tecnologías y arquitectura existentes en UVLHub y Flamapy, especialmente el lenguaje Python (su framework Flask) y el estándar UVL. Además, debe desarrollarse antes del 7 de octubre, la fecha límite para la entrega de la memoria.
Entregables principales	Los principales entregables del proyecto son el código fuente del generador automático integrado en UVLHub y la memoria del TFG.
Criterio de cierre	El proyecto se considerará finalizado cuando el generador

3.1.1 Interesados del proyecto

Interesado	Rol	Interés / expectativa	Implicación
Autor del proyecto	Responsable del desarrollo	Completar el Trabajo Fin de Grado mediante el desarrollo de una solución funcional, documentada y alineada con los objetivos establecidos.	Diaria
Tutor del TFG	Supervisión y validación académica	Garantizar el correcto desarrollo del proyecto, proporcionando orientación técnica y revisando los avances y entregables realizados.	Periódica
Grupo de investigación / ecosistema UVLHub-Flamapy	Entorno receptor de la solución	Disponer de una herramienta que permita generar modelos de características de forma automática y ampliar las capacidades existentes del ecosistema de variabilidad software.	Puntual
Tribunal del TFG	Evaluación final del proyecto	Valorar la calidad de la memoria, la defensa realizada y los resultados obtenidos durante el desarrollo del trabajo.	Al cierre

Tabla 3.2: Registro de interesados del proyecto.

3.2 Metodología

3.2.1 Metodología de desarrollo

Para el desarrollo del proyecto se ha seguido una metodología incremental e iterativa, adaptada a la naturaleza evolutiva del generador automático de modelos de características. Debido al carácter investigador y evolutivo del proyecto, la especificación inicial se refinó progresivamente durante el desarrollo conforme se analizaban las necesidades del generador y su integración dentro del ecosistema existente. [7].

En una primera etapa se desarrolló una versión inicial del generador orientada a la creación de modelos de características booleanos, junto con su integración dentro de UVLHub mediante la interfaz gráfica proporcionada para configurar el proceso

de generación. Una vez obtenida esta primera versión funcional, se realizaron ciclos sucesivos de implementación, validación y corrección que permitieron mejorar la estabilidad del sistema e incorporar nuevas funcionalidades pertinentes a los siguientes niveles de expresividad que se pretendió conseguir. Esta forma de hacerlo permitió validar cada nueva incorporación antes de continuar con las siguientes etapas y reducir el riesgo de introducir errores en funcionalidades previamente implementadas que se pudieran arrastrar a las siguientes.

Alternativas consideradas

No se optó por metodologías secuenciales como el modelo en cascada, ya que la naturaleza del proyecto requería realizar ajustes continuos conforme avanzaba el desarrollo y se identificaban nuevas necesidades relacionadas con la generación de modelos de características. Tampoco se aplicó una metodología Scrum completa, puesto que el desarrollo no se organizó mediante sprints, roles o hitos propios de este marco de trabajo.

Control de versiones

El control de versiones del proyecto se ha realizado mediante Git, utilizando GitHub como plataforma de almacenamiento y seguimiento de la evolución del código. Debido a la evolución del proyecto y a la necesidad de integrar el generador dentro de diferentes versiones de UVLHub, el desarrollo quedó distribuido entre varios repositorios y una solicitud de incorporación de cambios (*pull request*) en el repositorio oficial del proyecto.

Por este motivo, el repositorio `antjimort/uvlhub` no representa por sí solo todo el historial del Trabajo Fin de Grado. Dicho repositorio recoge principalmente la fase final de integración, refactorización, corrección y validación realizada entre mayo y septiembre de 2026. La actividad desarrollada entre mayo y julio corresponde a la fase principal de estabilización, mientras que entre el 14 y el 18 de septiembre se realizaron correcciones adicionales derivadas de la revisión técnica del tutor.

Recurso	Periodo relevante	Papel dentro del desarrollo
fm_generator	15/05/2025– 21/04/2026	Repositorio inicial del componente generador. Contiene 38 commits realizados por el autor relacionados con la implementación de niveles UVL, atributos, restricciones, cardinalidades y opciones de configuración. Los commits anteriores a mayo de 2025 no se consideran parte de la contribución del TFG.
uvlhub_v2	14/02/2026– 03/04/2026	Repositorio transitorio utilizado durante la migración de los cambios previos a una nueva versión de UVLHub. En él se realizaron ajustes de migración, persistencia de parámetros del asistente y avances en los niveles booleano y aritmético del generador.
PR #117	06/04/2026– 19/04/2026	Solicitud de incorporación de cambios en el repositorio oficial de UVLHub, titulada <i>feat: boolean level ready</i> . La PR contiene 46 commits en total, de los cuales 45 fueron realizados por el autor, y fue fusionada en la rama generator el 19 de abril de 2026.
uvlhub	05/2026– 09/2026	Repositorio utilizado durante la fase final de integración, estabilización y consolidación técnica. Incluye refactorizaciones de la arquitectura, correcciones de compatibilidad, adaptación de pruebas, automatización de despliegue y correcciones finales relacionadas con la validación de modelos UVL, PySAT y la integración del generador como <i>plugin</i> de Flamapy.

Tabla 3.3: Trazabilidad de los repositorios y recursos utilizados durante el desarrollo.

Los repositorios y la solicitud de incorporación de cambios anteriores no deben interpretarse como desarrollos independientes que puedan sumarse entre sí, ya que parte de los commits realizados en el repositorio **fm_generator** fueron posteriormente incorporados en el PR #117. La Tabla 3.3 permite identificar el origen de cada fase del desarrollo sin duplicar la dedicación contabilizada en el cronograma.

La convención *Conventional Commits* se adoptó progresivamente durante el proyecto. Por este motivo, algunos commits iniciales no siguen una nomenclatura

completamente normalizada. Sin embargo, en las fases posteriores se utilizaron prefijos como *feat* para la incorporación de nuevas funcionalidades, *fix* para la corrección de errores, *refactor* para reorganizaciones internas del código, *test* para cambios en las pruebas, *chore* para tareas de mantenimiento, *style* para ajustes de estilo y *ci* para modificaciones de integración continua.

Esta organización permite mantener la trazabilidad de la evolución completa del generador, desde la implementación inicial como componente independiente hasta su integración y estabilización dentro de UVLHub.

Herramientas de seguimiento

El seguimiento del desarrollo se ha realizado principalmente mediante el propio historial de GitHub, utilizando los commits realizados como registro de la evolución del proyecto y de las decisiones técnicas tomadas durante la implementación.

Además, las reuniones periódicas con el tutor han permitido revisar el estado del desarrollo para analizar las deficiencias encontradas con el propósito de corregirlas y llevar la calidad del generador al siguiente nivel. Los detalles de estas reuniones se detallarán en el siguiente apartado.

3.2.2 Metodología de validación y evaluación

La validación del proyecto se ha realizado mediante un proceso iterativo durante el desarrollo, combinando pruebas funcionales, pruebas automatizadas y revisiones periódicas con el tutor del proyecto.

Tras la implementación de cada nueva funcionalidad, se realizaron comprobaciones manuales para verificar que el generador producía modelos de características correctos según los parámetros configurados, incluyendo los diferentes niveles de expresividad soportados por UVL, así como la correcta interacción del usuario con el front-end desarrollado en UVLHub. Además, se ejecutaron pruebas unitarias automatizadas para detectar posibles errores o regresiones, incorporando nuevos casos de prueba únicamente cuando aportaban valor al sistema; es decir, que encontraran posibles bugs reales y relevantes.

La evaluación del cumplimiento del proyecto se ha realizado siguiendo los criterios de éxito definidos en la Tabla 1.2. Estos criterios permiten comprobar de forma objetiva aspectos como la generación automática de modelos UVL, la configuración del proceso de generación, el soporte de construcciones avanzadas del lenguaje, la integración dentro del ecosistema Flamapy y UVLHub, y la existencia de una batería de pruebas automatizadas.

Además de la validación técnica continua, se realizaron revisiones del estado del proyecto junto con el tutor. Cuando se alcanzó aproximadamente el 80 % de desarrollo,

se realizó una primera revisión en la que se concluyó que la solución era funcionalmente válida, pero presentaba ciertas deficiencias técnicas que debían ser corregidas antes de considerarla finalizada. Entre ellas se identificaron problemas relacionados con la integración del generador dentro de UVLHub, la pérdida de algunas funcionalidades existentes durante determinadas modificaciones (que posteriormente se recuperaron sin problema), la reducción de la cobertura de pruebas, la presencia de elementos no deseados en el repositorio y aspectos de calidad del código (documentación y limpieza de la implementación).

Tras aplicar las correcciones necesarias, se realizó una segunda revisión con el tutor para comprobar el estado de la nueva versión. En esta ocasión únicamente se detectaron ajustes menores, que fueron solucionados posteriormente, obteniendo finalmente la validación positiva del tutor.

3.3 Estructura de descomposición del trabajo (EDT)

Este proyecto se podría desglosar de la siguiente manera:

ID	Paquete de trabajo	Entregable verificable
1	Gestión y planificación del proyecto	Planificación inicial del proyecto, estudio de la solución propuesta por los investigadores a partir del trabajo de referencia y seguimiento del desarrollo realizado.
1.1	Planificación inicial y estudio de la solución	Acta de constitución, definición del alcance, planificación temporal del proyecto y análisis de la solución propuesta en el artículo de referencia.
1.2	Seguimiento y control	Reuniones de seguimiento, revisión del estado del desarrollo y gestión de los cambios realizados durante el proyecto.
2	Análisis y diseño de la solución	Diseño de la arquitectura del generador y definición de los componentes necesarios para su implementación.
2.1	Análisis de requisitos	Catálogo de requisitos funcionales y técnicos del generador automático de modelos.
2.2	Diseño del generador y su integración	Arquitectura definida para la generación de modelos, configuración del sistema e integración con UVLHub.
3	Desarrollo del generador automático	Implementación del generador capaz de producir modelos de características en formato UVL.

ID	Paquete de trabajo	Entregable verificable
3.1	Implementación de niveles de expresividad	Soporte implementado para generación de modelos booleanos, aritméticos y con tipos definidos por UVL.
3.2	Desarrollo de configuración e interfaz	Sistema de configuración del generador e integración del flujo de interacción mediante el front-end de UVLHub.
4	Pruebas y validación	Suite de pruebas automatizadas y validación funcional del generador integrado.
5	Documentación y defensa	Memoria final del Trabajo Fin de Grado y material necesario para la presentación de defensa.

Tabla 3.4: Estructura de descomposición del trabajo (EDT).

3.4 Cronograma e hitos (línea base)

Teniendo como fecha de inicio del proyecto el 12 de abril de 2025, el cronograma real quedó dividido en varias etapas de desarrollo, integración y validación. La Tabla 3.5 recoge los periodos de actividad identificables a partir del historial de los repositorios y de los hitos de integración del proyecto.

Periodo real	Fin previsto del paquete	EDT	Actividad principal	Hito o resultado
04/2025	04/2025	1.1	Análisis inicial y planificación del proyecto.	Planificación inicial y estudio de viabilidad del generador.
04/2025	04/2025	2.1–2.2	Análisis de requisitos y diseño de la solución.	Requisitos definidos y prototipo inicial preparado para validar la integración.
05/2025– 07/2025	05/2026	3	Primera iteración del generador y de su integración inicial con UVLHub.	Implementación de restricciones booleanas, cardinalidades, atributos manuales, configuración inicial del <i>wizard</i> e integración preliminar del componente.
08/2025– 01/2026	—	—	Periodo de inactividad del proyecto.	No se realizaron actividades ni se establecieron hitos durante este intervalo.
02/2026– 04/2026	05/2026	3	Reanudación del desarrollo, migración a UVLHub 2.0 e incorporación de nuevos niveles de expresividad.	Migración funcional, soporte del nivel booleano, funciones agregadas e integración mediante el PR #117, fusionado el 19 de abril de 2026.
05/2026– 07/2026	05/2026	3–4	Integración final, refactorización, adaptación de pruebas y validación del sistema.	Generador estabilizado, pruebas adaptadas a la nueva arquitectura y validación final del tutor.
14/09/2026– 18/09/2026	—	3–4	Correcciones finales y consolidación técnica posterior a la fase principal de validación.	Corrección de la generación de tipos y expresiones, validación de modelos UVL con <code>UVLReader</code> y <code>PySAT</code> , integración del generador como <i>plugin</i> de <code>Flamapy</code> , reorganización de las pruebas unitarias y limpieza de elementos no deseados del repositorio.

Tabla 3.5: Cronograma de ejecución e hitos del proyecto.

La distribución del esfuerzo se mantiene agrupada por paquetes de trabajo, ya que las horas reales se registraron por fases generales y no de forma independiente para cada una de las iteraciones temporales anteriores. Esta decisión evita asignar artificialmente horas concretas a periodos para los que no existe un registro horario detallado.

#	Fase	EDT	Obj.	Est. (h)	Real (h)	Hito / entregable
1	Análisis inicial y planificación del proyecto	1.1	O1	12	12	Prototipo inicial y planificación del desarrollo validada.
2	Análisis y diseño de la solución	2.1–2.2	O1–O2	25	25	Requisitos definidos, análisis de la solución propuesta y prototipo inicial desarrollado para validar la viabilidad de la integración.
3	Desarrollo e integración del generador automático	3	O1–O4	593	703	Generador implementado e integrado dentro de UVLHub con soporte para los niveles de expresividad definidos y corrección de errores detectados durante el desarrollo.
4	Pruebas y validación de la solución	4	O5 y O6	120	170	Suite de pruebas automatizadas, validaciones funcionales y revisiones finales del sistema completadas.
Total				750	910	

Tabla 3.6: Fases de desarrollo y esfuerzo asociado: línea base frente a valores reales.

La fase principal de desarrollo, integración y validación concluyó en julio de 2026. Posteriormente, entre el 14 y el 18 de septiembre de 2026, se realizó una fase adicional de correcciones y consolidación técnica como respuesta a las observaciones del tutor. Por tanto, el cierre técnico definitivo del proyecto se sitúa en septiembre de 2026.

La línea base preveía finalizar las fases principales en mayo de 2026. La finalización de la fase principal en julio supuso una desviación de dos meses, mientras que el cierre técnico definitivo en septiembre representó una desviación acumulada de cuatro meses respecto a la previsión inicial.

En términos de esfuerzo, las 910 horas reales de desarrollo superaron en 160 horas las 750 horas inicialmente estimadas, lo que representa una desviación del 21,3 %. El esfuerzo adicional se concentró en las fases de desarrollo e integración del generador y de pruebas y validación, debido a los problemas de compatibilidad con dependencias externas, la necesidad de realizar refactorizaciones, la adaptación de las pruebas y las correcciones finales realizadas en septiembre tras la revisión del tutor.

De las 910 horas de desarrollo, 400 corresponden al contrato de apoyo técnico asociado a TASOVA Plus y las 510 restantes son atribuibles al desarrollo del TFG. Junto con las 70 horas dedicadas a la redacción, maquetación y revisión de la memoria, la dedicación atribuible al TFG asciende a 580 horas.

#	Capítulos	EDT	Est.	Real	Hito / Entregable
1	Caps. 1–3: introducción, estado del arte y planificación	5	30 h	33 h	Borrador de contexto y plan aprobado por el tutor.
2	Caps. 4–7: núcleo técnico (requisitos, diseño, implementación y pruebas)	5	25 h	24 h	Núcleo técnico redactado y trazable con el repositorio.
3	Caps. 8–9: resultados y conclusiones	5	15 h	13 h	Memoria completa lista para revisión final y depósito.
Total			70 h	70 h	

Tabla 3.7: Fases de redacción de la memoria (estimadas vs. reales).

Las 70 horas reales indicadas en la Tabla 3.7 corresponden al trabajo completo de elaboración de la memoria, incluyendo la redacción y revisión de los capítulos, la organización de referencias, la preparación de tablas y figuras, la maquetación en L^AT_EX, la revisión de referencias cruzadas y la preparación de los apéndices.

La distribución de 33, 24 y 13 horas corresponde a bloques de capítulos y no a una estimación basada únicamente en su extensión. Las tareas técnicas necesarias para obtener resultados, ejecutar pruebas y recopilar evidencias se contabilizaron dentro de las fases de desarrollo y validación, evitando sumar dos veces el mismo esfuerzo.

# Fase	Est.	Real
Desarrollo (Tabla 3.6)	750 h	910 h
Memoria (Tabla 3.7)	70 h	70 h
Total	820 h	980 h

Tabla 3.8: Resumen global del proyecto (estimadas vs. reales).

El total de 980 horas representa la dedicación real acumulada durante el proyecto, pero no constituye íntegramente la carga académica del TFG. De este total, 400 horas corresponden al contrato de apoyo técnico asociado a TASOVA Plus y no forman parte de la dedicación académica del TFG. Las 580 horas restantes sí son atribuibles al TFG, incluyendo 510 horas de desarrollo y 70 horas dedicadas a la redacción, maquetación y revisión de la memoria.

Por tanto, la comparación con las 300 horas asociadas a un TFG de 12 ECTS debe realizarse utilizando las 580 horas propias del TFG y no las 980 horas acumuladas durante todo el proyecto. La dedicación específica del TFG fue superior a la carga académica teórica debido principalmente a la complejidad de la integración con UVLHub y Flamapy, la adaptación de las pruebas, las tareas adicionales de validación y las correcciones derivadas de las revisiones técnicas.

3.5 Plan de comunicaciones y seguimiento

Actividad	Cadencia	Vía	Resultado esperado
Reuniones con el tutor	Según avance del proyecto	Presencial	Revisión del estado del desarrollo, validación de avances, detección de posibles deficiencias y definición de las próximas tareas a realizar.
Mensajería asíncrona	Bajo demanda	Telegram y correo electrónico	Resolución de dudas sobre el diseño y desarrollo del sistema, recepción de indicaciones adicionales, concertación de tutorías y evaluación de avances menores.

Actividad	Cadencia	Vía	Resultado esperado
Seguimiento del desarrollo	Continua	Repositorio Git/GitHub	Trazabilidad de la evolución del proyecto mediante el historial de cambios y commits realizados durante la implementación.

Tabla 3.9: Plan de comunicaciones y seguimiento del proyecto.

Durante el desarrollo del proyecto se han mantenido diferentes reuniones presenciales con el tutor con el objetivo de revisar el estado de la solución y validar las decisiones tomadas durante la implementación. La primera reunión relevante tuvo lugar el 5 de mayo de 2026, cuando se alcanzó aproximadamente el 80 % del desarrollo. En ella se concluyó que la solución era funcionalmente aceptable, aunque presentaba ciertas deficiencias técnicas que debían ser corregidas.

Tras aplicar las correcciones necesarias, el 29 de junio de 2026 se realizó una segunda revisión del estado del sistema y el 10 de julio se validó positivamente la versión principal de la solución. Posteriormente, durante la revisión técnica final, el tutor señaló nuevos aspectos relacionados con la integración como *plugin* de Flamapy, la validación de los modelos generados y la organización de las pruebas. Estas observaciones dieron lugar a las correcciones realizadas entre el 14 y el 18 de septiembre de 2026, fecha en la que se considera cerrado técnicamente el desarrollo.

3.6 Gestión de riesgos

ID	Riesgo	Prob.	Impacto	Exposición	Mitigación
R1	Complejidad de integración del generador dentro del ecosistema UVLHub-Flamapy	Alta	Alto	Alta	Realizar un estudio inicial de la arquitectura existente, validar progresivamente los cambios realizados y comprobar la compatibilidad entre los diferentes componentes del sistema.

ID	Riesgo	Prob.	Impacto	Exposición	Mitigación
R2	Problemas derivados de dependencias externas y configuración del entorno de ejecución	Media	Alto	Alta	Mantener actualizadas las dependencias necesarias, documentar la configuración del entorno y realizar pruebas de integración tras modificar versiones o paquetes externos.
R3	Errores en la generación de modelos tras la incorporación de nuevas funcionalidades	Alta	Medio	Media	Realizar validaciones funcionales tras cada modificación e incorporar pruebas automatizadas que permitan detectar errores en las funcionalidades implementadas.
R4	Subestimación del esfuerzo necesario para completar la implementación y validación del sistema	Media	Alto	Media	Realizar un seguimiento periódico del progreso, priorizar las tareas críticas y reservar tiempo adicional para corrección de errores y mejora de las pruebas.

Tabla 3.10: Riesgos identificados, exposición y planes de mitigación.

3.6.1 Seguimiento de riesgos al cierre

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
R1	Sí	El riesgo apareció principalmente durante la fase inicial de comprensión de Flamapy y durante la integración del generador dentro de UVLHub. Para solucionarlo se estudió la arquitectura de Flamapy, se realizaron modificaciones sobre la estructura del código y se aplicó una estrategia de pruebas progresivas para validar cada incorporación realizada.	El generador se encuentra completamente integrado y cumple con la funcionalidad establecida inicialmente. No obstante, como ocurre en cualquier proyecto software, siempre existe margen de mejora relacionado con la reducción de deuda técnica y la optimización de la arquitectura interna.
R2	Sí	Se produjeron problemas relacionados con las dependencias externas del proyecto, especialmente con la actualización de Flamapy, la creación y configuración de los wheels necesarios para Pyodide y los problemas encontrados para que estos paquetes fueran localizados y utilizados correctamente. La solución consistió en actualizar dependencias, crear nuevos wheels cuando fue necesario e investigar la configuración requerida para su correcta integración.	El sistema funciona correctamente con las versiones actuales de las dependencias utilizadas. Sin embargo, futuras actualizaciones de estas tecnologías podrían requerir nuevos ajustes para mantener la compatibilidad.

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
R3	Sí	Durante el desarrollo se detectaron en varias ocasiones problemas en la generación de modelos debido a que determinadas modificaciones no producían los resultados esperados. Para solucionarlos se comprobaba inicialmente el fallo mediante la generación de diferentes modelos, posteriormente se analizaba la causa, se corregía la implementación y se realizaban nuevas validaciones manuales, actualizando las pruebas unitarias cuando aportaban valor.	La generación de modelos ha sido validada de forma exhaustiva siguiendo los criterios establecidos. Aun así, no se descarta la aparición de nuevos casos límite ante futuras ampliaciones o nuevos escenarios de uso.

ID	¿Se materializó? - Respuesta aplicada	Riesgo residual
R4	Sí La estimación inicial del esfuerzo necesario se vio superada debido a diversos factores relacionados con la calidad final del producto, como los errores detectados durante la implementación, la necesidad de mejorar el enfoque de las pruebas, los problemas de integración y las tareas de refactorización necesarias. La gestión de este riesgo se realizó priorizando las correcciones necesarias, reorganizando tareas y utilizando las revisiones con el tutor para obtener feedback sobre el estado del proyecto.	El principal riesgo residual está relacionado con la posible existencia de deuda técnica pendiente. Una mayor reducción de esta deuda mediante refactorizaciones y mejoras arquitectónicas podría facilitar futuras ampliaciones y reducir el esfuerzo necesario para incorporarlas.

Tabla 3.11: Seguimiento de riesgos al cierre: materialización, respuesta y residuales.

3.7 Control de cambios

ID	Fecha	Cambio	Causa	Impacto en la línea base	Aprobado por
C1	12/05/2025	Ampliación de las opciones de configuración del generador para aumentar la flexibilidad del proceso de generación.	Necesidad de proporcionar una mayor capacidad de adaptación de los modelos generados a distintos escenarios de uso.	Incremento del alcance funcional mediante la incorporación de nuevas opciones de configuración relacionadas con la generación de operadores y atributos.	Tutor / investigadores
C2	Durante el desarrollo	Ampliación del plazo previsto para completar la implementación y validación del generador automático.	La estimación inicial no contemplaba completamente la complejidad de la integración con el ecosistema UVLHub-Flamapy, así como los problemas surgidos durante la implementación y corrección de errores.	Desplazamiento del hito previsto de finalización del desarrollo y aumento del esfuerzo dedicado a la implementación y validación del sistema.	Tutor

ID	Fecha	Cambio	Causa	Impacto en la línea base	Aprobado por
C3	05/05/2026	Revisión del enfoque técnico y refactorización de la estructura interna del generador para mejorar su integración y mantenibilidad.	Durante la revisión del estado del proyecto se identificaron deficiencias técnicas relacionadas con la organización interna del código y la integración del generador dentro del ecosistema existente.	Necesidad de reorganizar componentes internos, mejorar la arquitectura de la solución y adaptar parte de la implementación existente.	Tutor, reu- nión de se- guimiento 05/05/2026
C4	05/2026	Sustitución del front-end inicial por la propuesta proporcionada por el tutor para la integración del generador en UVLHub.	La propuesta alternativa ofrecía una mejor adaptación al flujo existente de la plataforma y una experiencia de usuario más adecuada para configurar el proceso de generación.	Modificación de la implementación del front-end y adaptación del generador al nuevo flujo de interacción, manteniendo los objetivos funcionales definidos inicialmente.	Tutor

ID	Fecha	Cambio	Causa	Impacto en la línea base	Aprobado por
C5 14–	18/09/2026	Corrección y consolidación final del generador tras la revisión técnica del tutor.	Observaciones relativas a la integración como <i>plugin</i> de Flamapy, la validación de modelos UVL, la generación satisfacible y la organización de las pruebas.	Extensión del cierre técnico hasta septiembre de 2026, sin modificar el alcance funcional del proyecto.	Tutor

Tabla 3.12: Registro de control de cambios sobre la línea base.

3.8 Plan de calidad

El plan de calidad definido para el proyecto establece los criterios utilizados para garantizar la calidad tanto de la solución software desarrollada como de la documentación asociada al TFG.

- **Código:** se ha buscado mantener una implementación organizada, legible y mantenible, siguiendo una estructura coherente con la arquitectura definida para el generador. Para facilitar la comprensión del código y detectar posibles problemas de estilo, se ha utilizado una herramienta de *linting* integrada dentro del flujo de desarrollo mediante GitHub Actions, ya incorporada en la configuración de UVLHub. Además, durante el desarrollo se realizaron tareas de refactorización orientadas a mejorar la calidad interna del código y reducir posibles problemas derivados de la evolución del sistema. La trazabilidad de los cambios se ha mantenido mediante el control de versiones con Git y el uso de commits siguiendo la convención *Conventional Commits*. La siguiente imagen muestra los workflows usados:

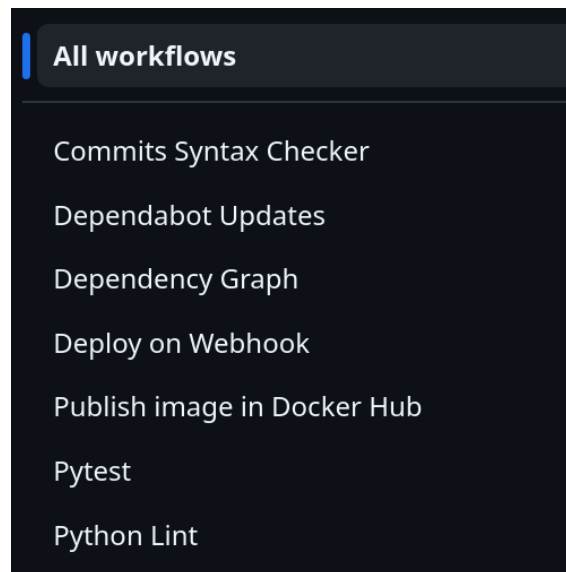


Figura 3.1: Arquitectura general de UVLHub y sus integraciones principales.
Elaboración propia.

- **Entregables:** cada bloque funcional desarrollado debía cumplir con los objetivos establecidos y aportar una funcionalidad verificable dentro del sistema. Para ello, las nuevas incorporaciones se validaban progresivamente antes de continuar con el desarrollo de nuevas capacidades, asegurando que los componentes implementados mantenían el comportamiento esperado.
- **Pruebas:** la calidad funcional del sistema se ha comprobado mediante la combinación de validaciones manuales y pruebas automatizadas. Tras la incorporación de nuevas funcionalidades se verificaba que los modelos generados cumplían los criterios esperados y que la interacción mediante el front-end funcionaba correctamente. Además, se incorporaron pruebas unitarias adicionales cuando estas aportaban valor al detectar posibles errores relevantes en el sistema.
- **Memoria:** la calidad de la documentación se ha garantizado mediante revisiones periódicas de la estructura y contenido de la memoria, así como mediante la revisión de los avances con el tutor del proyecto. Antes de la entrega final se realizará una comprobación de la coherencia entre los diferentes capítulos, referencias cruzadas y bibliografía utilizada.

3.9 Presupuesto

CapEx — Costes de capital

Recurso	Descripción	Coste real	Valor
Equipo de desarrollo	Equipo personal utilizado para la implementación, pruebas y mantenimiento del sistema.	0,00 €	X €
Licencias software	Herramientas de desarrollo y tecnologías empleadas durante el proyecto, basadas principalmente en software libre.	0,00 €	X €
Total CapEx		0,00 €	X €

Tabla 3.13: Costes de capital (CapEx).

Los recursos utilizados durante el desarrollo no supusieron una inversión económica directa conocida, ya que se empleó principalmente infraestructura propia y herramientas basadas en software libre o de coste nulo. Por tanto, el coste real asociado a los costes de capital del proyecto es de 0,00 €.

OpEx — Costes operacionales

Personal

El coste de personal corresponde a la dedicación asociada al contrato de apoyo técnico vinculado al proyecto TASOVA Plus. Dicho contrato tuvo una duración de cuatro meses, entre abril y julio de 2025, y contemplaba una dedicación de 25 horas semanales junto con una remuneración bruta mensual de 930,60 €.

La tarifa utilizada para calcular el coste estimado se ha fijado de forma uniforme en 9,31 € por hora y se aplica a las 750 horas de desarrollo previstas inicialmente. Por este motivo, los costes estimados de la tabla se obtienen multiplicando las horas estimadas de cada fase por dicha tarifa.

El coste real refleja únicamente la remuneración correspondiente al contrato de TASOVA Plus. Las cuatro mensualidades abonadas durante el periodo contractual suponen un coste real total de $4 \times 930,60 = 3722,40$ £, equivalente aproximadamente a 9,306 € por hora para las 400 horas contractuales. Las horas realizadas específicamente para el TFG se desarrollaron fuera del periodo financiado y no representan un coste directo adicional de personal.

Actividad	H. estimadas	H. TASA-VA	H. TFG	H. reales	Coste estimado (€)	Coste real (€)
Análisis inicial y planificación del proyecto	12	12	0	12	111,72	111,67
Análisis y diseño de la solución	25	25	0	25	232,75	232,65
Desarrollo e integración del generador automático	593	363	340	703	5520,83	3378,08
Pruebas y validación de la solución	120	0	170	170	1117,20	0,00
Total personal	750	400	510	910	6982,50	3722,40

Tabla 3.14: Desglose de horas y costes de personal.

Herramientas y servicios de desarrollo

Herramienta	Licencia utilizada	Coste real	Valor de mercado
GitHub	Plataforma utilizada para almacenar el código fuente y gestionar el control de versiones.	0 €	0 €
Render	Servicio utilizado para desplegar una versión accesible de UVLHub durante las revisiones funcionales del sistema.	0 €	X € / mes
Python, Flask, Flasky y librerías asociadas	Tecnologías y dependencias utilizadas durante el desarrollo bajo licencias abiertas.	0 €	0 €
Total herramientas		0 €	X €

Tabla 3.15: Costes de herramientas y servicios de desarrollo (OpEx).

Resumen del presupuesto (TCO)

Categoría	Tipo	Coste real	Valor de mercado
Infraestructura y licencias	CapEx	0,00 €	X €
Personal	OpEx	3722,40 €	X €
Herramientas y servicios	OpEx	0,00 €	X €
TCO total		3722,40 €	X €

Tabla 3.16: Resumen del presupuesto del proyecto (TCO = CapEx + OpEx).

El coste real del proyecto está determinado principalmente por los costes de personal asociados al contrato de apoyo técnico vinculado al proyecto TASOVA PLUS. El resto de recursos empleados no han supuesto costes adicionales debido al uso de herramientas de código abierto y servicios con planes gratuitos. Cabe destacar que, aunque el desarrollo continuó posteriormente fuera del periodo financiado, dicho esfuerzo adicional no representa un coste económico directo dentro del presupuesto del proyecto.

3.10 Cierre del proyecto y lecciones aprendidas

Al finalizar el proyecto, todos los paquetes de trabajo definidos en la EDT han sido completados. Las fases planificadas se han ejecutado en su totalidad, sin necesidad de descartar ningún paquete de trabajo. Los cambios relevantes realizados durante el desarrollo han quedado registrados en la Sección 3.7, manteniendo la coherencia con el alcance inicial definido.

La desviación temporal global del proyecto se ha concentrado principalmente en las fases de desarrollo e integración del generador automático y pruebas y validación de la solución, debido a la complejidad añadida durante la integración con componentes externos y a la necesidad de realizar iteraciones adicionales para asegurar el correcto funcionamiento del sistema.

Lección	Qué se haría distinto
----------------	------------------------------

La estimación inicial del esfuerzo necesario para las fases de integración y validación fue inferior al esfuerzo realmente requerido.	Realizar una estimación más conservadora, incluyendo un margen adicional para problemas derivados de dependencias externas y correcciones durante la fase final.
El conocimiento inicial sobre algunas tecnologías utilizadas en el proyecto no era suficiente para prever completamente las dificultades de integración.	Dedicar una fase inicial más extensa al estudio de las herramientas y componentes externos antes de comenzar la implementación.
La estrategia de pruebas podría haberse definido con mayor profundidad desde las primeras etapas del desarrollo.	Diseñar desde el inicio una planificación de pruebas más completa, identificando las funcionalidades críticas y los casos de validación necesarios.
La acumulación progresiva de cambios durante el desarrollo generó cierta deuda técnica.	Realizar revisiones y refactorizaciones periódicas del código para mantener una arquitectura más mantenible durante todo el ciclo de desarrollo.

Tabla 3.17: Lecciones aprendidas al cierre del proyecto.

En conjunto, el proyecto se ha completado con una dedicación real de 980 horas, de las cuales 910 corresponden al desarrollo y 70 a la elaboración de la memoria. Aunque la planificación inicial se vio afectada por un periodo de inactividad y por varias iteraciones adicionales de integración y validación, todos los paquetes de trabajo definidos en la EDT fueron completados.

Con el proyecto planificado, ejecutado y cerrado, el siguiente capítulo precisa qué debe hacer exactamente la solución desarrollada. El Capítulo 4 presenta el catálogo de requisitos funcionales y no funcionales que sirve como base para el diseño, la implementación y la validación del sistema.

4 ANÁLISIS DE REQUISITOS

4.1 Descripción del sistema

El sistema desarrollado consiste en un generador automático de modelos de características integrado dentro del ecosistema de herramientas asociado a la variabilidad software. La solución consiste en una herramienta que sigue una arquitectura compatible con Flamapy y su integración dentro de UVLHub, permitiendo que los usuarios puedan configurar y ejecutar el proceso de generación mediante una interfaz web.

El objetivo principal del sistema es facilitar la creación automática de modelos de características expresados en UVL con distintos niveles de complejidad y expresividad. Estos modelos permiten disponer de conjuntos de prueba más completos para la evaluación, validación y mejora de herramientas relacionadas con la Ingeniería de Líneas de Productos Software (SPL), especialmente aquellas orientadas al análisis de variabilidad. De esta forma, el sistema aborda la necesidad de disponer de modelos con características avanzadas que permitan evaluar el comportamiento de dichas herramientas en diferentes escenarios.

La entrada principal del sistema consiste en una configuración definida por el usuario mediante los parámetros disponibles en la interfaz proporcionada por UVLHub. Estos parámetros permiten establecer las características del modelo que se desea generar, incluyendo aspectos relacionados con su estructura jerárquica, atributos y restricciones.

Como resultado, el sistema produce modelos expresados en el lenguaje UVL con los niveles de expresividad soportados. Estos modelos pueden visualizarse directamente tras finalizar el proceso de generación dentro de la plataforma UVLHub y también pueden descargarse como ficheros independientes con extensión `.uvl` para su posterior utilización en otras herramientas del ecosistema de variabilidad software.

El sistema puede dividirse principalmente en los siguientes subsistemas:

- **Subsistema de configuración del generador:** responsable de gestionar los parámetros definidos por el usuario mediante la interfaz web proporcionada en UVLHub. Este subsistema permite establecer las características del modelo que se desea generar y preparar la configuración necesaria para ejecutar el proceso desde el navegador.
- **Subsistema de generación automática:** constituye el núcleo funcional del sistema y es el encargado de producir los modelos de características a partir de

la configuración proporcionada, haciendo uso de los componentes y mecanismos disponibles dentro del framework Flamapy.

- **Subsistema de integración con UVLHub:** encargado de conectar la operación generadora desarrollada con el módulo *generator* de UVLHub, permitiendo que la funcionalidad de la herramienta pueda ser utilizada dentro de la plataforma mediante los mecanismos definidos por esta.

Los actores principales que interactúan con el sistema son los siguientes:

- **Usuario investigador o desarrollador:** configura los parámetros del generador y obtiene modelos UVL para su utilización en procesos de evaluación, experimentación o validación de herramientas de variabilidad software. El acceso al generador no requiere autenticación previa.
- **Herramientas de análisis de variabilidad:** utilizan los modelos generados como entrada para realizar diferentes tareas relacionadas con el análisis y procesamiento de modelos de características.

4.2 Requisitos de información

El sistema gestiona la información necesaria para configurar el proceso de generación automática de modelos de características. Esta información se representa mediante un conjunto de entidades de configuración que permiten definir los parámetros utilizados durante la generación.

La entidad principal del modelo conceptual es **FmgeneratorModel**, que agrupa las diferentes configuraciones específicas del generador. Cada una de estas configuraciones representa un aspecto concreto del modelo que se desea generar, como la nomenclatura, los niveles de expresividad UVL soportados, las características, la estructura jerárquica, las restricciones o los atributos.

Estas entidades son utilizadas posteriormente por la operación de generación automática para producir modelos de características expresados en UVL. La operación de generación no se considera una entidad de información, sino una funcionalidad del sistema que será descrita en los requisitos funcionales.

RI-01: FmgeneratorModel

La entidad **FmgeneratorModel** representa la configuración global del generador automático de modelos de características. Contiene los parámetros generales del proceso y las configuraciones específicas asociadas a cada aspecto del modelo generado.

Atributo	Tipo	Descripción
num_models	int	Número de modelos que se desean generar.
seed	int	Semilla utilizada para controlar la generación aleatoria.
ensure_satisfiable	bool	Indica si los modelos generados deben ser satisfacibles.
naming	NamingConfig	Configuración relacionada con la nomenclatura de los elementos generados.
levels	LevelConfig	Configuración de los niveles de expresividad soportados por UVL.
features	FeaturesConfig	Configuración relacionada con la generación de características.
hierarchy	HierarchyConfig	Configuración de la estructura jerárquica del modelo.
constraints	ConstraintsConfig	Configuración de las restricciones generadas.
attributes	AttributesConfig	Configuración de los atributos asociados a las características.

Tabla 4.1: Atributos del requisito de información RI-01.

RI-02: NamingConfig

La entidad **NamingConfig** representa la configuración utilizada para definir la nomenclatura de los elementos generados por el sistema.

Atributo	Tipo	Descripción
name_prefix	str	Prefijo utilizado para los nombres generados.
include_feature_count_suffix	bool	Indica si se incluye el número de características en el nombre del modelo generado.
include_constraint_count_suffix	bool	Indica si se incluye el número de restricciones en el nombre del modelo generado.

Atributo	Tipo	Descripción
----------	------	-------------

Tabla 4.2: Atributos del requisito de información RI-02.

RI-03: LevelConfig

La entidad **LevelConfig** representa los niveles de expresividad UVL que estarán disponibles durante el proceso de generación.

Atributo	Tipo	Descripción
boolean_level	bool	Indica si se habilita la generación de modelos booleanos.
arithmetic_level	bool	Indica si se habilitan elementos aritméticos en los modelos generados.
type_level	bool	Indica si se permite el uso de tipos en características y atributos.
group_cardinality	bool	Indica si se habilitan cardinalidades de grupo.
feature_cardinality	bool	Indica si se habilitan cardinalidades individuales de características.
aggregate_functions	bool	Indica si se permite la generación de funciones agregadas.
string_constraints	bool	Indica si se habilitan restricciones sobre valores de tipo cadena.

Tabla 4.3: Atributos del requisito de información RI-03.

RI-04: FeaturesConfig

La entidad **FeaturesConfig** contiene los parámetros relacionados con la generación de características del modelo.

Atributo	Tipo	Descripción
min_features	int	Número mínimo de características que puede contener el modelo generado.

Atributo	Tipo	Descripción
max_features	int	Número máximo de características que puede contener el modelo generado.
dist_boolean	float	Probabilidad de generación de características booleanas.
dist_integer	float	Probabilidad de generación de características enteras.
dist_real	float	Probabilidad de generación de características reales.
dist_string	float	Probabilidad de generación de características de tipo cadena.
min_feature_cardinality	int / list[int]	Valor mínimo permitido para las cardinalidades de características.
max_feature_cardinality	int / list[int]	Valor máximo permitido para las cardinalidades de características.
prob_feature_cardinality	float	Probabilidad de aplicar cardinalidades a las características.

Tabla 4.4: Atributos del requisito de información RI-04.

RI-05: HierarchyConfig

La entidad **HierarchyConfig** representa la configuración relacionada con la estructura jerárquica del modelo generado.

Atributo	Tipo	Descripción
max_tree_depth	int	Profundidad máxima permitida para la jerarquía del modelo.
dist_optional	float	Probabilidad de generar relaciones opcionales.
dist_mandatory	float	Probabilidad de generar relaciones obligatorias.
dist_alternative	float	Probabilidad de generar relaciones alternativas.
dist_or	float	Probabilidad de generar relaciones OR.
group_cardinality_min	int	Cardinalidad mínima para grupos de características.

Atributo	Tipo	Descripción
group_cardinality_max	int	Cardinalidad máxima para grupos de características.
dist_group_cardinality	float	Probabilidad de generar relaciones con cardinalidad de grupo.

Tabla 4.5: Atributos del requisito de información RI-05.

RI-06: ConstraintsConfig

La entidad **ConstraintsConfig** contiene los parámetros utilizados para definir la generación de restricciones entre elementos del modelo.

Atributo	Tipo	Descripción
min_constraints	int	Número mínimo de restricciones generadas.
max_constraints	int	Número máximo de restricciones generadas.
extra_constraint_representativeness	float	Factor utilizado para aumentar la representatividad de las restricciones.
min_vars_per_constraint	int	Número mínimo de variables utilizadas en una restricción.
max_vars_per_constraint	int	Número máximo de variables utilizadas en una restricción.
ctc_dist_boolean	float	Probabilidad de generar restricciones booleanas.
ctc_dist_integer	float	Probabilidad de generar restricciones enteras.
ctc_dist_real	float	Probabilidad de generar restricciones reales.
ctc_dist_string	float	Probabilidad de generar restricciones sobre cadenas.
prob_not	float	Probabilidad de aplicar negación en restricciones booleanas.
prob_and	float	Probabilidad de utilizar operadores AND.
prob_or_ct	float	Probabilidad de utilizar operadores OR.

Atributo	Tipo	Descripción
prob_implication	float	Probabilidad de utilizar implicaciones.
prob_equivalence	float	Probabilidad de utilizar equivalencias.
prob_sum	float	Probabilidad de utilizar operaciones de suma.
prob_subtract	float	Probabilidad de utilizar operaciones de resta.
prob_multiply	float	Probabilidad de utilizar operaciones de multiplicación.
prob_divide	float	Probabilidad de utilizar operaciones de división.
prob_equals	float	Probabilidad de utilizar comparaciones de igualdad.
prob_less	float	Probabilidad de utilizar comparaciones menor que.
prob_greater	float	Probabilidad de utilizar comparaciones mayor que.
prob_less_equals	float	Probabilidad de utilizar comparaciones menor o igual que.
prob_greater_equals	float	Probabilidad de utilizar comparaciones mayor o igual que.
prob_sum_function	float	Probabilidad de utilizar funciones agregadas de suma.
prob_avg_function	float	Probabilidad de utilizar funciones agregadas de promedio.
prob_len_function	float	Probabilidad de utilizar funciones de longitud sobre cadenas.

Tabla 4.6: Atributos del requisito de información RI-06.

RI-07: AttributesConfig

La entidad **AttributesConfig** representa la configuración relacionada con los atributos asociados a las características generadas.

Atributo	Tipo	Descripción
random_attributes	bool	Indica si los atributos deben generarse automáticamente o definirse manualmente.
min_attributes	int / None	Número mínimo de atributos generados en modo aleatorio.
max_attributes	int / None	Número máximo de atributos generados en modo aleatorio.
dist_boolean_atr	float	Probabilidad de seleccionar el tipo booleano en la generación aleatoria de atributos.
dist_integer_atr	float	Probabilidad de seleccionar el tipo entero en la generación aleatoria de atributos.
dist_real_atr	float	Probabilidad de seleccionar el tipo real en la generación aleatoria de atributos.
dist_string_atr	float	Probabilidad de seleccionar el tipo cadena en la generación aleatoria de atributos.
attributes_list	list[dict]	Lista de atributos definidos manualmente para la generación. Cada definición puede incluir su tipo, dominio, probabilidad de asociación y participación en restricciones.
attribute_attach_prob	list[float]	Probabilidades de asociación de atributos a características.
attribute_in_constraint	list[bool]	Indica si cada atributo definido manualmente puede participar en restricciones.

Tabla 4.7: Atributos del requisito de información RI-07.

4.3 Requisitos funcionales

El sistema debe proporcionar las funcionalidades necesarias para permitir la configuración, ejecución y obtención de modelos de características generados automáticamente. Los requisitos funcionales identificados se muestran en la Tabla 4.8.

ID	Nombre	Descripción	Prior.
RF-01	Configuración del generador	El sistema debe permitir al usuario definir los parámetros necesarios para el proceso de generación, incluyendo aspectos relacionados con las características, niveles de expresividad, estructura jerárquica, restricciones y atributos del modelo generado.	Alta
RF-02	Validación de configuración	El sistema debe comprobar que los parámetros introducidos por el usuario constituyen una configuración válida antes de iniciar el proceso de generación. Las restricciones y condiciones específicas aplicables a dicha validación se describen en la subsección 4.5.	Alta
RF-03	Generación automática de modelos UVL	El sistema debe permitir generar automáticamente modelos de características expresados en UVL a partir de la configuración proporcionada por el usuario.	Alta
RF-04	Soporte de niveles de expresividad UVL	El sistema debe permitir generar modelos incorporando diferentes niveles de expresividad del lenguaje UVL, incluyendo construcciones booleanas, aritméticas y de tipos cuando hayan sido seleccionadas en la configuración.	Alta

ID	Nombre	Descripción	Prior.
RF-05	Visualización del modelo generado	El sistema debe mostrar el modelo de características generado dentro de la plataforma UVLHub tras finalizar correctamente el proceso de generación.	Media
RF-06	Descarga del modelo generado	El sistema debe permitir descargar el modelo generado como un fichero independiente con extensión <code>.uvl</code> para su posterior utilización en otras herramientas del ecosistema de variabilidad software.	Alta

Tabla 4.8: Requisitos funcionales del sistema.

4.4 Requisitos no funcionales

ID	Nombre	Descripción	Prior.
RNF-01	Arquitectura modular y mantenibilidad	El sistema debe mantener una estructura modular basada en la arquitectura de extensiones de Flamapy, siguiendo las convenciones establecidas por el ecosistema. Esta organización debe facilitar el mantenimiento del código, la incorporación de nuevas funcionalidades y la integración con otros componentes del framework.	Alta
RNF-02	Compatibilidad con modelos UVL	Los modelos generados deben cumplir la especificación UVL soportada por las herramientas del ecosistema de variabilidad software, permitiendo su posterior utilización en procesos de análisis y evaluación de modelos de características.	Alta

ID	Nombre	Descripción	Prior.
RNF-03	Ejecución eficiente del proceso de generación	El sistema debe minimizar la carga computacional sobre el servidor mediante la ejecución del proceso de generación en el entorno del cliente cuando sea posible. Para ello, se emplean tecnologías como Pyodide y WebAssembly (WASM), permitiendo ejecutar el código del generador directamente en el navegador y reduciendo los tiempos de respuesta asociados a la comunicación y procesamiento remoto.	Alta
RNF-04	Usabilidad de la interfaz de configuración	La interfaz proporcionada para configurar el generador debe permitir al usuario seleccionar y comprender los parámetros disponibles de forma clara, facilitando la creación de configuraciones válidas y reduciendo posibles errores durante el proceso de configuración.	Media
RNF-05	Calidad del software	El código desarrollado debe cumplir los mecanismos de control de calidad establecidos en el proyecto, incluyendo las comprobaciones automáticas mediante integración continua y las herramientas de análisis estático utilizadas para mejorar la legibilidad y mantenibilidad del código.	Alta

Tabla 4.9: Requisitos no funcionales del sistema.

4.5 Reglas de negocio

ID	Nombre	Descripción
RN-01	Validación de parámetros generales	Los parámetros generales del generador deben cumplir las restricciones mínimas necesarias para permitir la ejecución del proceso de generación. El número de modelos a generar (<code>num_models</code>) debe encontrarse comprendido entre 1 y 1000. Además, la semilla utilizada para controlar la aleatoriedad del proceso (<code>seed</code>) debe ser un valor positivo.
RN-02	Validación de niveles de expresividad	Los niveles de expresividad del generador deben respetar la jerarquía establecida entre niveles principales (<i>major levels</i>) y niveles secundarios (<i>minor levels</i>). El nivel booleano (<code>boolean_level</code>) constituye el nivel base del generador y debe permanecer siempre habilitado. La activación de un nivel principal implica la disponibilidad de los niveles secundarios asociados. Por otro lado, un nivel secundario únicamente puede activarse cuando el nivel principal necesario para soportar su funcionalidad se encuentra habilitado. En concreto, los niveles deben cumplir las siguientes dependencias: ■ El nivel aritmético (<code>arithmetic_level</code>) permite la generación de expresiones numéricas y es necesario para habilitar los niveles secundarios relacionados con cardinalidades, funciones agregadas y operaciones aritméticas. ■ El nivel de tipos (<code>type_level</code>) requiere la activación previa del nivel aritmético. ■ Los niveles secundarios no pueden permanecer activos si el nivel del que dependen está deshabilitado.

ID	Nombre	Descripción
RN-03	Validación de configuración del árbol de características	La configuración del árbol de características generado debe cumplir las restricciones necesarias para garantizar una estructura válida del modelo. Estas validaciones afectan al número de características, la estructura jerárquica y las distribuciones probabilísticas asociadas. Las restricciones establecidas son las siguientes: ▪ Los parámetros <code>min_features</code> y <code>max_features</code> deben ser mayores o iguales que uno, y el número mínimo de características no puede superar al número máximo configurado. ▪ La profundidad máxima del árbol (<code>max_tree_depth</code>) debe encontrarse comprendida entre uno y el número máximo de características configurado (<code>max_features</code>). ▪ Las probabilidades asociadas a las relaciones jerárquicas deben encontrarse entre 0 y 1 y formar una distribución cuya suma sea exactamente uno. ▪ Cuando se habilite la cardinalidad de características (<code>feature_cardinality</code>), los parámetros asociados deben cumplir las siguientes restricciones: • Los valores <code>min_feature_cardinality</code> y <code>max_feature_cardinality</code> deben ser mayores o iguales que uno, y el valor mínimo no puede superar al máximo. • La probabilidad de asignación de cardinalidad (<code>prob_feature_cardinality</code>) debe encontrarse dentro del rango comprendido entre 0 y 1.

ID	Nombre	Descripción
		■ Cuando se habilite la cardinalidad de grupos (<code>group_cardinality</code>), los parámetros asociados deben cumplir las siguientes restricciones: • Los valores <code>group_cardinality_min</code> y <code>group_cardinality_max</code> deben ser mayores o iguales que uno, y el valor mínimo no puede superar al máximo. • Ambos valores no pueden superar el número máximo de características configurado (<code>max_features</code>). ■ La distribución de tipos de características únicamente debe estar disponible cuando el nivel de tipos (<code>type_level</code>) esté habilitado y sus probabilidades deben sumar exactamente uno.

ID	Nombre	Descripción
RN-04	Validación de configuración de restricciones	La configuración de restricciones entre características y atributos debe cumplir las siguientes reglas: ■ Los límites del número de restricciones (<code>min_constraints</code> y <code>max_constraints</code>) deben ser mayores o iguales que uno y el mínimo no puede superar al máximo. ■ El número mínimo de variables utilizadas en una restricción (<code>min_vars_per_constraint</code>) debe ser menor o igual que el número máximo permitido (<code>max_vars_per_constraint</code>), siendo ambos valores mayores o iguales que uno. ■ El parámetro <code>extra_constraint_representativeness</code> debe ser mayor o igual que uno. ■ Las probabilidades utilizadas en la generación de restricciones deben encontrarse dentro del rango comprendido entre 0 y 1. ■ La distribución asociada a los tipos de restricciones (<code>ctc_dist_boolean</code>, <code>ctc_dist_integer</code>, <code>ctc_dist_real</code> y <code>ctc_dist_string</code>) debe sumar exactamente uno cuando los niveles de expresividad correspondientes estén habilitados. ■ La distribución de operadores booleanos (<code>prob_and</code>, <code>prob_or_ct</code>, <code>prob_implication</code> y <code>prob_equivalence</code>) debe sumar exactamente uno. La probabilidad asociada al operador NOT (<code>prob_not</code>) se considera de forma independiente y debe encontrarse dentro del rango comprendido entre 0 y 1.

ID	Nombre	Descripción
		■ Cuando el nivel aritmético esté habilitado y las funciones agregadas estén desactivadas, la distribución de operaciones aritméticas básicas (<code>prob_sum</code>, <code>prob_subtract</code>, <code>prob_multiply</code> y <code>prob_divide</code>) debe sumar uno. ■ Cuando las funciones agregadas estén activadas, las probabilidades <code>prob_sum_function</code> y <code>prob_avg_function</code> se incorporan a la distribución de operaciones aritméticas. En este caso, la suma conjunta de las cuatro operaciones básicas y las dos funciones agregadas debe ser igual a uno, admitiendo una tolerancia absoluta de 10^{-6} debida a la representación numérica. ■ Cuando el nivel aritmético esté habilitado, la distribución de operadores de comparación (<code>prob_equals</code>, <code>prob_less</code>, <code>prob_greater</code>, <code>prob_less_equals</code> y <code>prob_greater_equals</code>) debe sumar exactamente uno.

ID	Nombre	Descripción
RN-05	Validación de configuración de atributos	La configuración de atributos debe cumplir las restricciones asociadas al modo de generación seleccionado: ■ En modo de generación automática (<code>random_attributes</code>), los límites <code>min_attributes</code> y <code>max_attributes</code> deben estar definidos, deben ser mayores o iguales que cero y el mínimo no puede superar al máximo. ■ Cuando se utiliza la generación automática de atributos, las probabilidades <code>dist_boolean_atr</code>, <code>dist_integer_atr</code>, <code>dist_real_atr</code> y <code>dist_string_atr</code> deben encontrarse en el intervalo $[0, 1]$ y formar una distribución cuya suma sea uno. ■ En modo de definición manual, cada atributo debe cumplir las restricciones asociadas a su tipo: • Los atributos booleanos deben definir al menos un valor posible. • Los atributos de tipo entero, real o cadena deben establecer un valor mínimo y máximo válidos, siendo el mínimo menor o igual que el máximo. ■ En modo manual, los límites <code>min_attributes</code> y <code>max_attributes</code> deben tomar el valor <code>None</code>. ■ En modo manual, la probabilidad de asociación de cada atributo, almacenada mediante el parámetro <code>attach_probability</code>, debe pertenecer al intervalo $[0, 1]$. Esta comprobación se realiza durante la validación de <code>AttributesConfig</code>.

Tabla 4.10: Reglas de negocio del dominio.

4.6 Restricciones del proyecto

ID	Tipo	Descripción
RES-01	Tecnológica	El desarrollo debe integrarse dentro del ecosistema existente de herramientas de variabilidad software, utilizando como base el framework Flamapy y la plataforma UVLHub. Esta condición limita la arquitectura de la solución, ya que el generador debe seguir los mecanismos de extensión definidos por Flamapy, adaptarse a la estructura proporcionada por UVLHub y utilizar las librerías propias del framework y componentes externos necesarios para la generación y representación de modelos de características.
RES-02	Tecnológica / interoperabilidad	Los modelos generados deben utilizar el lenguaje UVL como formato de salida, debido a que constituye el estándar empleado dentro del ecosistema de herramientas con el que se integra el proyecto.
RES-03	Organizativa	El desarrollo debe realizarse dentro del contexto del grupo de investigación y de los proyectos software existentes asociados, respetando los procedimientos, recursos y estructuras de trabajo establecidos.
RES-04	Temporal	El proyecto está condicionado por la planificación académica del TFG, estableciendo un periodo limitado para realizar las fases de análisis, diseño, implementación, validación y documentación de la solución.

Tabla 4.11: Restricciones del proyecto.

4.7 Casos de uso

4.7.1 Diagrama general de casos de uso

4.7.2 Diagrama general de casos de uso

El diagrama de casos de uso representa las principales interacciones entre los actores externos y el sistema generador de modelos UVL. El usuario investigador o

desarrollador es el encargado de configurar los parámetros del generador y, una vez finalizado el proceso, puede visualizar los modelos generados dentro de la plataforma o descargarlos para su posterior utilización. El sistema se encarga de validar la configuración proporcionada y ejecutar el proceso de generación automática de modelos de características UVL. Los modelos descargados pueden ser utilizados posteriormente por herramientas externas de análisis de variabilidad para realizar procesos de evaluación y análisis sobre los modelos generados.

La Figura 4.1 muestra el diagrama general de casos de uso del sistema.

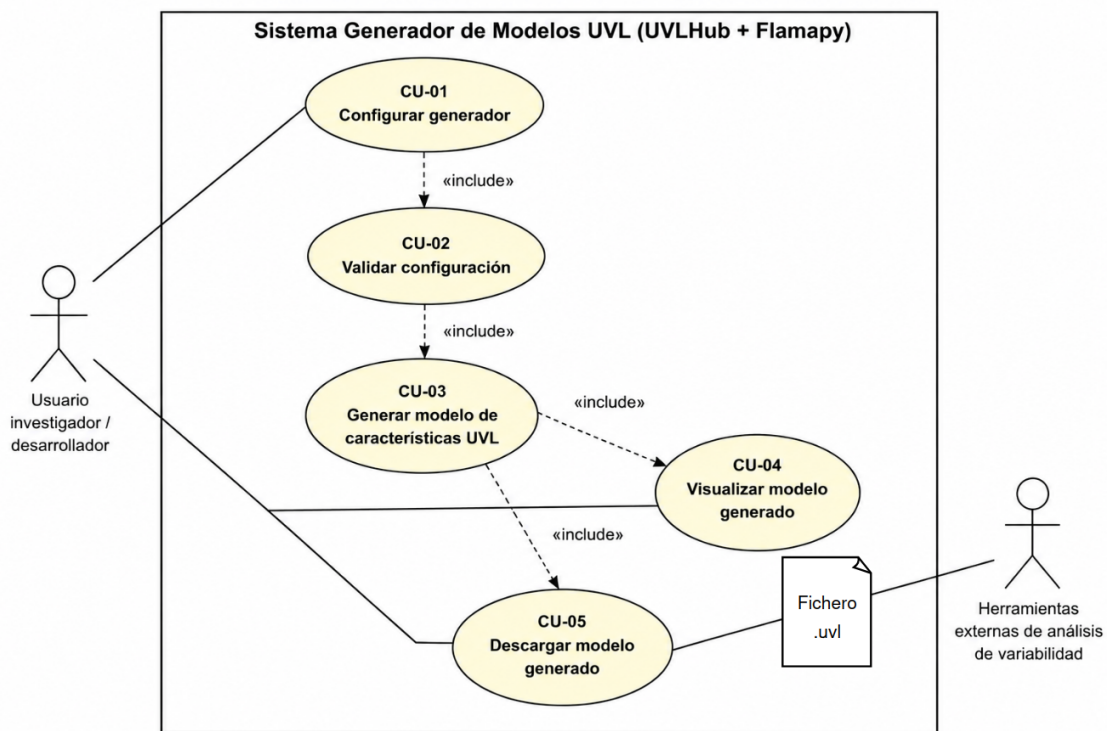


Figura 4.1: Diagrama general de casos de uso del sistema. Elaboración propia

4.7.3 Especificación de casos de uso principales

CU-01 — Configurar generador	
Actores	Usuario investigador o desarrollador.
Descripción	Permite al usuario definir la configuración necesaria para el proceso de generación automática de modelos de características UVL. El usuario puede seleccionar los niveles de expresividad y establecer los parámetros relacionados con características, estructura jerárquica, restricciones y atributos.
Precondiciones	El usuario debe acceder a la interfaz de configuración del generador disponible dentro de UVLHub.
Flujo principal	1. El usuario accede al formulario de configuración del generador. 2. El usuario establece los parámetros asociados a la generación del modelo en cada <i>step</i>. 3. El sistema registra la configuración proporcionada.
Flujo alternativo	Si los valores introducidos no cumplen las restricciones establecidas, el sistema informa al usuario de los errores detectados durante la introducción de dichos valores o antes de pasar al siguiente <i>step</i> .
Postcondiciones	Si la configuración de cada <i>step</i> es válida, se procede en el último <i>step</i> a la generación de los modelos.

Tabla 4.12: Especificación del caso de uso CU-01.

CU-02 — Validar configuración	
Actores	Sistema generador de modelos UVL.
Descripción	Permite comprobar que la configuración proporcionada por el usuario cumple las reglas de negocio definidas antes de ejecutar el proceso de generación automática.
Precondiciones	Debe existir una configuración del generador proporcionada por el usuario.
Flujo principal	1. El sistema recibe la configuración definida por el usuario y analiza sus parámetros. 3. El sistema comprueba las restricciones asociadas a características, jerarquía, restricciones y atributos en cada <i>step</i>, respectivamente. 4. El sistema valida las distribuciones probabilísticas y dependencias entre parámetros.
Flujo alternativo	Si alguna regla de negocio no se cumple, el sistema devuelve los errores detectados e impide continuar con la generación hasta corregir la configuración.
Postcondiciones	Si la configuración de cada <i>step</i> es válida, se le permite al usuario pasar al siguiente <i>step</i> .

Tabla 4.13: Especificación del caso de uso CU-02.

CU-03 — Generar modelo de características UVL

Actores	Sistema generador de modelos UVL.
Descripción	Permite generar automáticamente un modelo de características expresado en UVL a partir de una configuración previamente validada.
Precondiciones	Las dependencias necesarias deben ser cargadas correctamente al acceder al primer <i>step</i> . Además, la configuración proporcionada por el usuario debe ser validada en cada <i>step</i> .
Flujo principal	1. Cuando el usuario llega al último <i>step</i>, el sistema inicializa el proceso de generación utilizando la configuración proporcionada. 2. El sistema genera, para cada modelo y de forma aleatoria, la estructura jerárquica del modelo de características, incorporando los niveles de expresividad, atributos y restricciones pertinentes, cumpliendo con la configuración seleccionada por el usuario. 3. El sistema genera la representación final de los modelos en formato UVL y los almacena temporalmente para su visualización o descarga.
Flujo alternativo	Si se produce un error durante el proceso de generación, el sistema informa del fallo y no genera los modelos solicitados.
Postcondiciones	Existe un/unos modelo/s de características UVL generado/s correctamente disponible/s para su posterior visualización o descarga.

Tabla 4.14: Especificación del caso de uso CU-03.

CU-04 — Visualizar modelo generado	
Actores	Usuario investigador o desarrollador.
Descripción	Permite al usuario consultar visualmente el modelo de características generado dentro de la plataforma UVLHub.
Precondiciones	Debe existir al menos un modelo generado correctamente por el sistema.
Flujo principal	1. El sistema carga la representación del modelo generado. 2. El sistema muestra el modelo dentro de la interfaz de UVLHub al hacer clic en el botón correspondiente. 3. El usuario inspecciona la estructura y elementos generados.
Flujo alternativo	Si no se ha podido realizar la generación previa a la visualización, el sistema lanza un error informativo.
Postcondiciones	El usuario puede consultar el modelo generado dentro de la plataforma.

Tabla 4.15: Especificación del caso de uso CU-04.

CU-05 — Descargar modelo generado	
Actores	Usuario investigador o desarrollador.
Descripción	Permite descargar en formato zip los modelos generados como ficheros independientes en formato UVL para su posterior utilización dentro o fuera de la plataforma.
Precondiciones	Debe existir un modelo de características generado correctamente.
Flujo principal	1. El usuario solicita la descarga del modelo generado haciendo clic en el botón correspondiente. 2. El sistema prepara el zip con los ficheros UVL generados. El usuario puede ver el archivo en la bandeja de descargas de su navegador.
Flujo alternativo	Si el modelo no se encuentra disponible o se produce un error durante la generación del fichero, el sistema informa al usuario y cancela la descarga.
Postcondiciones	El modelo UVL queda disponible como fichero independiente para su almacenamiento o análisis mediante herramientas externas.

Tabla 4.16: Especificación del caso de uso CU-05.

4.8 Matriz de trazabilidad

4.8.1 Requisitos funcionales ↔ casos de uso

	CU-01	CU-02	CU-03	CU-04	CU-05
RF-01	×				
RF-02		×			
RF-03			×		
RF-04			×		
RF-05				×	
RF-06					×

Tabla 4.17: Matriz de trazabilidad entre requisitos funcionales y casos de uso.

4.8.2 Requisitos ↔ pruebas

Requisito	Tipo de verificación	Evidencia
RF-01	Integración / sistema	<code>test_full_happy_path_produces_valid_params</code> (<code>test_wizard_flow.py</code>), que comprueba que el usuario puede completar el flujo de configuración y establecer los parámetros necesarios para la generación.
RF-02	Pruebas unitarias	<code>test_step1_rejects_batch_size_that_could_break_generation</code> , <code>test_step2_rejects_minor_levels_without_required_major_level</code> y <code>test_step3_rejects_invalid_relation_distribution</code> (<code>test_validators.py</code>), que comprueban el rechazo de configuraciones inválidas.
RF-03	Prueba de extremo a extremo	<code>test_step6_generates_one_uvl_model_with_pyodide</code> (<code>test_selenium.py</code>), que verifica que el proceso de generación se completa desde la interfaz y produce un modelo.
RF-04	Pruebas de integración	<code>test_everything_on_every_family_represented</code> (<code>test_wizard_outputs.py</code>), que comprueba que las configuraciones relacionadas con los diferentes niveles de expresividad se reflejan en los modelos generados.
RF-05	Prueba de extremo a extremo	<code>test_step6_generates_one_uvl_model_with_pyodide</code> (<code>test_selenium.py</code>), considerando en este caso la comprobación de que el modelo generado se muestra correctamente en la interfaz.
RF-06	Pruebas de integración	<code>test_filename_suffixes_applied_to_generated_files</code> (<code>test_wizard_outputs.py</code>), que comprueba la aplicación de los sufijos configurados a los ficheros generados.
RNF-01	Inspección estructural e integración	Revisión de la separación entre el modelo de configuración, las clases especializadas, la operación <code>GenerateFeatureModel</code> y la integración con <code>Flamapy</code> y <code>UVLHub</code> . Esta revisión se complementa con la ejecución de las pruebas de integración del módulo <code>generator</code> .

Requisito	Tipo de verificación	Evidencia
RNF-02	Validación de formato y prueba de sistema	La prueba <code>test_step6_generates_one_uvl_model_with_pyodide</code> (<code>test_selenium.py</code>) comprueba que el resultado se genera y se entrega en formato UVL. La validez sintáctica del modelo se verifica adicionalmente mediante su lectura o validación con el parser UVL utilizado por Flamapy.
RNF-03	Prueba de extremo a extremo	<code>test_step6_generates_one_uvl_model_with_pyodide</code> (<code>test_selenium.py</code>), que verifica que el proceso de generación se ejecuta en el navegador mediante Pyodide y que el resultado se devuelve a la interfaz. También está el <code>test_generate_feature_model_is_discoverable_by_flamapy</code> , que verifica que Flamapy localiza esta operación mediante <code>DiscoverMetamodels</code>
RNF-04	Comprobación manual de usabilidad	Recorrido manual del <i>wizard</i> de configuración, comprobando que los parámetros se presentan agrupados por funcionalidad, que las opciones dependientes se muestran de forma coherente y que los mensajes de validación permiten corregir una configuración inválida. La organización de las vistas se describe en la Sección 5.4.1.
RNF-05	Integración continua y análisis estático	Ejecución de los flujos de GitHub Actions configurados para el proyecto, incluyendo las comprobaciones de calidad del código y la suite automatizada de pruebas. Los workflows utilizados se muestran en la Figura 3.1 y la suite de pruebas se describe en la Sección 7.2.

Tabla 4.18: Matriz de trazabilidad entre requisitos funcionales y no funcionales y sus verificaciones.

Un mismo caso de prueba puede aportar evidencia para varios requisitos cuando sus comprobaciones cubren aspectos diferentes del sistema. En esos casos, la matriz especifica en cada fila qué propiedad concreta se verifica. Las inspecciones estructurales y las comprobaciones manuales se identifican explícitamente y no se presentan como pruebas automatizadas.

Con los requisitos funcionales, no funcionales, reglas de negocio y restricciones del proyecto definidos y trazados, queda establecido el comportamiento esperado del sistema y las condiciones que debe cumplir la solución desarrollada. El siguiente capítulo aborda cómo satisfacer estos requisitos mediante la definición de la arquitectura del sistema, los componentes principales y las decisiones de diseño adoptadas durante el desarrollo.

5 DISEÑO DE LA SOLUCIÓN

5.1 Modelo conceptual del dominio

El modelo conceptual utilizado para el desarrollo de la solución representa la estructura estándar mediante la cual se implementa un componente compatible con el ecosistema Flamapy. En este caso, dicho modelo ha sido utilizado para representar un generador automático de modelos de características expresados en UVL, definiendo las entidades necesarias para configurar y ejecutar el proceso de generación. La solución se ha desarrollado siguiendo el paradigma de programación orientada a objetos de Python, donde cada entidad del modelo se implementa mediante clases organizadas en archivos con extensión `.py`. Esta estructura permite separar las responsabilidades de los diferentes componentes de la herramienta, diferenciando la configuración del generador de la operación encargada de producir los modelos de características.

La relación del modelo conceptual con el resto del sistema se establece mediante su integración con UVLHub. La clase `GenerateFeatureModel` constituye el punto principal de entrada para la operación de generación y es invocada desde el módulo *generator* de UVLHub cuando el usuario solicita la creación de nuevos modelos de características. Antes de realizar dicha llamada, la plataforma UVLHub se encarga de validar la configuración proporcionada por el usuario, actuando como una barrera que evita que la herramienta generadora reciba parámetros inválidos o incoherentes. En cuanto a la persistencia, el sistema no requiere un mecanismo de almacenamiento permanente, ya que está diseñado para permitir al usuario visualizar o descargar los modelos generados inmediatamente después de cada ejecución y comenzar posteriormente una nueva generación con una configuración diferente.

5.1.1 Estructura del modelo

Modelo conceptual del generador de modelos de características

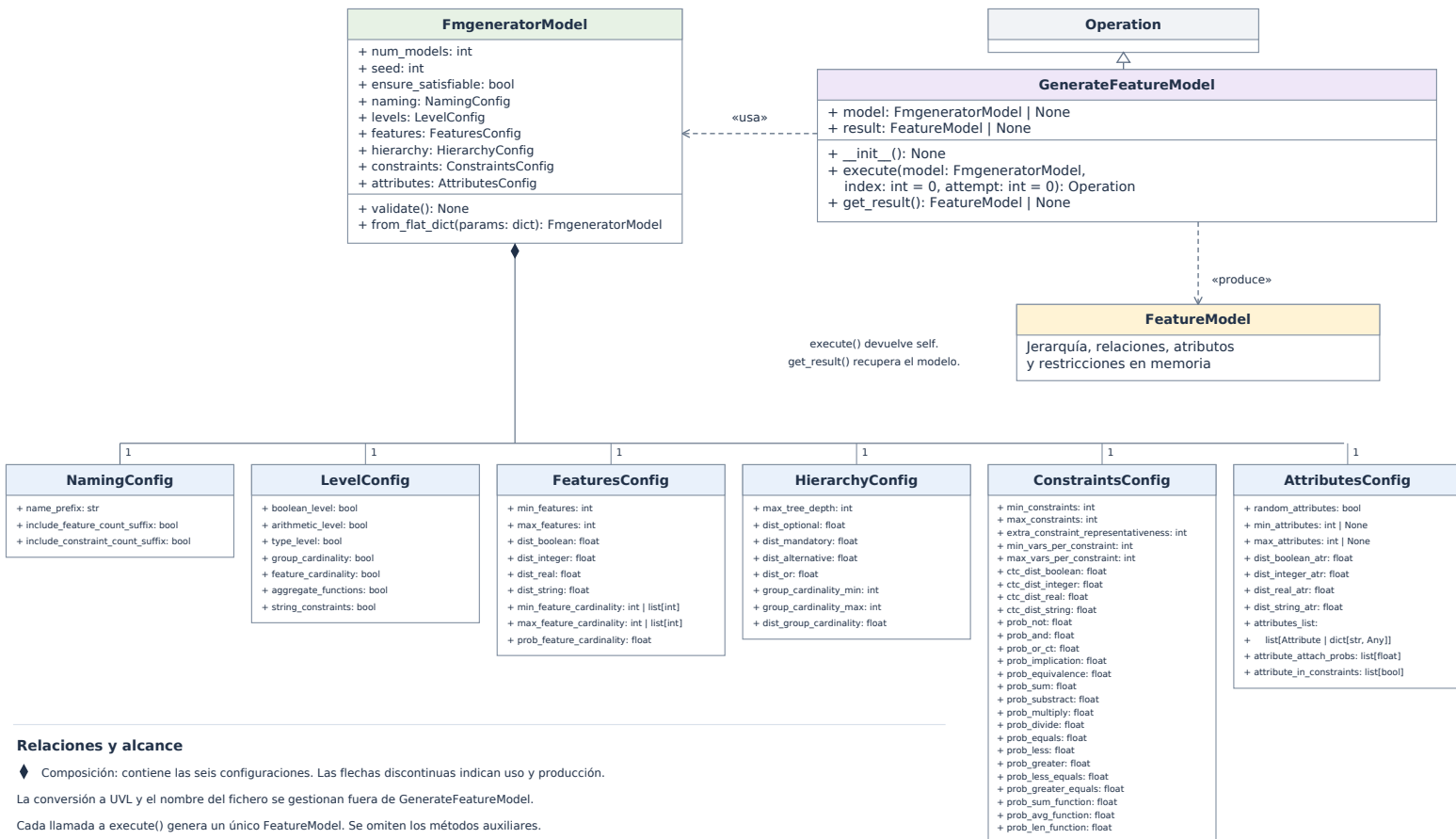


Figura 5.1: Modelado conceptual del sistema. Elaboración propia

El modelo conceptual representa la estructura interna del componente desarrollado siguiendo una arquitectura compatible con Flamapy. En este caso, dicho modelo ha sido adaptado para representar un generador automático de modelos de características expresados en UVL, separando la configuración del generador, la operación encargada de ejecutar la generación y el resultado producido.

Las entidades principales que componen el modelo son las siguientes:

FmgeneratorModel

La entidad **FmgeneratorModel** representa la configuración global del generador de modelos de características. Actúa como entidad raíz del modelo conceptual y agrupa las diferentes configuraciones necesarias para definir las características del modelo que se desea generar, incluyendo aspectos relacionados con la nomenclatura, los niveles de expresividad UVL, la estructura jerárquica, las restricciones y los

atributos. Esta entidad constituye la representación completa del estado del generador antes de iniciar el proceso de generación.

NamingConfig

La entidad **NamingConfig** representa la configuración relacionada con la generación de nombres de los elementos creados durante el proceso de generación. Su finalidad es establecer las reglas utilizadas para definir la nomenclatura de los modelos y sus elementos asociados.

LevelConfig

La entidad **LevelConfig** representa la configuración de los niveles de expresividad disponibles en el lenguaje UVL durante la generación. Esta entidad permite definir las capacidades del lenguaje que estarán habilitadas en los modelos generados, estableciendo las construcciones que podrán aparecer en el resultado final.

FeaturesConfig

La entidad **FeaturesConfig** representa la configuración asociada a la generación de características del modelo. Su responsabilidad es definir las condiciones que controlan la creación de los elementos principales que forman parte de la estructura del modelo de características.

HierarchyConfig

La entidad **HierarchyConfig** representa la configuración relacionada con la estructura jerárquica del modelo de características generado. Esta entidad define los parámetros necesarios para construir el árbol de características y establecer las relaciones jerárquicas entre sus elementos.

ConstraintsConfig

La entidad **ConstraintsConfig** representa la configuración utilizada para la generación de restricciones entre los elementos del modelo. Su objetivo es definir cómo se crean las relaciones adicionales entre características y atributos, permitiendo generar modelos con diferentes niveles de complejidad y expresividad.

AttributesConfig

La entidad **AttributesConfig** representa la configuración asociada a la generación de atributos vinculados a las características del modelo. Esta entidad permite

establecer cómo se incorporan atributos al modelo generado, tanto mediante generación automática como mediante configuraciones definidas manualmente.

GenerateFeatureModel

La clase `GenerateFeatureModel` representa la operación encargada de generar un objeto `FeatureModel` a partir de la configuración definida en `FmgeneratorModel`. Hereda de `Operation` y dispone de un constructor sin parámetros. Su método `execute` recibe la configuración, el índice del modelo y el número de intento, valida los parámetros y coordina la generación de la jerarquía, los atributos y las restricciones.

La configuración recibida se almacena en el atributo `model`, mientras que el objeto generado se guarda en `result`. El método `execute` devuelve la propia operación, y el resultado se obtiene mediante `get_result`. Cada ejecución genera un único modelo de características. La conversión a texto UVL y la asignación del nombre del fichero se realizan fuera de esta clase.

FeatureModel generado

El objeto `FeatureModel` representa el resultado de una ejecución del generador. Contiene la raíz del modelo, la jerarquía de características, sus relaciones, los atributos y las restricciones. Constituye una representación en memoria compatible con el metamodelo de características de Flamapy. Su transformación a texto UVL se realiza posteriormente mediante `UVLWriter`, fuera de la operación `GenerateFeatureModel`.

5.1.2 Restricciones del dominio

- **Dependencia entre niveles de expresividad UVL:** El nivel booleano no puede no existir porque no tiene sentido debido a ser el nivel más básico de los modelos, al igual que no puede no existir el Type Level sin el Arithmetic Level, ya que para crear *constraints* con los tipos proporcionados por este nivel hace falta el Arithmetic Level para procesarlos. A este respecto, hablamos tanto de existencia (a nivel conceptual) como también se comentó anteriormente de activación (a nivel técnico y funcional)
- **Validez estructural de los modelos de características generados:** A nivel conceptual, no se puede determinar como tal la coherencia o validez de los modelos generados. Para poder determinarla, ya existen los motores de razonamiento actuales, como SAT, para modelos únicamente con Boolean Level. Es objetivo de este proyecto proporcionar modelos con mayor expresividad (es decir, que incluyan Arithmetic y Type level) para poder diseñar motores de razonamiento que permitan evaluar la validez de modelos con estos niveles de expresividad.

- **Reproducibilidad mediante el uso de semillas:** El usuario puede establecer la semilla utilizada para controlar la aleatoriedad del proceso de generación. Esta decisión permite repetir una ejecución utilizando la misma semilla y los mismos parámetros de configuración, siempre que se mantengan la versión del generador y las condiciones de ejecución. De esta forma, se facilita la reproducción de experimentos, la comparación de resultados y el análisis de posibles errores. Para ello, la clase `FmgeneratorModel` incorpora el atributo `seed` como parte de la configuración del generador.

5.1.3 Espacio de soluciones / configuraciones

El espacio de configuraciones del generador está determinado por los valores que pueden tomar los diferentes parámetros configurables de la clase *FmgeneratorModel*. Para calcular el número de configuraciones posibles sería necesario revisar los límites entre los cuales puede variar cada parámetro. Debido a que algunos de ellos no presentan un límite superior establecido, como ocurre con el número de características, además de que existen parámetros numéricos reales, el espacio de configuraciones posibles es ilimitado.

Asimismo, el uso de mecanismos pseudoaleatorios permite explorar distintas soluciones para una misma configuración mediante la variación de la semilla. Esto favorece la diversidad de los modelos generados, aunque el uso de semillas diferentes no garantiza que todos los resultados sean distintos.

5.2 Arquitectura del sistema

5.2.1 Visión general

La arquitectura diseñada organiza los diferentes componentes del sistema en capas lógicas, separando las responsabilidades asociadas a la interacción con el usuario, la gestión de la configuración, la generación de modelos y la infraestructura necesaria para su ejecución.

La **capa de presentación** está formada por la interfaz web proporcionada por UVLHub, mediante la cual el usuario configura los diferentes parámetros del generador. Esta interfaz sigue un diseño basado en un flujo tipo *wizard*, permitiendo organizar la configuración del modelo en diferentes pasos y facilitar la comprensión del proceso por parte del usuario.

La **capa de API/controladores** está formada principalmente por las *routes* del módulo *generator* de UVLHub. Estas funciones se encargan de gestionar la comunicación entre los diferentes pasos del flujo de configuración, enviando y recibiendo los

parámetros introducidos por el usuario hasta completar la configuración necesaria para ejecutar el proceso de generación.

La **capa de dominio/lógica** está formada por la lógica interna del generador desarrollado siguiendo la arquitectura de extensiones de Flamapy, junto con los servicios encargados de gestionar la configuración dentro de UVLHub. Por un lado, la herramienta contiene el modelo de configuración y la lógica necesaria para producir modelos de características expresados en UVL. Por otro lado, los servicios se encargan de reconstruir y validar los parámetros recibidos entre los diferentes pasos del generador mediante las *routes*, garantizando que la información utilizada durante la generación sea coherente.

La **capa de infraestructura** está formada por los elementos necesarios para la ejecución y despliegue de la aplicación. Para el desarrollo local se ha utilizado Docker junto con sus correspondientes contenedores y configuraciones, permitiendo construir y levantar el entorno completo de la aplicación de forma reproducible. Además, se ha utilizado Render como plataforma de despliegue principalmente para facilitar al tutor la revisión del funcionamiento de la aplicación durante el desarrollo del proyecto. Asimismo, esta capa incluye el entorno de ejecución basado en WebAssembly mediante Pyodide, utilizado para ejecutar el generador directamente en el navegador del usuario y cargar las dependencias necesarias para su funcionamiento.

No existe una **capa de persistencia** en la arquitectura diseñada, debido a que los modelos generados no se almacenan de forma permanente. El sistema está orientado a que el usuario pueda visualizar o descargar los modelos generados tras cada ejecución y comenzar posteriormente una nueva generación con una configuración diferente.

5.2.2 Diagrama de componentes

El diagrama de componentes mostrado en la Figura 5.2 representa la distribución de los principales módulos del sistema y la comunicación existente entre ellos.

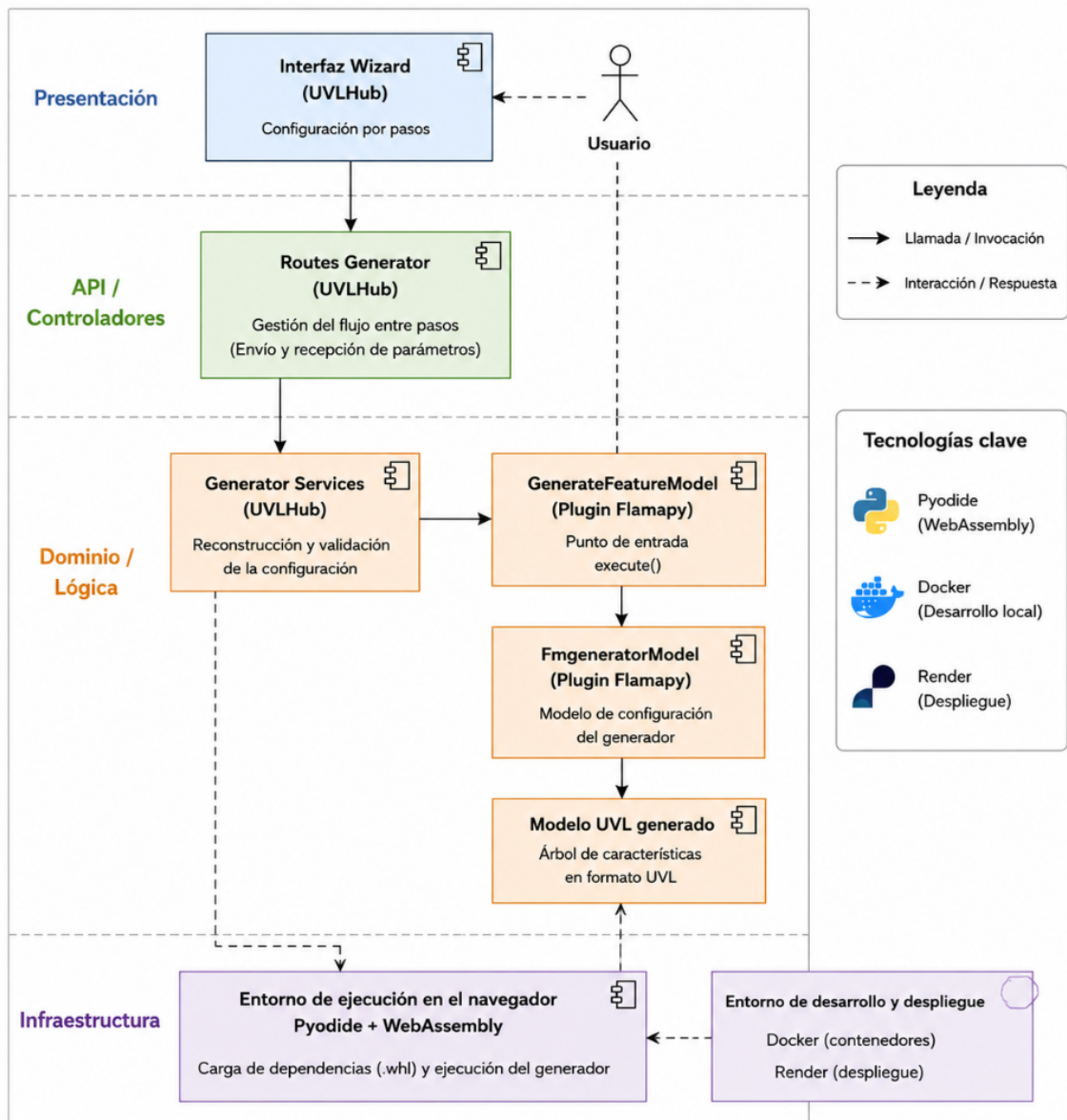


Figura 5.2: Diagrama de componentes del sistema. Elaboración propia

5.2.3 Diagrama de secuencia del flujo principal

1. **Inicio de la configuración:** El usuario accede al generador mediante la interfaz basada en *wizard* proporcionada por UVLHub y comienza a introducir los parámetros necesarios para definir las características del modelo que desea generar.
2. **Comunicación entre los diferentes pasos del *wizard*:** Las *routes* del módulo *generator* reciben los parámetros introducidos por el usuario en cada paso del flujo de configuración y se encargan de enviarlos al siguiente paso junto con la información necesaria para continuar con la configuración del generador.

En este punto ya existen validaciones implementadas en el propio frontend que se encargan de realizar comprobaciones básicas, por ejemplo que la suma de las distribuciones sea exactamente 1, entre otras.

3. **Actualización y validación de los parámetros:** Tras cada paso, las rutas y los servicios internos de UVLHub comprueban los parámetros recibidos y actualizan el diccionario de configuración almacenado en la sesión de Flask. Estas comprobaciones permiten detectar valores inválidos antes de continuar con el asistente. En caso de error, se informa al usuario para que corrija los parámetros correspondientes. En esta fase todavía no se construye la instancia de `FmgeneratorModel`.
4. **Obtención de la configuración y preparación de Pyodide:** En el último paso del asistente, el navegador obtiene la configuración en formato JSON mediante el endpoint `/generator/random/params-json`. Antes de ejecutar el generador, se prepara el entorno de Pyodide y se cargan las dependencias necesarias para ejecutar el código Python en el navegador mediante WebAssembly.
5. **Construcción de la configuración y ejecución del generador:** Dentro de Pyodide, el archivo `fmgen_wrapper.py` utiliza el método `from_flat_dict` para construir la instancia de `FmgeneratorModel` a partir de los parámetros recibidos. Una vez validada la configuración, se ejecuta la operación `GenerateFeatureModel` y los modelos obtenidos se transforman a formato UVL. Si se produce un error durante este proceso, se informa al usuario de que no ha sido posible completar la generación.
6. **Visualización y descarga del resultado:** Una vez generado correctamente el modelo UVL, este se devuelve a la interfaz de UVLHub, permitiendo al usuario visualizarlo en la plataforma o descargarlo para su posterior utilización.

5.3 Patrones de diseño aplicados

- **Arquitectura por capas (*Layered Architecture*):** La solución sigue una organización basada en capas lógicas, separando las responsabilidades asociadas a la interfaz de usuario, comunicación con el sistema, lógica de generación e infraestructura. Este patrón permite reducir el acoplamiento entre componentes y facilitar la evolución del sistema, permitiendo modificar una capa sin afectar al resto siempre que se mantengan sus interfaces de comunicación.
- **Arquitectura basada en componentes independientes (*Component-based Architecture*):** El generador desarrollado se estructura como un componente independiente compatible con el ecosistema Flamapy, utilizando los

mecanismos y modelos proporcionados por el framework para interactuar con él. Esta decisión permite separar la lógica de generación del resto de la aplicación, favoreciendo la reutilización, el mantenimiento y la evolución del sistema.

- **Patrón de interacción *Wizard*:** La interfaz de configuración del generador utiliza un patrón de interacción basado en *wizard*, mediante el cual la introducción de parámetros se divide en diferentes pasos independientes. Este patrón permite organizar una gran cantidad de opciones de configuración de forma progresiva, facilitando la comprensión del usuario y permitiendo realizar validaciones durante el proceso de configuración. Además, se incorporó un panel de progreso que permite consultar en todo momento el estado actual de la configuración realizada.

5.4 Diseño de la interfaz de usuario

5.4.1 Mapa de navegación

El mapa de navegación del generador sigue una estructura secuencial basada en un flujo tipo *wizard*, donde la configuración del modelo de características se divide en diferentes etapas independientes. Esta organización permite separar los diferentes grupos de parámetros configurables según su funcionalidad, facilitando al usuario la definición progresiva de la configuración necesaria para realizar la generación del modelo.

Las vistas principales del sistema son las siguientes:

Step 1: Batch

Esta vista permite establecer los parámetros generales del proceso de generación, incluyendo el número de modelos a generar, la semilla utilizada para controlar la aleatoriedad y la nomenclatura de los modelos generados.

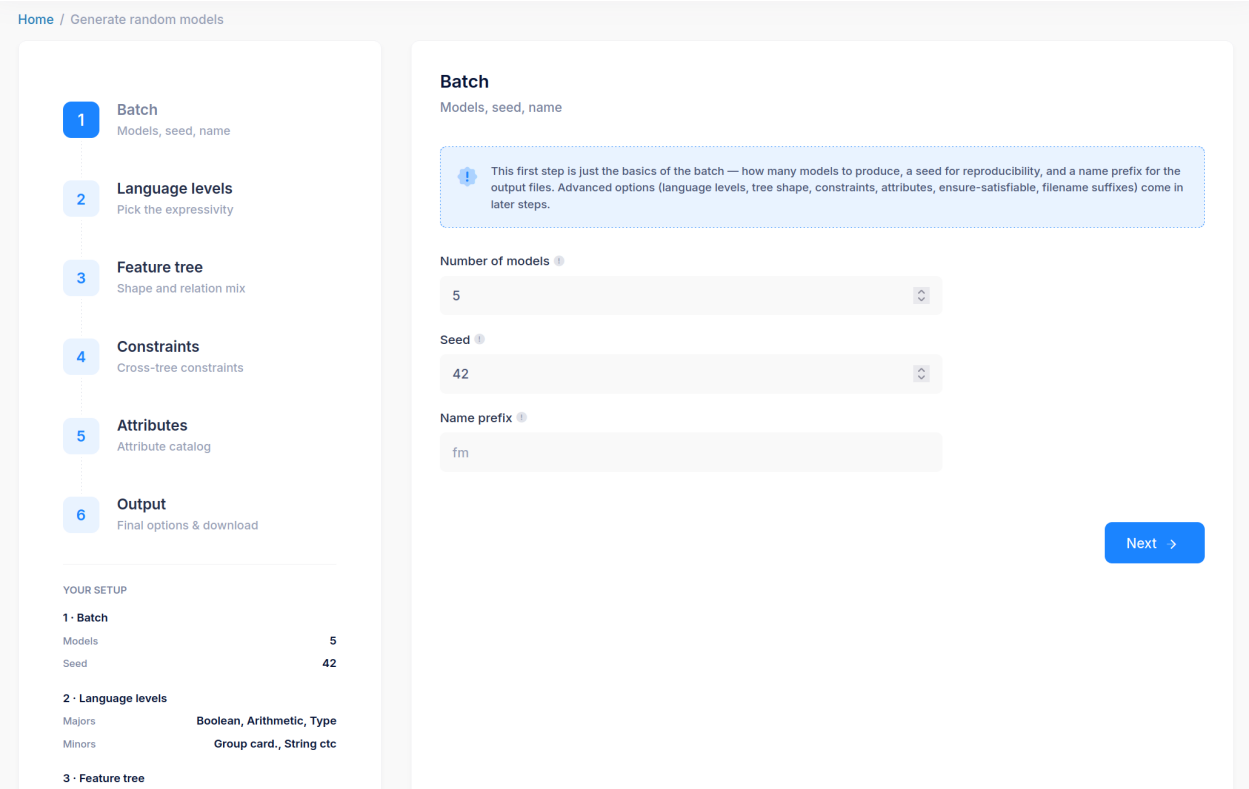


Figura 5.3: Interfaz del Step 1: Batch. Elaboración propia

Step 2: Language levels

Esta vista permite seleccionar los niveles de expresividad UVL que estarán disponibles durante la generación, incluyendo la activación de los diferentes niveles principales y secundarios soportados.

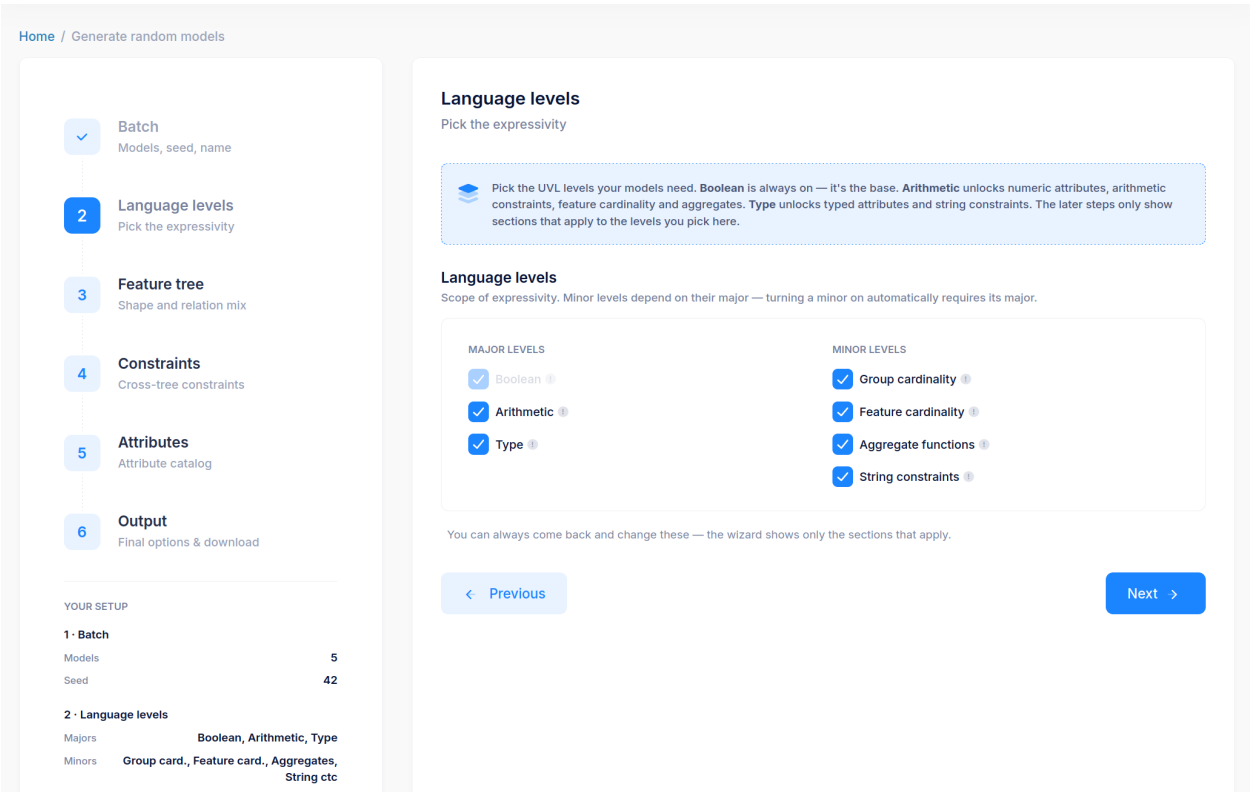


Figura 5.4: Interfaz del Step 2: Language levels. Elaboración propia

Step 3: Feature tree

Esta vista permite configurar los parámetros relacionados con la generación de la estructura del árbol de características, incluyendo el número de características, la profundidad máxima y la distribución de relaciones jerárquicas.

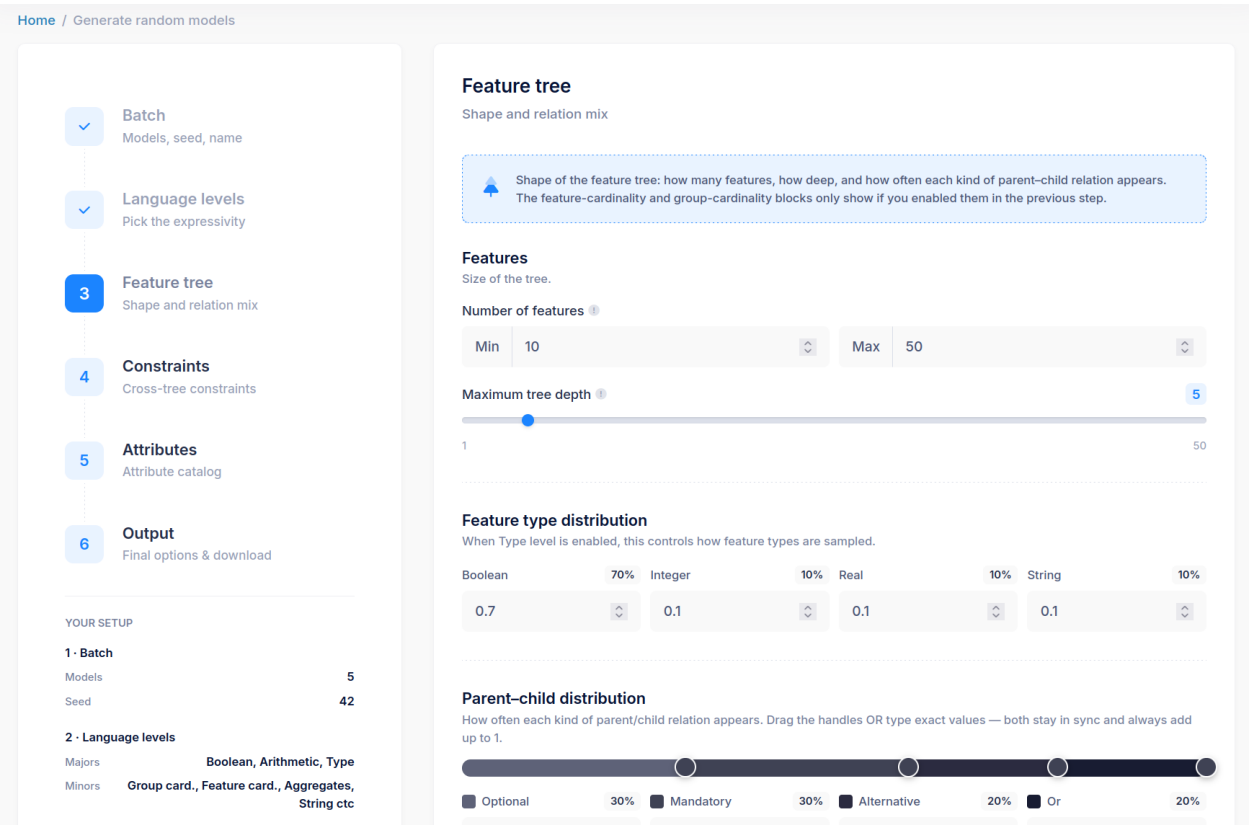


Figura 5.5: Interfaz del Step 3: Feature tree. Elaboración propia

1 · Batch

Models5

Seed42

2 · Language levels

MajorsBoolean, Arithmetic, Type

MinorsGroup card., Feature card., Aggregates, String etc

3 · Feature tree

Features10 – 50

Depth ≤5

Rel mixopt 30% · mand 30% · alt 20% · or 20%

Group card.[1..6]

4 · Constraints

Count1 – 10

Vars/ctc1 – 10

Type mixB 100%

Parent-child distribution

How often each kind of parent/child relation appears. Drag the handles OR type exact values — both stay in sync and always add up to 1.

Optional30%

Mandatory30%

Alternative20%

Or20%

0.3000

0.3000

0.2000

0.2000

Group cardinality0%

0.0000

Feature cardinality ⓘ

Attach probability ⓘ

0.0

Min instances

2

Max instances

5

Min must be ≤ Max.

Group cardinality ⓘ

Children selectable per group ⓘ

Min1

Max6

< Previous

Next >

Figura 5.6: Interfaz del Step 3: Feature tree. Elaboración propia

Step 4: Constraints

Esta vista permite definir los parámetros asociados a la generación de restricciones entre características y atributos, incluyendo la distribución de los diferentes tipos de restricciones y operadores disponibles.

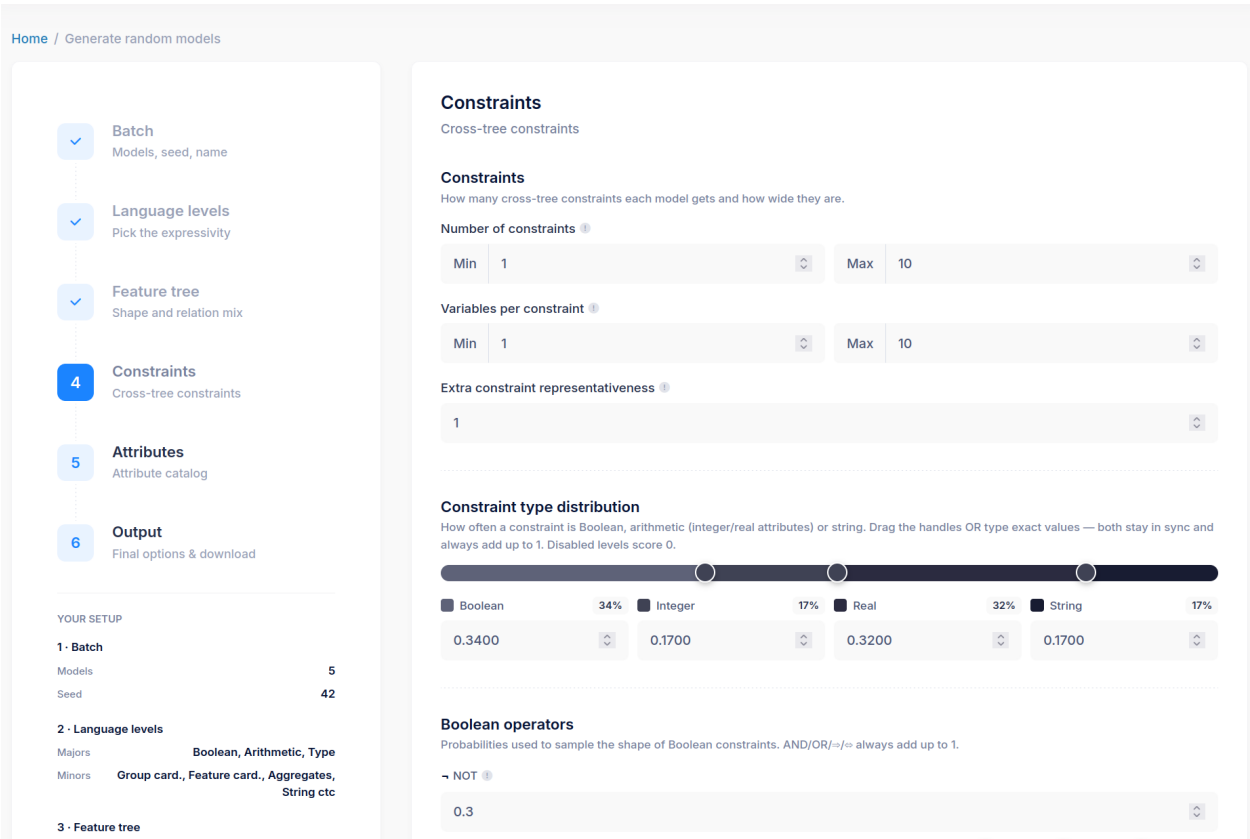


Figura 5.7: Interfaz del Step 4: Constraints. Elaboración propia

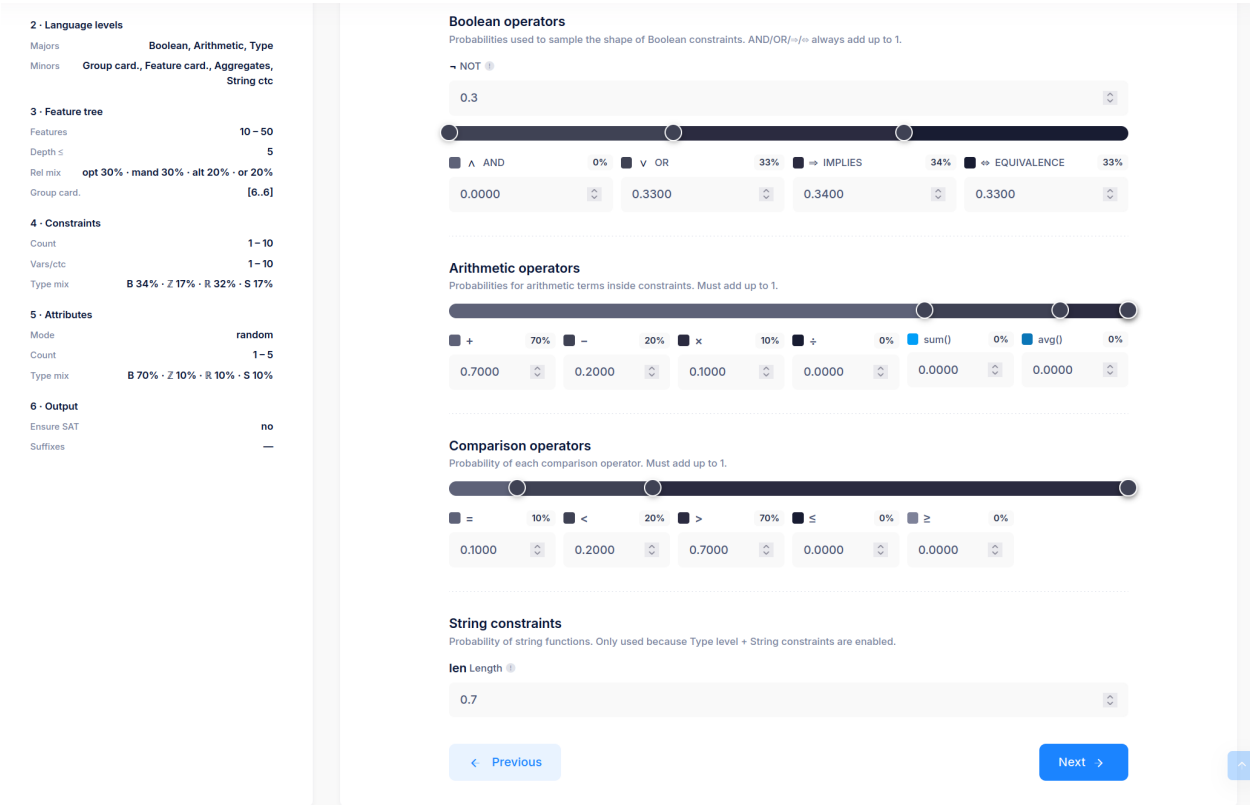


Figura 5.8: Interfaz del Step 4: Constraints. Elaboración propia

Step 5: Attributes

Esta vista permite configurar los parámetros relacionados con la generación de atributos asociados a las características, incluyendo la posibilidad de generar atributos aleatorios mediante una distribución de tipos configurable.

Home / Generate random models

Attributes
Attribute catalog

Attributes
Decide whether attributes are sampled randomly or if you want to define each one by hand.

☒ Generate attributes randomly ⓘ

Number of attributes ⓘ

Min 1 Max 5

Min must be ≤ Max.

Attribute type distribution
When sampling each attribute, how likely is each UVL type to be chosen. Drag the handles OR type exact values — both stay in sync. Integer/Real need Arithmetic level, String needs Type level; disabled types score 0.

Boolean 70% Integer 10% Real 10% String 10%

0.7000 0.1000 0.1000 0.1000

[< Previous](#) [Next >](#)

YOUR SETUP

1 · Batch

Models 5
Seed 42

2 · Language levels

Majors Boolean, Arithmetic, Type
Minors Group card., Feature card., Aggregates, String etc

3 · Feature tree

Figura 5.9: Interfaz del Step 5: Attributes. Elaboración propia

Home / Generate random models

Attributes
Attribute catalog

Attributes
Decide whether attributes are sampled randomly or if you want to define each one by hand.

☐ Generate attributes randomly ⓘ

There are 1 attribute card(s) with missing required fields. Complete them before continuing.

+ Add attribute

Attribute #1 Remove

Name: e.g. Seats Type: Integer

Min: Max:

Attach probability:

☒ Use in constraints

< Previous Next >

YOUR SETUP

1 - Batch
Models: 5
Seed: 42

2 - Language levels
Majors: Boolean, Arithmetic, Type
Minors: Group card., Feature card., Aggregates, String etc

3 - Feature tree
Features: 10 - 50

5 - Attributes
Attribute catalog

6 - Output
Final options & download

Figura 5.10: Interfaz del Step 5: Attributes. Elaboración propia

Step 6: Output

Esta vista permite ejecutar el proceso de generación del modelo UVL utilizando la configuración proporcionada en los pasos anteriores. Además, permite configurar opciones adicionales como la satisfacibilidad (únicamente para nivel booleano) y los sufijos utilizados en los archivos generados.

Una vez finalizada la generación, el usuario puede visualizar el modelo obtenido dentro de la plataforma o descargarlo para su posterior utilización.

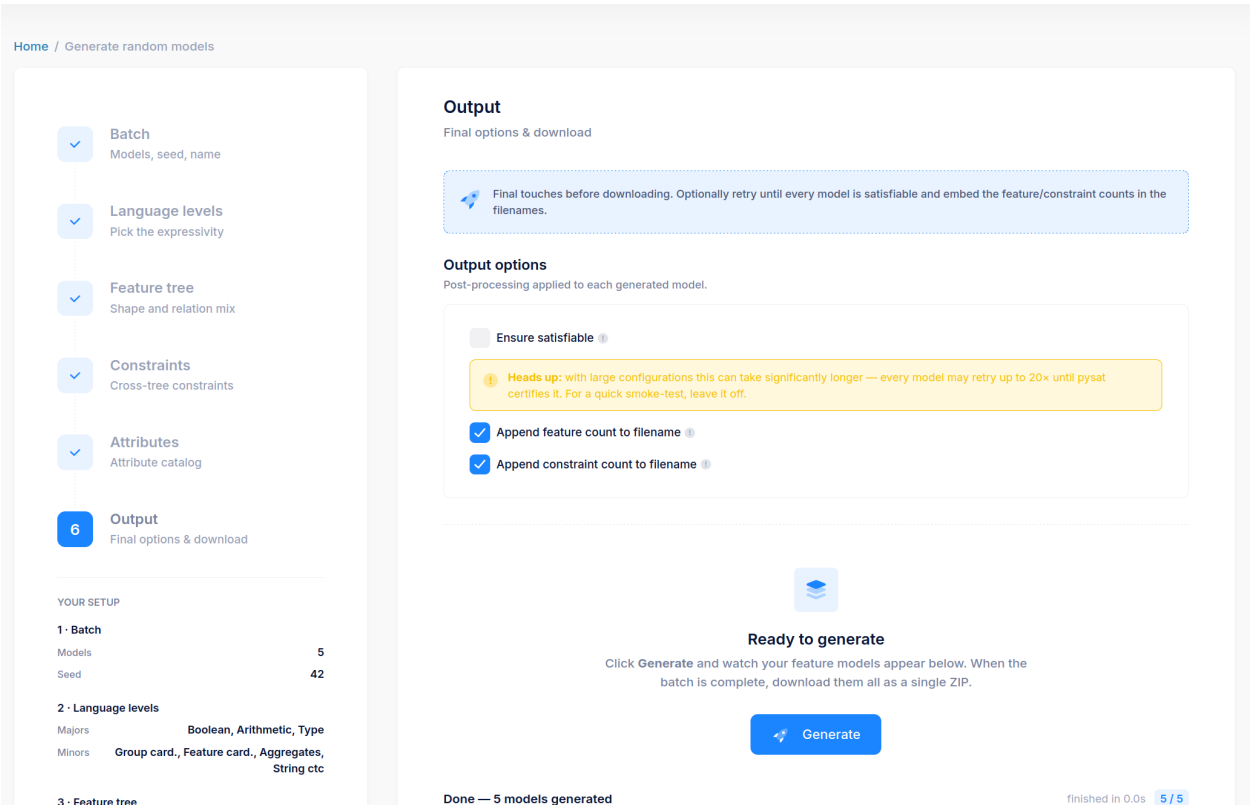


Figura 5.11: Interfaz del Step 6: Output. Elaboración propia

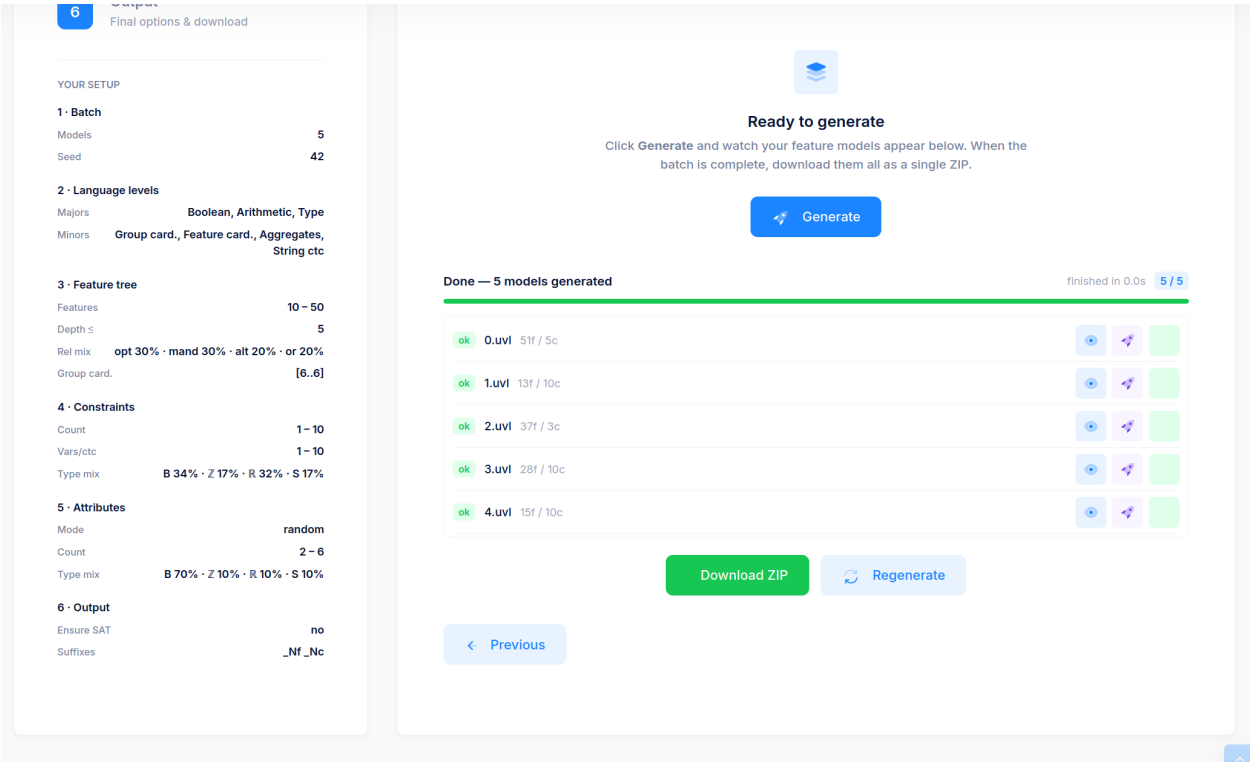


Figura 5.12: Interfaz del Step 6: Output. Elaboración propia

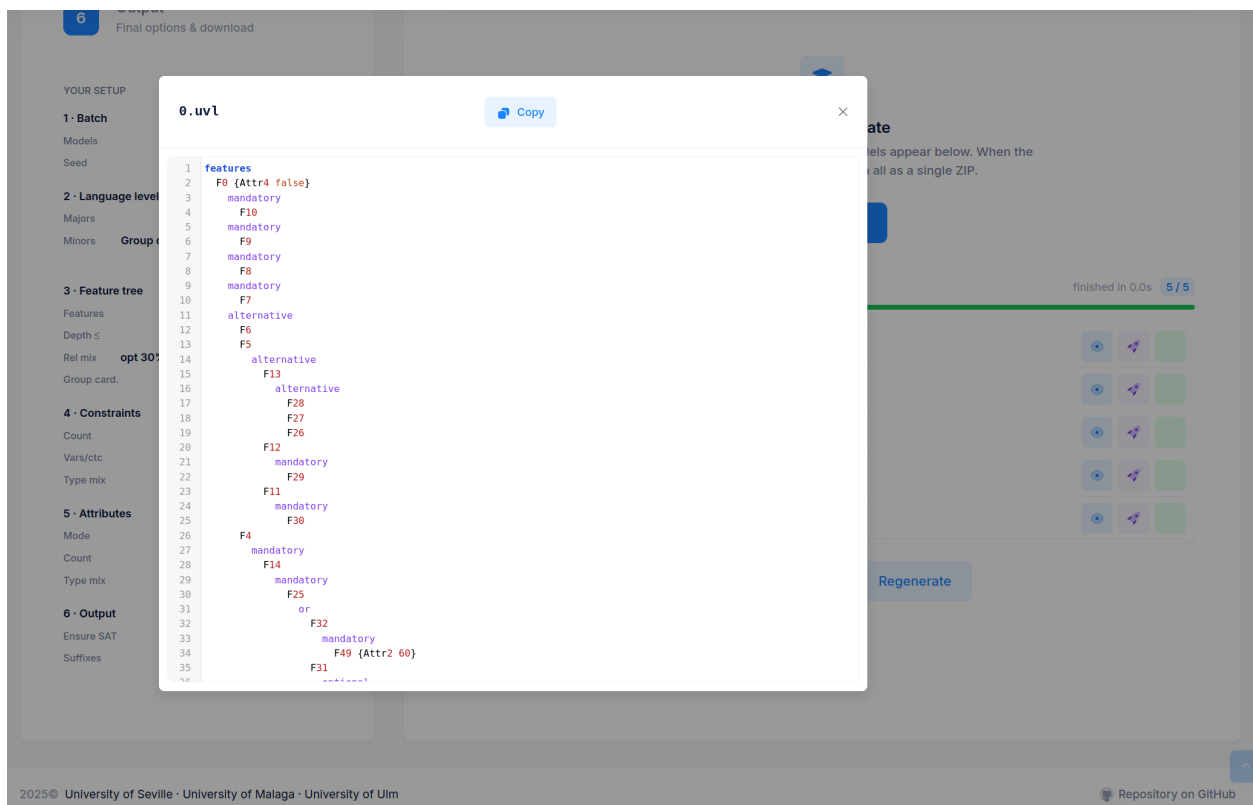


Figura 5.13: Interfaz del Step 6: Output. Elaboración propia

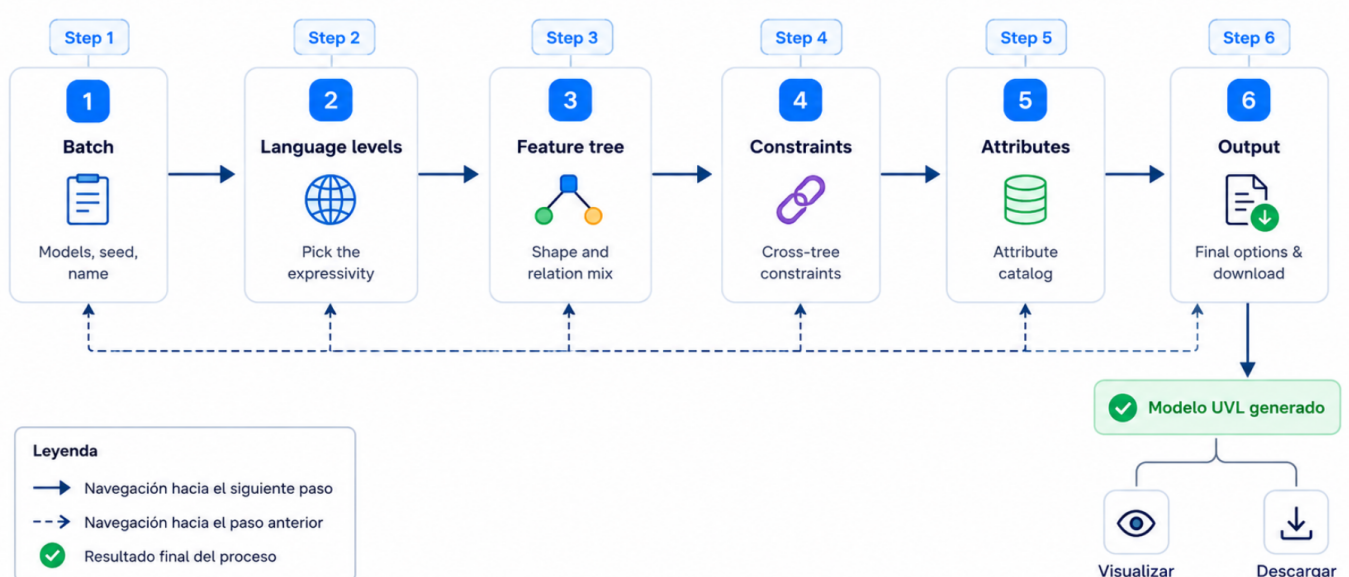


Figura 5.14: Mapa de navegación del generador UVL. Elaboración propia

5.4.2 Wireframes y mockups

Durante la fase inicial del desarrollo se realizó un prototipo de la interfaz con el objetivo de cubrir los requisitos iniciales identificados para el sistema. Los *wireframes* mostrados a continuación corresponden a dicho prototipo inicial y no representan

completamente la versión final de la interfaz, ya que durante el desarrollo se fueron incorporando diferentes mejoras y modificaciones orientadas a mejorar la usabilidad y adaptar el sistema a las nuevas necesidades surgidas durante el desarrollo.

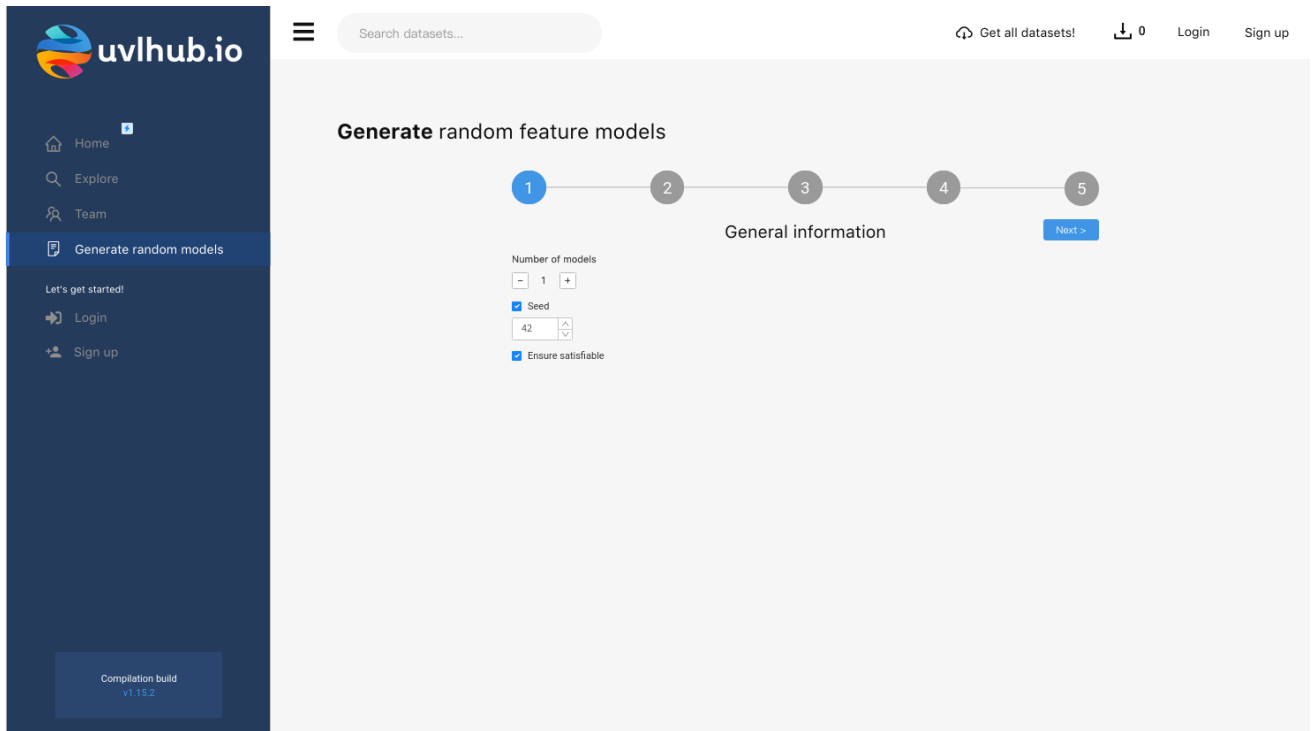


Figura 5.15: Wireframe inicial del Step 1 del generador. Elaboración propia

The image shows a web application interface for generating random feature models. The header includes the 'uvlhub.io' logo, a search bar, and navigation links like 'Get all datasets!', 'Login', and 'Sign up'. A sidebar on the left contains links for 'Home', 'Explore', 'Team', and 'Generate random models'. The main content area is titled 'Generate random feature models' and features a progress bar with five steps. Step 2, 'Tree hierarchy', is the active step. Below the progress bar, the 'Features' section includes a 'Number of features' dropdown set to 'Select one' and a numeric input set to '1'. The 'Distribution (as a per unit value)' section has sliders for Boolean (0.7), Integer (0.2), Real (0.7), and String (0.0). The 'Language level' section has radio buttons for 'Arithmetic level' (selected) and 'Type level'. A note states: 'Note: Arithmetic level includes Boolean level, and Type level include all of them.' The 'Feature cardinality' section is checked and includes min/max dropdowns and numeric inputs for both 'Min' and 'Max' categories. The 'Attach probability' is set to 0.05. The 'Maximum tree depth' is set to 5. The 'Groups' section at the bottom has sliders for 'Optional' (0.7), 'Mandatory' (0.2), 'Alternative' (0.7), and 'Or' (0.0).

uvlhub.io

Search datasets...

Get all datasets! 0 Login Sign up

Generate random feature models

1 2 3 4 5

< Previous Tree hierarchy Next >

Features

Number of features

Select one

1

Distribution (as a per unit value)

Boolean: 0.7 Integer: 0.2 Real: 0.7 String: 0.0

Language level. Boolean level is selected by default.

☒ Arithmetic level
☐ Type level

Note: Arithmetic level includes Boolean level, and Type level include all of them.

☒ Feature cardinality (only if Arithmetic level is selected)

Min: Select one

Min: 1 Max: 3

Max: Select one

Min: 1 Max: 3

Attach probability: 0.05

Maximum tree depth: 5

Groups

Optional: 0.7 Mandatory: 0.2 Alternative: 0.7 Or: 0.0

Figura 5.16: Wireframe inicial del Step 2 del generador. Elaboración propia

The wireframe shows the 'Generate random feature models' interface at Step 3, 'Constraints'. The left sidebar contains the 'uvlhub.io' logo and navigation links: Home, Explore, Team, and 'Generate random models' (highlighted). Below these are links for 'Let's get started!', 'Login', and 'Sign up'. A 'Compilation build v1.15.2' badge is at the bottom. The main content area has a progress bar with steps 1-5, where Step 3 is active. A search bar 'Search datasets...' and links 'Get all datasets!', '0', 'Login', and 'Sign up' are at the top. The 'Constraints' section includes: 'Number of models' (dropdown 'Select one', Min: 1, Max: 4), 'Extra constraint representativeness' (input 0.7), 'Variables per constraint' (dropdown 'Select one', Min: 1, Max: 3), 'Distribution' (Boolean: 0.7, Integer: 0.2, Real: 0.7, String: 0.0), 'Boolean level' (logic: ! 0.7 & 0.2 | 0.7 ==> 0.0 <=> 0.0), 'Arithmetic level constraints' (operators: +, -, *, / with inputs 0.7, 0.2, 0.7, 0.0 and comparison: = 0.0 < 0.2, > 0.7 <= 0.0 >= 0.0, sum(), avg()), and 'Type level constraints' (len() 0.7). A '< Previo' button is on the left and a 'Next >' button is on the right.

Figura 5.17: Wireframe inicial del Step 3 del generador. Elaboración propia

The wireframe shows the 'Generate random feature models' interface at Step 4, 'Attributes'. The layout is identical to Step 3. The progress bar shows Step 4 as active. The 'Attributes' section includes: 'Add' and 'Delete last one' buttons, a form for 'Name' (input 'Seats'), 'Type' (dropdown 'Integer'), 'Value' (dropdown 'Select one'), 'Min' (1) and 'Max' (3) with a range separator, 'Attach probability' (input 0.7), and a checked 'Use in constraints' checkbox. A second identical form is shown below for 'Name' (input 'License Plate'), 'Type' (dropdown 'String'), 'Length' (dropdown 'Select one'), 'Min' (1) and 'Max' (3), and 'Attach probability' (input 0.7). A 'Finish' button is at the bottom right.

Figura 5.18: Wireframe inicial del Step 4 del generador. Elaboración propia

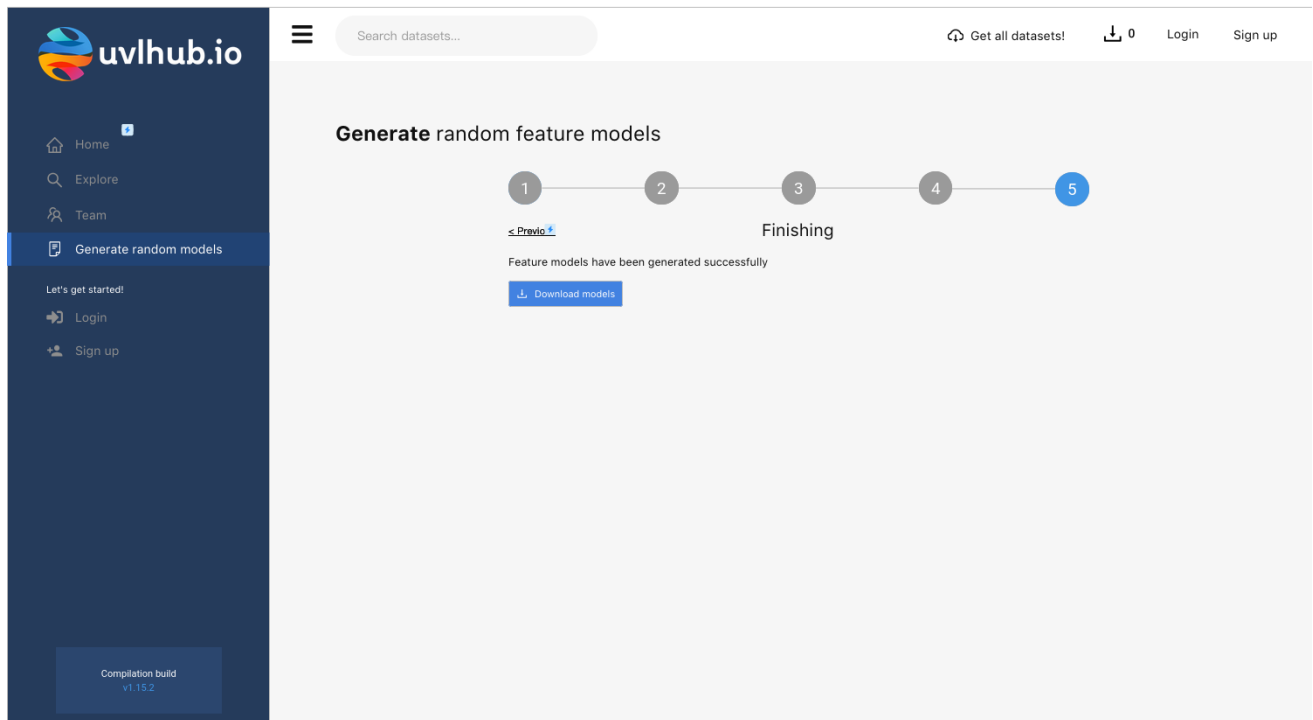


Figura 5.19: Wireframe inicial del Step 5 del generador. Elaboración propia

5.4.3 Decisiones de diseño visual

- **Paleta de colores:** La interfaz mantiene la identidad visual proporcionada por UVLHub, utilizando principalmente tonos azules, blancos y grises para los diferentes elementos de la interfaz. Además, se utilizan colores específicos para representar diferentes estados del sistema, empleando rojo para mensajes de error, amarillo anaranjado para avisos y verde para operaciones completadas correctamente.
- **Tipografía:** Se mantiene la tipografía utilizada por UVLHub, basada en las familias *Inter*, *sans-serif* y *Helvetica*, utilizadas como estándares dentro de la plataforma, conservando la coherencia visual con el resto de la aplicación y proporciona una lectura clara de los elementos de la interfaz.
- **Organización de la interfaz y flujo de configuración:** Debido a la cantidad de parámetros necesarios para configurar el generador, se decidió utilizar un diseño basado en un flujo tipo *wizard*, separando la configuración en diferentes pasos independientes. Esta decisión permite reducir la complejidad percibida por el usuario y mejorar la comprensión del proceso de configuración.

Además, se incorporó un panel lateral de progreso que permite consultar en todo momento el estado actual del proceso, los pasos completados y un resumen de la configuración seleccionada sin necesidad de volver atrás durante el flujo.

- **Componentes visuales y sistema de diseño:** La interfaz utiliza los componentes proporcionados por el sistema de diseño utilizado en UVLHub, basado principalmente en Bootstrap. Además, se desarrollaron componentes específicos para el generador, como el selector de distribuciones probabilísticas utilizado para configurar la generación de modelos, permitiendo adaptar la interfaz a las necesidades particulares del sistema.



Figura 5.20: Componente selector de distribuciones. Elaboración propia

- **Ayudas visuales y gestión de la información:** Para facilitar la configuración de un sistema con una gran cantidad de parámetros, se decidió mostrar únicamente las opciones relacionadas con los niveles de expresividad seleccionados por el usuario. También se incorporaron textos explicativos y *tooltips* en determinados parámetros con el objetivo de proporcionar contexto adicional sobre su funcionamiento.
- **Responsive y adaptación visual:** Aunque la aplicación está orientada principalmente a su uso desde equipos de escritorio, la interfaz se ha diseñado utilizando una estructura adaptable que permite su correcta visualización en diferentes tamaños de pantalla.

5.5 Esquema de base de datos

5.5.1 Modelo entidad-relación

En el sistema diseñado no existe un esquema de base de datos asociado al generador, debido a que no se realiza persistencia de los modelos generados ni de las configuraciones utilizadas durante el proceso de generación. Esta decisión de diseño se debe a que la herramienta está orientada a permitir al usuario configurar el generador, obtener los modelos de características resultantes y comenzar inmediatamente después una nueva generación con una configuración diferente.

Los parámetros introducidos durante el flujo de configuración se mantienen únicamente durante la ejecución actual del generador, siendo utilizados para crear la instancia de configuración de la clase *FmgeneratorModel*. Una vez finalizada

la generación, el resultado puede ser visualizado o descargado por el usuario, sin necesidad de almacenarse de forma permanente dentro del sistema.

5.5.2 Migraciones / versionado del esquema

Al no haber persistencia de datos, no existen migraciones de ningún tipo.

5.6 Flujo de datos principal

El flujo de datos del sistema comienza con los parámetros introducidos por el usuario mediante la interfaz basada en *wizard*. Las rutas del módulo **generator** de UVLHub reciben estos datos y, junto con los servicios internos, realizan las comprobaciones correspondientes a cada paso. La configuración se mantiene en forma de diccionario dentro de la sesión de Flask, permitiendo conservar los valores introducidos durante el recorrido por el asistente.

Una vez completada la configuración, el navegador obtiene estos parámetros en formato JSON mediante el endpoint `/generator/random/params-json`. Dentro del entorno de ejecución proporcionado por Pyodide, el archivo `fmgen_wrapper.py` utiliza el método `from_flat_dict` para construir una instancia de `FmgeneratorModel`. Esta instancia se utiliza para ejecutar la operación `GenerateFeatureModel`, que produce los objetos `FeatureModel`. Posteriormente, estos se transforman a formato UVL y se ponen a disposición del usuario para su visualización o descarga. De esta forma, la construcción del modelo de configuración y la generación se realizan en el navegador, de acuerdo con el requisito RNF-03.

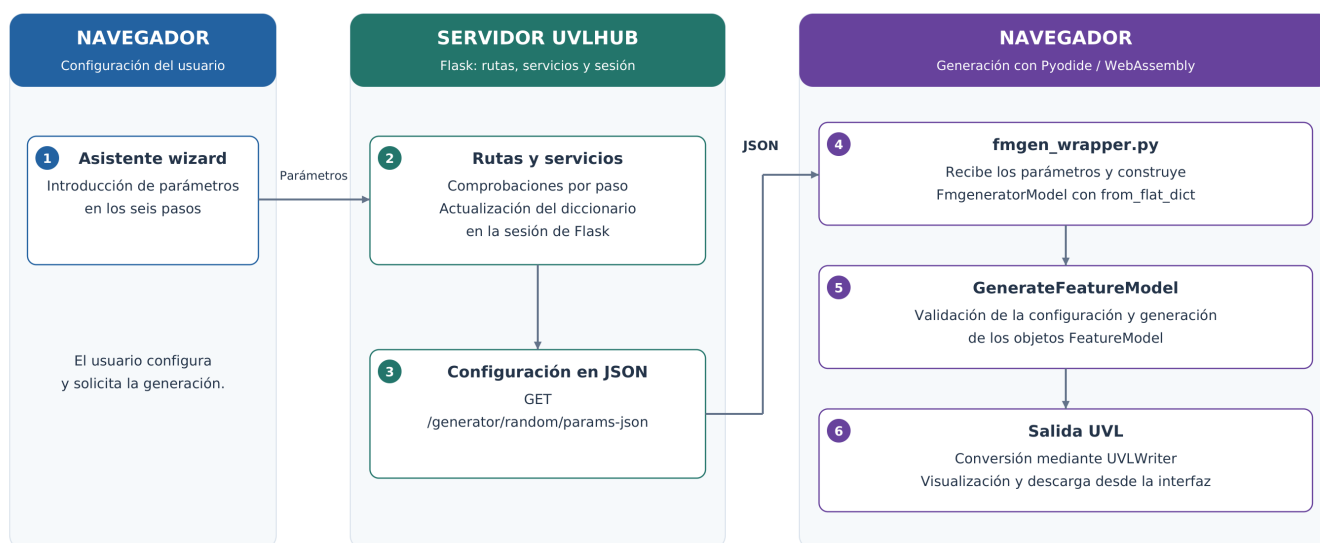


Figura 5.21: Flujo de datos principal. Elaboración propia

5.6.1 Estructura de la representación intermedia

La representación intermedia de la configuración consiste en un diccionario de parámetros almacenado temporalmente en la sesión de Flask. Este diccionario permite mantener los valores introducidos por el usuario entre los diferentes pasos del *wizard*, sin construir todavía una instancia del modelo de dominio del generador.

Las rutas y los servicios del módulo **generator** gestionan y comprueban los parámetros correspondientes a cada paso. Cuando el navegador solicita la configuración, el servidor proporciona su representación en formato JSON. A partir de esta información, `fmgen_wrapper.py` construye la instancia de `FmgeneratorModel` mediante `from_flat_dict` dentro de Pyodide, en el navegador del usuario.

```

435 def load_step_state(step: int, defaults: dict):
436     wizard = session.get("wizard", {})
437     saved = wizard.get(str(step), {})
438
439     out = defaults.copy()
440     out.update(saved)
441
442     return out
443
579
580 if has_saved:
581     defaults.update(
582         {
583             "prob_plus": params_dict.get("PROB_SUM", step4_ui_defaults["prob_plus"]),
584             "prob_minus": params_dict.get("PROB_SUBSTRACT", step4_ui_defaults["prob_minus"]),
585             "prob_times": params_dict.get("PROB_MULTIPLY", step4_ui_defaults["prob_times"]),
586             "prob_div": params_dict.get("PROB_DIVIDE", step4_ui_defaults["prob_div"]),
587             "prob_sum": params_dict.get("PROB_SUM_FUNCTION", step4_ui_defaults["prob_sum"]),
588             "prob_avg": params_dict.get("PROB_AVG_FUNCTION", step4_ui_defaults["prob_avg"]),
589             "prob_eq": params_dict.get("PROB_EQUALS", step4_ui_defaults["prob_eq"]),
590             "prob_lt": params_dict.get("PROB_LESS", step4_ui_defaults["prob_lt"]),
591             "prob_gt": params_dict.get("PROB_GREATER", step4_ui_defaults["prob_gt"]),
592             "prob_leq": params_dict.get("PROB_LESS_EQUALS", step4_ui_defaults["prob_leq"]),
593             "prob_geq": params_dict.get("PROB_GREATER_EQUALS", step4_ui_defaults["prob_geq"]),
594             "prob_len": (
595                 params_dict.get("PROB_LEN_FUNCTION", step4_ui_defaults["prob_len"])
596                 if params_dict.get("TYPE_LEVEL") and params_dict.get("STRING_CONSTRAINTS")
597                 else 0.0
598             ),
599         }
600     )
601 else:
602     defaults.update(step4_ui_defaults)
603     if not (params_dict.get("TYPE_LEVEL") and params_dict.get("STRING_CONSTRAINTS")):
604         defaults["prob_len"] = 0.0
605
606 values = load_step_state(4, defaults)

```

Figura 5.22: Ejemplo de uso de la *session*. Elaboración propia

Esta representación se utiliza para mantener el estado temporal del asistente y transferir los parámetros al entorno de generación del navegador. No constituye un registro permanente de las configuraciones utilizadas ni de los modelos obtenidos.

5.6.2 Mapeo entre niveles

El mapeo entre el diccionario de parámetros y el modelo interno de configuración se realiza en el navegador del usuario. Una vez recibida la configuración en formato JSON, `fmgen_wrapper.py` utiliza el método `from_flat_dict` de `FmgeneratorModel`

para convertir los parámetros en los atributos y objetos de configuración asociados. Esta transformación permite pasar de la representación utilizada durante el *wizard* a la representación orientada a objetos que necesita el generador.

Algunos ejemplos de este mapeo son:

- *NUM_MODELS* $\rightarrow$ *num_models*: número de modelos de características que serán generados.
- *TYPE_LEVEL* $\rightarrow$ *levels.type_level*: activación de las construcciones asociadas al nivel de tipos del lenguaje UVL.
- *MIN_FEATURES* / *MAX_FEATURES* $\rightarrow$ *features.min_features* / *features.max_features*: rango utilizado para determinar el número de características del modelo generado.

Una vez construida la instancia de `FmgeneratorModel`, la configuración se valida y se utiliza para ejecutar la operación `GenerateFeatureModel` dentro de Pyodide. El objeto `FeatureModel` obtenido se transforma posteriormente a una representación textual en UVL. Tanto la construcción de la configuración como la ejecución del generador tienen lugar en el navegador.

5.6.3 Trazabilidad e identificadores

En el sistema no se mantiene trazabilidad persistente entre los modelos generados y la configuración utilizada para generarlos. La información de la configuración únicamente se mantiene durante la ejecución mediante la *session* de Flask.

Con la arquitectura del sistema, los modelos conceptuales, los flujos de datos y las principales decisiones de diseño definidos, queda establecida la solución desde un punto de vista estructural y conceptual. El siguiente capítulo aborda la implementación de dicha solución, detallando las tecnologías utilizadas y cómo los diferentes componentes definidos en este diseño han sido desarrollados para obtener el sistema final.

6 IMPLEMENTACIÓN

6.1 Análisis de tecnologías

La implementación del sistema requiere la utilización de tecnologías compatibles con la arquitectura existente de UVLHub y con el ecosistema Flamapy. Por ello, la selección de tecnologías se ha realizado teniendo en cuenta principalmente la integración con la plataforma existente, la facilidad de desarrollo y la necesidad de disponer de un entorno reproducible para la ejecución y despliegue del sistema.

Necesidad técnica 1: Integración con la plataforma existente

El generador debe integrarse dentro de UVLHub, una plataforma web existente desarrollada utilizando Flask. Por este motivo, la tecnología utilizada debe permitir mantener la arquitectura actual basada en rutas, servicios y plantillas Jinja2, evitando introducir cambios estructurales innecesarios en la aplicación.

Necesidad técnica 2: Desarrollo de la herramienta generadora

La lógica de generación debe implementarse siguiendo la arquitectura de extensiones proporcionada por Flamapy. Para ello, la tecnología utilizada debe permitir desarrollar una herramienta independiente que encapsule la configuración del generador y la operación encargada de producir modelos de características expresados en UVL.

Necesidad técnica 3: Ejecución y despliegue reproducible

El sistema debe disponer de un entorno de ejecución reproducible tanto para el desarrollo local como para facilitar las revisiones del proyecto. Por ello, es necesario utilizar tecnologías que permitan contenerizar la aplicación, gestionar sus dependencias y proporcionar una versión accesible del sistema desplegada en un entorno remoto.

Comparativa de tecnologías del backend

- **Flask:** Framework utilizado por UVLHub sobre el que se ha integrado el generador. Su principal ventaja es la compatibilidad directa con la arquitectura existente, permitiendo reutilizar las rutas, servicios y mecanismos de configuración ya implementados.
- **Django:** Framework web completo con numerosas funcionalidades integradas, como ORM o gestión de usuarios. Sin embargo, su utilización habría requerido adaptar la arquitectura existente de UVLHub, introduciendo una complejidad innecesaria.
- **FastAPI:** Framework orientado principalmente al desarrollo de APIs de alto rendimiento. Aunque ofrece ventajas para servicios desacoplados, no resulta adecuado en este caso al no encajar con la arquitectura basada en Flask utilizada por UVLHub.

Comparativa de tecnologías del frontend

- **Jinja2, JavaScript y Bootstrap:** Tecnologías utilizadas por UVLHub para la construcción de la interfaz web. En particular, Jinja2 constituye el motor de plantillas integrado con Flask, permitiendo generar las vistas de la aplicación utilizando una arquitectura compatible con el backend existente. Esta opción permite incorporar el flujo tipo *wizard* desarrollado para el generador manteniendo la coherencia visual y estructural con el resto de la plataforma.
- **React:** Biblioteca ampliamente utilizada para el desarrollo de interfaces dinámicas basadas en componentes reutilizables. Sin embargo, su utilización habría requerido introducir una arquitectura frontend independiente de la empleada actualmente por UVLHub, aumentando la complejidad de integración del generador.
- **Vue.js:** Framework orientado al desarrollo de interfaces reactivas mediante componentes. Aunque ofrece una solución ligera para construir interfaces modernas, no fue seleccionado debido a que no forma parte de la arquitectura frontend existente en UVLHub.

Análisis de la base de datos

El sistema desarrollado no requiere una base de datos propia, ya que la naturaleza del generador no implica el almacenamiento permanente de configuraciones ni de modelos generados. La información necesaria durante el flujo de configuración se mantiene temporalmente mediante la *session* de Flask, permitiendo ejecutar el

proceso completo sin introducir una capa de persistencia adicional, lo que hace innecesario realizar aquí una comparación.

Plataformas de despliegue

- **Render:** Plataforma utilizada para desplegar una versión accesible de UVLHub durante las revisiones del proyecto. Su elección estuvo motivada tanto por la facilidad de configuración e integración con el repositorio del proyecto como por la experiencia previa personal con esta plataforma, evitando la necesidad de invertir tiempo adicional en el aprendizaje de una nueva tecnología de despliegue.
- **Servicios cloud tradicionales (AWS, Azure, Google Cloud):** Ofrecen una mayor capacidad de configuración y escalabilidad, aunque requieren una gestión más compleja de infraestructura. Debido a que el objetivo del despliegue era facilitar la revisión funcional del sistema y no proporcionar un servicio preparado para producción, no resultaban necesarios para este proyecto.
- **Heroku:** Plataforma orientada al despliegue sencillo de aplicaciones web con una filosofía similar a Render. Aunque habría permitido obtener una solución equivalente, no fue seleccionada al disponer ya de experiencia previa y una configuración establecida con Render.

Tecnologías finales seleccionadas

- **Desarrollo backend:** Python, Flask y Flamapy, utilizando la estructura base para la creación de extensiones proporcionada por este último.
- **Desarrollo frontend:** Plantillas Jinja2, JavaScript y Bootstrap, manteniendo la arquitectura visual existente de UVLHub.
- **Persistencia:** No aplica, al no requerirse almacenamiento permanente de configuraciones ni modelos generados.
- **Contenerización:** Docker, utilizado para construir y ejecutar el entorno completo de desarrollo local mediante contenedores.
- **Despliegue:** Render, utilizado para proporcionar una versión accesible del sistema durante las revisiones del proyecto.

6.2 Tecnologías utilizadas

Tras la selección de tecnologías realizada en el apartado anterior, esta sección describe cómo se han utilizado durante la implementación del sistema, indicando los principales componentes en los que aparecen.

Backend y lógica del servidor

La implementación del backend de UVLHub está desarrollada utilizando Python y Flask. Dentro del módulo *generator* se encuentran las rutas encargadas de gestionar el flujo de configuración del generador, implementadas en el archivo `app/features/generator/routes.py`, así como los servicios internos definidos en `app/features/generator/wizard.py`, responsables de reconstruir y validar la configuración introducida por el usuario.

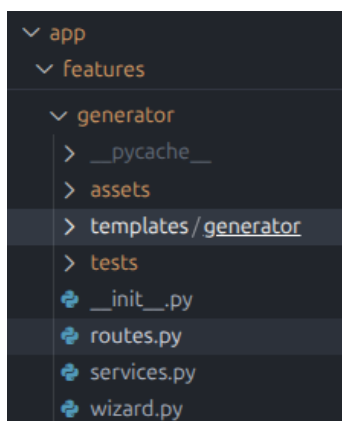


Figura 6.1: Estructura del módulo generator de UVLHub. Elaboración propia

Además, Flask proporciona el mecanismo de *session*, utilizado durante el flujo de configuración para mantener temporalmente los parámetros del generador en forma de diccionario entre los diferentes pasos del *wizard*. El servidor expone esta información en formato JSON mediante el endpoint `/generator/random/params-json`. Posteriormente, el navegador utiliza estos parámetros para construir la instancia de `FmgeneratorModel` mediante `from_flat_dict`, dentro del archivo `fmgen_wrapper.py` ejecutado con Pyodide.

Generador compatible con Flamapy

La lógica principal de generación se ha implementado como un componente compatible con la arquitectura de extensiones proporcionada por Flamapy. Este componente se encuentra dentro del directorio `fm_generator` del proyecto UVLHub y contiene el modelo de configuración del generador mediante la clase *Fmgenera-*

torModel, junto con las diferentes clases encargadas de representar los parámetros específicos de generación.

La ejecución del proceso se encapsula en la operación `GenerateFeatureModel`, ubicada dentro del módulo de operaciones de la herramienta. Su método `execute` utiliza la configuración definida en `FmgeneratorModel` para construir un objeto `FeatureModel`, que se recupera mediante `get_result`. La conversión de este objeto a formato UVL se realiza posteriormente mediante `UVLWriter`.

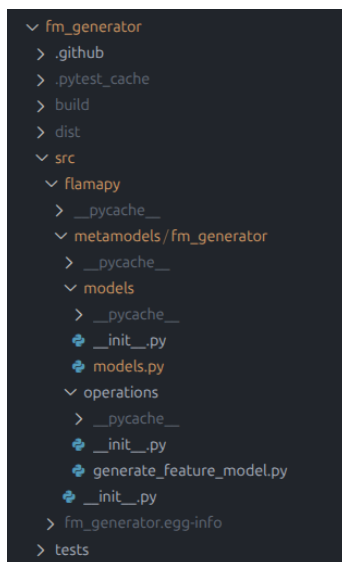


Figura 6.2: Estructura de la herramienta generadora basada en Flamapy.
Elaboración propia

Capa de presentación

La interfaz web del generador mantiene la arquitectura frontend existente en UVLHub, utilizando plantillas Jinja2 integradas con Flask para la generación de las vistas, junto con JavaScript y Bootstrap para gestionar la interacción y los componentes visuales de la aplicación. Las diferentes pantallas que forman el flujo tipo *wizard* se encuentran definidas como plantillas HTML dentro del directorio `app/features/generator/templates/generator`.

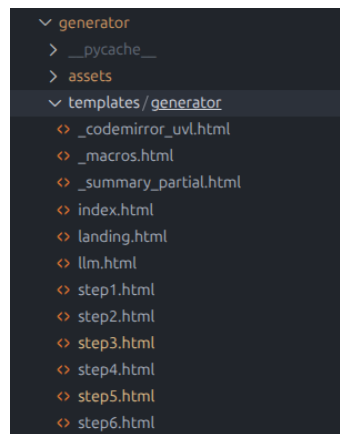


Figura 6.3: Plantillas HTML utilizadas para los pasos del *wizard*. Elaboración propia

Ejecución mediante Pyodide y WebAssembly

Para permitir la ejecución del generador directamente en el navegador del usuario se ha utilizado Pyodide, un entorno basado en WebAssembly que permite ejecutar código Python en el cliente. Los scripts encargados de configurar Pyodide y cargar las dependencias necesarias para la generación se encuentran en el directorio `app/features/generator/assets/js`, donde también se incluyen los paquetes *wheel* necesarios para la ejecución de la herramienta generadora.

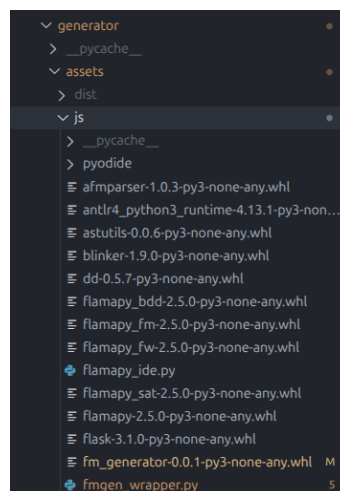


Figura 6.4: Scripts y dependencias utilizadas para la ejecución con Pyodide. Elaboración propia

Infraestructura de desarrollo

Durante el desarrollo local se ha utilizado Docker junto con Docker Compose para definir y ejecutar el entorno completo necesario para UVLHub. Los ficheros de configuración asociados se encuentran en el directorio `docker`, donde se incluyen las definiciones necesarias tanto para los entornos de desarrollo como para los entornos de producción.

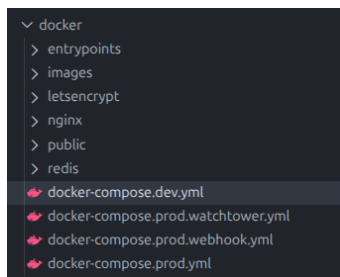


Figura 6.5: Configuración de Docker en UVLHub. Elaboración propia

Aunque el entorno contiene diferentes servicios auxiliares necesarios para el funcionamiento global de UVLHub, el único servicio con una relación directa con el generador es Redis. Este servicio se utiliza durante el despliegue de revisiones para almacenar temporalmente la información asociada a la *session* de Flask, permitiendo mantener la configuración actual del usuario durante el flujo de generación.

Suite de pruebas

Las pruebas del sistema se ejecutan mediante *Rosemary*, una CLI (*Command Line Interface*) incluida dentro del entorno de UVLHub [22].

La suite de pruebas incluye comprobaciones sobre diferentes componentes del sistema, incluyendo pruebas de flujo del *wizard*, validación de parámetros y pruebas de interfaz mediante Selenium, permitiendo verificar el correcto funcionamiento de la interacción del usuario con la aplicación.

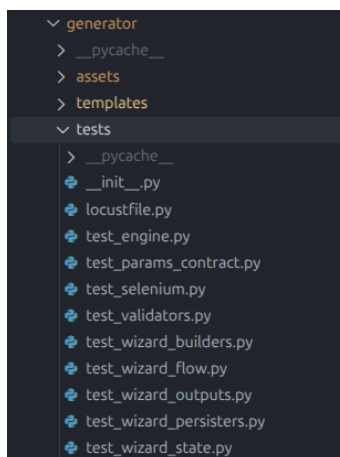


Figura 6.6: Suite de pruebas del módulo generator. Elaboración propia

Automatización CI/CD

Para automatizar las tareas de integración y despliegue se ha utilizado GitHub Actions. Los flujos definidos permiten realizar comprobaciones técnicas previas sobre el código desarrollado antes de completar el despliegue de la aplicación.

Además, se ha configurado la automatización del despliegue, permitiendo actualizar la versión desplegada del sistema de forma automática tras superar los controles establecidos.

6.3 Módulo de validación / lógica principal

El módulo de validación se estructura en dos niveles diferenciados. Por un lado, el servicio `GeneratorWizardService` del módulo `generator` realiza una primera validación durante el proceso de configuración, comprobando que los valores introducidos por el usuario cumplen las restricciones básicas necesarias para continuar el flujo.

Por otro lado, el modelo de dominio `FmgeneratorModel` incorpora la validación semántica de la configuración completa del generador, comprobando la coherencia entre sus diferentes parámetros antes de iniciar la generación.

La generación de modelos de características se encapsula en la operación `GenerateFeatureModel`, que recibe una instancia de `FmgeneratorModel` y genera un único objeto `FeatureModel` por ejecución. El método `execute` devuelve la propia operación, mientras que `get_result` permite recuperar el modelo generado. Antes de ejecutar la generación, el método `execute` realiza una última validación mediante:

```
1 self.model.validate()
```

garantizando que únicamente configuraciones consistentes alcanzan la fase de generación.

6.3.1 Validación sintáctica / comprobaciones básicas

La validación sintáctica se encuentra implementada principalmente en el fichero `wizard.py`, mediante un conjunto de funciones asociadas a cada paso del asistente de configuración:

```
1 validate_step1_form()  
2 validate_step2_form()  
3 validate_step3_form()  
4 validate_step4_form()  
5 validate_step5_form()  
6 validate_step6_form()
```

Estas funciones comprueban que los valores introducidos por el usuario cumplen las restricciones básicas necesarias antes de realizar la validación semántica del modelo de dominio.

Validación de parámetros generales

Durante el primer paso del asistente se validan los parámetros básicos de generación. El número de modelos y la semilla deben cumplir:

$$1 \leq NUM_MODELS \leq 1000$$

$$SEED \in \mathbb{Z}^+$$

Validación de rangos numéricos

Los parámetros asociados a cantidades, límites y tamaños deben mantenerse dentro de rangos válidos. De forma general:

$$VALUE \geq 1$$

Además, todos los pares de límites mínimo-máximo cumplen:

$$MIN_VALUE \leq MAX_VALUE$$

Esta restricción se aplica a parámetros como número de features, número de restricciones, número de atributos o rangos definidos para atributos manuales.

Validación de distribuciones probabilísticas

Las configuraciones probabilísticas utilizadas durante la generación deben representar distribuciones válidas. Para cualquier conjunto de probabilidades asociado a una decisión aleatoria:

$$\forall p_i \in P, \quad 0 \leq p_i \leq 1$$

$$\sum_{p_i \in P} p_i = 1$$

Estas comprobaciones se aplican a las distribuciones de relaciones entre features, operadores booleanos, operaciones aritméticas, tipos de restricciones y tipos de atributos.

Validación de dependencias básicas

Las opciones del asistente presentan dependencias entre niveles de generación que deben cumplirse antes de continuar con la configuración:

$$FEATURE_CARDINALITY \Rightarrow ARITHMETIC_LEVEL$$

$$AGGREGATE_FUNCTIONS \Rightarrow ARITHMETIC_LEVEL$$

$$STRING_CONSTRAINTS \Rightarrow TYPE_LEVEL$$

Validación de atributos manuales

En el modo de definición manual, cada atributo debe cumplir las restricciones básicas de configuración:

$$name(Attribute) \neq \emptyset$$

$$0 \leq attach_probability(Attribute) \leq 1$$

Para atributos con dominio definido mediante rangos:

$$0 \leq MIN_ATTRIBUTE_VALUE \leq MAX_ATTRIBUTE_VALUE$$

Además, la posibilidad de utilizar un atributo dentro de una restricción depende de los niveles activos del generador:

$$UseInConstraints(Attribute_{Integer|Real}) \Rightarrow Arithmetic_Level$$

$$UseInConstraints(Attribute_{String}) \Rightarrow Type_Level \wedge String_Constraints$$

De esta forma, la validación evita configuraciones incompatibles antes de que sean procesadas por la lógica de generación.

6.3.2 Validación semántica y restricciones de dominio

La validación semántica se encuentra encapsulada en el modelo `FmgeneratorModel`, definido dentro de `fm_generator`. Este modelo agrupa la configuración completa del generador mediante diferentes bloques especializados:

Configuración	Responsabilidad
NamingConfig	Nombres y opciones de salida
LevelConfig	Niveles de expresividad UVL
FeaturesConfig	Parámetros de features
HierarchyConfig	Estructura jerárquica
ConstraintsConfig	Restricciones generadas
AttributesConfig	Gestión de atributos

Tabla 6.1: Bloques de configuración de `FmgeneratorModel`

La validación completa se ejecuta mediante el método:

```
1 FmgeneratorModel.validate()
```

que comprueba la coherencia global de la configuración antes de iniciar la generación.

En las comprobaciones de normalización, cada probabilidad debe pertenecer al intervalo $[0, 1]$. La implementación admite una diferencia absoluta de hasta 10^{-6} entre la suma de las probabilidades y uno, para contemplar las imprecisiones de la representación numérica.

Coherencia de niveles UVL

Los niveles de expresividad presentan dependencias jerárquicas entre ellos. Las reglas aplicadas son:

$$BOOLEAN_LEVEL = True$$

$$TYPE_LEVEL \Rightarrow ARITHMETIC_LEVEL$$

$$FEATURE_CARDINALITY \Rightarrow ARITHMETIC_LEVEL$$

$$AGGREGATE_FUNCTIONS \Rightarrow ARITHMETIC_LEVEL$$

$$STRING_CONSTRAINTS \Rightarrow TYPE_LEVEL$$

Validación de la jerarquía de features

La estructura jerárquica generada debe ser compatible con el número máximo de features:

$$1 \leq MAX_TREE_DEPTH \leq MAX_FEATURES$$

Además, la selección probabilística de relaciones entre features requiere una distribución normalizada:

$$P_{optional} + P_{mandatory} + P_{alternative} + P_{or} + P_{group} = 1$$

Validación de restricciones

Las restricciones generadas dependen de los niveles activos. La distribución de tipos de restricciones se valida mediante:

$$\sum_i P(ConstraintType_i) = 1$$

donde los tipos disponibles quedan determinados por:

$$P(Integer) > 0 \Rightarrow ARITHMETIC_LEVEL$$

$$P(Real) > 0 \Rightarrow ARITHMETIC_LEVEL$$

$$P(String) > 0 \Rightarrow TYPE_LEVEL \wedge STRING_CONSTRAINTS$$

Las distribuciones de operadores deben estar normalizadas. Para operadores booleanos:

$$P(AND) + P(OR) + P(Implication) + P(Equivalence) = 1$$

Cuando está activo el nivel aritmético, la distribución de operaciones se valida teniendo en cuenta si las funciones agregadas están habilitadas. Si están desactivadas:

$$P(+) + P(-) + P(*) + P(/) = 1$$

Si las funciones agregadas están activadas, sus probabilidades se incorporan a la misma distribución:

$$P(+) + P(-) + P(*) + P(/) + P(sum) + P(avg) = 1$$

La distribución de operadores de comparación se valida de forma separada:

$$P(==) + P(<) + P(>) + P(\leq) + P(\geq) = 1$$

Validación de atributos

La configuración de atributos diferencia entre generación aleatoria y definición manual.

Para atributos aleatorios:

$$0 \leq MIN_ATTRIBUTES \leq MAX_ATTRIBUTES$$

y la distribución de tipos debe cumplir:

$$P(Boolean) + P(Integer) + P(Real) + P(String) = 1$$

Para atributos definidos manualmente:

$$MIN_ATTRIBUTES = NONE$$

$$MAX_ATTRIBUTES = NONE$$

Además, la utilización de atributos en restricciones depende de los niveles disponibles:

$$UseInConstraints(Attribute_{Integer|Real}) \Rightarrow Arithmetic_Level$$

$$UseInConstraints(Attribute_{String}) \Rightarrow Type_Level \wedge String_Constraints$$

En el modo de definición manual, cada elemento de `attributes_list` puede incluir el parámetro `attach_probability`, que determina la probabilidad de asociar el atributo a una característica. Cuando se proporciona, su valor debe cumplir:

$$0 \leq attach_probability(Attribute) \leq 1$$

6.3.2.1. Consumo por otros módulos

Una vez validada la configuración, la instancia de `FmgeneratorModel` es utilizada por la operación `GenerateFeatureModel`, encargada de construir el modelo final compatible con FlamaPy.

Esta operación utiliza dicha configuración para generar:

- La jerarquía de features.
- Los tipos y cardinalidades.

- Los atributos asociados.
- Las restricciones UVL.

La corrección de estas reglas se verifica mediante la suite de pruebas ubicada en:

```
1 tests/test_validators.py
```

donde se incluyen casos que comprueban el rechazo de configuraciones inconsistentes, como distribuciones no normalizadas, dependencias incumplidas o rangos inválidos.

6.4 Módulo de generación de artefactos

La generación se encuentra encapsulada en la operación `GenerateFeatureModel`, perteneciente a `fm_generator`. Esta operación recibe como entrada una instancia validada de `FmgeneratorModel`, que contiene todos los parámetros necesarios para controlar la generación, y produce como resultado un objeto `FeatureModel`.

El flujo principal de generación se resume en:

$$FmgeneratorModel \xrightarrow{\text{GenerateFeatureModel}} FeatureModel$$

El modelo resultante contiene la estructura jerárquica de features, sus relaciones, atributos asociados y restricciones expresadas mediante el formato interno de FlamaPy, permitiendo posteriormente su transformación a formatos compatibles como UVL.

La ejecución principal se realiza mediante el método:

```
1 GenerateFeatureModel.execute()
```

que valida nuevamente la configuración recibida y coordina las diferentes fases necesarias para construir el objeto `FeatureModel`, cuya transformación a formato UVL se realiza posteriormente.

6.4.1 Cálculos o transformaciones específicas

La operación `GenerateFeatureModel` constituye el núcleo del proceso de generación. Su funcionamiento se divide en varias fases encargadas de transformar la configuración definida en `FmgeneratorModel` en un objeto `FeatureModel`. Cada fase está encapsulada mediante métodos independientes que separan la generación de la jerarquía, atributos, restricciones y configuración específica de UVL.

El algoritmo principal de la operación se muestra en el siguiente pseudocódigo:

```
1 FUNCION GenerateFeatureModel.execute:
```

```

2      INICIO
3          VALIDAR configuracion del generador
4          INICIALIZAR semilla aleatoria
5          feature_model, features =
6              GENERAR jerarquia de features
7          ASIGNAR atributos a las features generadas
8          GENERAR restricciones del modelo
9          GENERAR directivas UVL necesarias
10         DEVOLVER feature_model
11     FIN

```

Código 6.1: Esquema en pseudocódigo de la operación `GenerateFeatureModel.execute`.

Inicialización de la generación

Antes de comenzar la generación se ejecuta el método `_seed_generation()`, encargado de configurar la semilla utilizada por el generador aleatorio.

```

1  FUNCION _seed_generation(index, attempt):
2      INICIO
3          CALCULAR seed =
4              modelo.seed + index + (attempt * SAT_SEED_STRIDE)
5          CONFIGURAR generador aleatorio con seed
6      FIN

```

Código 6.2: Esquema en pseudocódigo del método `_seed_generation()`.

Generación de la jerarquía de features

El método `_generate_hierarchy()` construye la estructura principal del modelo. Inicialmente crea la feature raíz y determina el número total de features que serán generadas.

```

1  FUNCION _generate_hierarchy:
2      INICIO
3          CREAR feature raíz F0
4          DETERMINAR numero_features aleatorio
5          CREAR lista de features
6          PARA cada feature:
7              ASIGNAR tipo booleano o tipado UVL
8          PARA cada nivel de profundidad:
9              SELECCIONAR features del nivel actual
10             SELECCIONAR tipo de relacion padre-hijo

```

```
11         CREAR relaciones entre features
12         ASIGNAR cardinalidades
13     DEVOLVER FeatureModel y features generadas
14 FIN
```

Código 6.3: Esquema en pseudocódigo del método `_generate_hierarchy()`.

La creación de relaciones depende de las distribuciones configuradas por el usuario, pudiendo generar relaciones obligatorias, opcionales, alternativas, disyuntivas o grupos con cardinalidad.

Asignación de atributos

El método `_assign_attributes()` determina el modo de generación de atributos seleccionado.

```
1 FUNCION _assign_attributes(features):
2     INICIO
3         SI modo_aleatorio:
4             GENERAR atributos aleatorios
5         EN OTRO CASO:
6             ASIGNAR atributos definidos manualmente
7         ASOCIAR cada atributo con su feature correspondiente
8     FIN
```

Código 6.4: Esquema en pseudocódigo del método `_assign_attributes()`.

En el modo aleatorio, el tipo y dominio de cada atributo se obtiene mediante las distribuciones configuradas. En el modo manual, se utilizan las definiciones proporcionadas por el usuario durante la configuración del generador.

Generación de restricciones

La generación de restricciones se realiza mediante el método `_add_constraints()`, encargado de construir expresiones compatibles con el metamodelo de FlamaPy.

```
1 FUNCION _add_constraints(feature_model, features):
2     INICIO
3         CREAR conjuntos de features y atributos disponibles
4         DETERMINAR numero de restricciones
5         REPETIR para cada restriccion:
6             SELECCIONAR variables compatibles
7             DETERMINAR tipo de expresion:
8                 - booleana
9                 - aritmetica
```

```

10         - cadenas
11         CONSTRUIR arbol sintactico AST
12         ANADIR restriccion al FeatureModel
13     FIN

```

Código 6.5: Esquema en pseudocódigo del método `_add_constraints()`. Cada restricción se representa mediante un árbol sintáctico abstracto (AST) que permite expresar operaciones lógicas, aritméticas y funciones específicas de UVL.

Generación de directivas UVL

Finalmente, el método `_build_uvl_includes()` analiza los niveles activados y genera las directivas necesarias para representar correctamente el modelo en UVL.

```

1 FUNCION _build_uvl_includes:
2     INICIO
3         CREAR lista de directivas
4         SI group_cardinality activo:
5             ANADIR Boolean.group-cardinality
6         SI feature_cardinality activo:
7             ANADIR Arithmetic.feature-cardinality
8         SI aggregate_functions activo:
9             ANADIR Arithmetic.aggregate-function
10        SI string_constraints activo:
11            ANADIR Type.string-constraints
12        DEVOLVER lista de directivas
13    FIN

```

Código 6.6: Esquema en pseudocódigo del método `_build_uvl_includes()`.

6.4.2 Síntesis del artefacto final

Preparación de la configuración

Los parámetros definidos mediante el asistente se obtienen del servidor en formato JSON. En el navegador, el archivo `fmgen_wrapper.py`, ejecutado dentro de Pyodide, utiliza el método `from_flat_dict` para transformarlos en una instancia de `FmgeneratorModel`. Antes de iniciar la generación, dicha configuración se valida para comprobar la coherencia entre sus parámetros.

Generación del modelo intermedio

La operación `GenerateFeatureModel` utiliza la configuración validada para construir un objeto `FeatureModel` que contiene la jerarquía de features, atributos y restricciones generadas.

$$FmgeneratorModel \rightarrow FeatureModel$$

Transformación a formato UVL

El modelo generado es transformado mediante las operaciones proporcionadas por FlamaPy a una representación textual compatible con UVL.

$$FeatureModel \rightarrow UVL$$

Generación del artefacto final

Finalmente, la representación UVL obtenida se almacena como fichero de salida, constituyendo el artefacto final generado por el sistema y listo para ser utilizado por herramientas compatibles con el lenguaje UVL.

6.4.3 Operaciones complementarias

6.5 API REST / capa de servicios

6.5.1 Endpoints implementados

Endpoints de entrada al generador

- **GET /generator:** inicializa el asistente de generación, reiniciando el estado almacenado en sesión y mostrando la página inicial.
- **GET /generator/random:** redirige al primer paso del flujo de generación aleatoria.

Endpoints del asistente de configuración

El flujo principal de generación aleatoria se divide en seis etapas. Cada paso admite peticiones `GET` para mostrar el formulario correspondiente y `POST` para procesar los valores introducidos por el usuario.

- **GET/POST /generator/random/step1:** configuración inicial del generador, incluyendo número de modelos y parámetros generales.

- **GET/POST /generator/random/step2**: configuración de los niveles de expresividad UVL disponibles.
- **GET/POST /generator/random/step3**: definición de la estructura jerárquica del árbol de features.
- **GET/POST /generator/random/step4**: configuración de restricciones y operadores utilizados durante la generación.
- **GET/POST /generator/random/step5**: configuración de atributos asociados a las features.
- **GET/POST /generator/random/step6**: configuración final, generación del modelo y gestión de la salida.

Endpoints auxiliares

- **GET /generator/random/params-json**: devuelve la configuración actual del asistente en formato JSON.
- **POST /generator/random/summary-refresh/<step>**: actualiza dinámicamente el resumen de parámetros mostrado durante la configuración.

6.5.2 Descarga / exportación de artefactos

Una vez finalizada la generación, el modelo de características obtenido se exporta en un formato compatible con UVL para su descarga en ZIP y posterior uso. El sistema no almacena permanentemente los modelos generados, sino que permite reproducir la generación a partir de la configuración utilizada y la semilla aleatoria definida.

6.6 Integración continua

- **Validación de commits**: comprobación automática de los cambios introducidos mediante GitHub Actions, verificando que las modificaciones cumplen las condiciones establecidas antes de su integración.
- **Linting y análisis estático**: ejecución de herramientas de análisis del código para detectar errores de estilo, problemas potenciales y mantener una calidad homogénea.
- **Pruebas**: ejecución automática de la suite de tests del proyecto en los cambios realizados, asegurando que las nuevas modificaciones no afectan a la funcionalidad existente.

- **Despliegue:** automatización de la actualización del entorno de ejecución tras superar correctamente las comprobaciones anteriores.

Con la implementación del generador, la integración de sus componentes y la automatización de los procesos de validación y despliegue, queda establecido el funcionamiento completo del sistema desarrollado. El siguiente capítulo presenta la estrategia de pruebas aplicada, orientada a verificar que la solución cumple con los requisitos definidos y que los distintos módulos funcionan correctamente.

7 PRUEBAS Y VALIDACIÓN

7.1 Estrategia de pruebas

La estrategia de pruebas aplicada tiene como objetivo verificar el correcto funcionamiento del generador integrado en UVLHub, validando tanto los componentes individuales como el flujo completo de configuración y generación.

Las pruebas desarrolladas se organizan en los siguientes niveles:

- **Pruebas unitarias:** enfocadas en validar componentes aislados del sistema, incluyendo las reglas de validación, la construcción de configuraciones y la lógica interna del generador.
- **Pruebas de integración:** comprueban la interacción entre los distintos componentes del módulo `generator`, incluyendo la gestión del estado del , la persistencia de parámetros y la generación de salidas.
- **Pruebas de sistema:** verifican el funcionamiento completo del flujo desde la perspectiva del usuario, incluyendo la interacción con el asistente de configuración mediante pruebas automatizadas.
- **Pruebas manuales:** se han realizado comprobaciones manuales del flujo completo de generación, verificando el comportamiento de la aplicación durante el desarrollo.

La ejecución de las pruebas automatizadas se realiza mediante la suite de tests del proyecto, utilizando el entorno definido mediante Docker para garantizar la disponibilidad de las dependencias necesarias y la reproducibilidad de los resultados.

7.2 Pruebas unitarias

Las pruebas unitarias se han centrado en validar los componentes internos del generador y la lógica asociada al asistente de configuración. Los principales grupos de casos cubiertos son:

- **Generador de modelos de características:** se valida la operación `GenerateFeatureModel.execute`, comprobando que los parámetros del objeto `FmgeneratorModel` se aplican correctamente durante la generación. Se verifican aspectos como el número de modelos generados, la reproducibilidad mediante semillas, los límites de características y profundidad del árbol, así

como la generación de relaciones jerárquicas, atributos y restricciones según los niveles UVL habilitados.

- **Generación de restricciones y atributos:** se comprueba que la activación de los diferentes niveles del generador produce únicamente los elementos permitidos. Se validan restricciones booleanas, aritméticas y de tipo string, además de operadores agregados (`sum`, `avg`), funciones sobre cadenas y distribución de tipos de atributos. También se verifica que los atributos configurados para no participar en restricciones no aparecen en las expresiones generadas.
- **Validación del modelo de parámetros:** se prueban las comprobaciones realizadas por `FmgeneratorModel`, incluyendo la validación de distribuciones de probabilidad, rangos de generación y coherencia entre niveles activados. Además, se comprueba la correcta conversión desde el diccionario plano de parámetros utilizado por UVLHub hacia la estructura interna del generador.
- **Validadores del asistente de configuración:** se prueban los métodos de validación de cada paso del *wizard* (`validate_step1_form` a `validate_step5_form`), comprobando casos como límites inválidos de modelos o características, distribuciones incorrectas, configuraciones incompatibles entre niveles y restricciones imposibles de generar.
- **Gestión del estado y construcción de parámetros:** se valida la persistencia temporal de la configuración almacenada en la sesión de Flask mediante las funciones de gestión del estado del *wizard*. También se comprueba la normalización de distribuciones y la correcta propagación de los valores entre pasos mediante los constructores de parámetros.

7.3 Pruebas de integración

Las pruebas de integración se han centrado en verificar la comunicación entre los diferentes módulos que intervienen en el proceso de generación, desde la configuración realizada mediante el *wizard* de UVLHub hasta la obtención del modelo UVL generado.

Los principales casos cubiertos son:

- **Integración del *wizard* con la capa de servicios:** se prueba el recorrido completo del asistente mediante las rutas `/generator/random/step1` a `/generator/random/step6`, verificando la interacción entre los controladores Flask y los métodos de `GeneratorWizardService` encargados de procesar cada etapa.

- **Integración de la configuración persistida con el generador:** se comprueba que la configuración almacenada durante el flujo del *wizard* se recupera correctamente mediante el endpoint `/generator/random/params-json` y puede ser transformada en una instancia válida de `FmgeneratorModel`.
- **Integración del modelo de parámetros con la generación UVL:** se valida que una configuración completa procedente del *wizard* es utilizada correctamente por `GenerateFeatureModel`, generando artefactos UVL que reflejan las opciones seleccionadas, como niveles UVL, cardinalidades, tipos de atributos, restricciones y opciones de nomenclatura.
- **Integración del flujo de navegación y estado del asistente:** se verifican los escenarios de navegación hacia atrás entre pasos, comprobando que los valores almacenados en la sesión Flask permanecen consistentes y que una configuración parcialmente modificada no provoca estados inválidos durante la generación.

7.4 Pruebas de extremo a extremo

Las pruebas de extremo a extremo se han implementado mediante Selenium WebDriver, validando el comportamiento del sistema desde la interfaz web hasta la ejecución del proceso de generación en el navegador.

El flujo principal validado es el siguiente:

1. **Carga del generador:** se verifica la correcta inicialización de la interfaz mediante el acceso a la ruta `/generator/random/step1` y la disponibilidad de los elementos necesarios para comenzar la configuración.
2. **Ejecución del proceso de generación:** se automatiza la interacción con el *wizard* completo, comprobando la transición entre las diferentes vistas y la correcta respuesta de la interfaz ante las acciones del usuario.
3. **Inicialización del entorno Pyodide:** se valida la carga del runtime JavaScript necesario para ejecutar el generador en el navegador antes de iniciar la generación.
4. **Generación y visualización del modelo UVL:** se comprueba la ejecución de la generación desde la interfaz, verificando el estado final del proceso, la aparición del resultado generado y la visualización del contenido UVL obtenido.

Además, se han probado escenarios adicionales relacionados con la interacción del usuario:

- Navegación entre pasos mediante los controles de la interfaz, verificando que el comportamiento visual coincide con el estado esperado.
- Activación y desactivación de opciones dependientes dentro del formulario, como niveles aritméticos, cardinalidades y opciones avanzadas.
- Ejecución del generador con configuraciones completas desde la interfaz, comprobando la disponibilidad de las acciones finales sobre el modelo generado.

7.5 Cobertura y métricas de calidad

La calidad de la solución se ha evaluado mediante pruebas automatizadas y cobertura de líneas, utilizando `pytest` y `coverage.py`. Se ha establecido un umbral mínimo del **80 %** sobre el código de producción.

Este umbral busca mantener una cobertura honesta. Las líneas no cubiertas corresponden principalmente a comprobaciones defensivas, configuraciones inválidas bloqueadas por el *wizard* y rutas de error cuya ejecución forzada no aportaría evidencia funcional relevante.

La medición excluye las dependencias externas, las cachés, los artefactos generados, los ficheros auxiliares de carga y los propios tests. Aunque los informes incluyen estos ficheros para facilitar el análisis, no se utilizan para calcular la cobertura del código de producción.

Fichero	Líneas	Sin cubrir	Cobertura
app/features/generator/__init__.py	2	0	100 %
app/features/generator/assets/js/ fmgen_wrapper.py	49	0	100 %
app/features/generator/routes.py	214	6	97 %
app/features/generator/wizard.py	810	158	80 %
fm_generator/src/.../models/models.py	325	42	87 %
fm_generator/src/.../operations/ generate_feature_model.py	666	73	89 %
Código de producción de UVLHub	1075	164	85 %
Código de producción de fm_generator	997	115	88 %
Total de código de producción	2072	279	87 %

Tabla 7.1: Cobertura de líneas del código de producción del generador.

El informe de UVLHub muestra una cobertura global del 91 %, pero dicho valor incluye también los ficheros de prueba. Al excluirllos, el código de producción del generador alcanza un 85 %. Del mismo modo, el informe del paquete `fm_generator` muestra un 91 % incluyendo sus tests, mientras que la cobertura específica del código de producción es del 88 %.

Los ficheros con menor cobertura son `wizard.py`, con un 80 %, y `models.py`, con un 87 %. A pesar de ello, los caminos principales de validación, persistencia de parámetros, generación, serialización y lectura de modelos se encuentran cubiertos. El motor `GenerateFeatureModel`, responsable de construir los modelos, alcanza un 89 %.

La suite completa está formada por 220 pruebas de UVLHub y 44 pruebas del paquete `fm_generator`, todas superadas en sus respectivas ejecuciones. Las dos ejecuciones del paquete independiente no representan pruebas adicionales, sino una repetición de las mismas pruebas, una de ellas con medición de cobertura.

Conjunto de pruebas	N.º de pruebas	Resultado
Pruebas de UVLHub	220	220/220 pasan
Pruebas de <code>fm_generator</code>	44	44/44 pasan
Total de la suite	264	264/264 pasan

Tabla 7.2: Resumen de la suite automatizada de pruebas.

La suite combina pruebas unitarias sobre validadores, constructores, persistencia de parámetros y reglas del motor, con pruebas de integración del flujo del *wizard* y de la generación de modelos. Además, incluye 12 pruebas con Selenium para verificar los flujos principales desde la interfaz web.

Destaca la prueba asociada al objetivo O1, que recorre 26 combinaciones válidas de niveles principales y secundarios. Para cada combinación se generan tres modelos, lo que supone 78 modelos en total. Cada modelo se serializa a UVL y se vuelve a cargar mediante `UVLReader`. En las configuraciones booleanas, además, el modelo se transforma a PySAT y se ejecuta la operación de satisfacibilidad. Esta prueba comprueba la compatibilidad del resultado con las herramientas que consumirán posteriormente los modelos generados.

La suite aporta evidencia para los objetivos de generación de modelos con diferentes niveles de expresividad (O1), configuración del proceso de generación (O2), integración con UVLHub y Flamapy (O3) y disponibilidad de una batería automatizada de validación (O5). La relación detallada entre requisitos y pruebas se recoge en la Tabla 4.18.

La estrategia de pruebas desarrollada permite verificar el correcto funcionamiento del sistema y comprobar el cumplimiento de los principales requisitos definidos. Una vez validada la implementación, el siguiente capítulo analiza los resultados obtenidos y evalúa en qué medida la solución desarrollada alcanza los objetivos planteados inicialmente.

8 RESULTADOS Y DISCUSIÓN

8.1 Diseño de la evaluación

La evaluación se centra en comprobar la capacidad del generador para producir modelos UVL con diferentes niveles de expresividad, la influencia de los parámetros de configuración sobre los modelos obtenidos, la incorporación de construcciones avanzadas del lenguaje, la integración dentro del ecosistema UVLHub y Flamapy, y la disponibilidad de una batería de pruebas automatizadas que permita validar su funcionamiento.

Durante el proceso de desarrollo se han utilizado diferentes configuraciones del generador y se han analizado los modelos UVL resultantes, comprobando que las características seleccionadas durante la configuración se reflejan correctamente en los artefactos generados. Estas comprobaciones se han complementado con la ejecución de la suite de pruebas automatizadas y con validaciones manuales de los principales flujos de uso de la aplicación.

La evaluación se realizó en el siguiente entorno:

- **Sistema operativo:** Ubuntu 24.04.4 LTS.
- **Navegador:** Firefox 148.0.2 (64 bits).
- **Entorno de ejecución del generador:** Pyodide 0.27.7, utilizando Python 3.12.7 sobre WebAssembly.
- **Contenedores:** Docker 29.7.2, empleado para ejecutar UVLHub durante las pruebas de integración.

El equipo anfitrión utilizado fue un Slimbook Executive con un procesador Intel Core i7-13700H y 32 GB de memoria RAM. No obstante, estos datos no determinan directamente el rendimiento de la generación, ya que el código del generador se ejecuta en un único hilo de WebAssembly dentro del navegador.

8.2 Resultados

8.2.1 Resultados sobre el objetivo O1

El objetivo O1 consistía en desarrollar un generador automático capaz de producir modelos UVL con diferentes niveles de complejidad y expresividad, incluyendo

construcciones avanzadas del lenguaje que permitan ampliar la variedad de modelos disponibles para el análisis de variabilidad.

Para verificar este objetivo se han generado modelos utilizando distintas configuraciones del generador, comprobando que los parámetros seleccionados se reflejan correctamente en los modelos UVL obtenidos y que estos mantienen una estructura válida conforme al lenguaje.

La Figura 8.1 muestra la configuración utilizada para activar los niveles principales y secundarios de expresividad. Los modelos resultantes se incluyen en el Apéndice A.2.

YOUR SETUP	
1 · Batch	
Models	2
Seed	4266
2 · Language levels	
Majors	Boolean, Arithmetic, Type
Minors	Group card., Feature card., Aggregates, String ctc
3 · Feature tree	
Features	12 – 14
Depth ≤	6
Rel mix	opt 21% · mand 22% · alt 18% · or 26% · group 13%
Feat card.	50% [..]
Group card.	[3..6]
4 · Constraints	
Count	10 – 12
Vars/ctc	2 – 10
Type mix	B 31% · Z 24% · R 17% · S 28%
5 · Attributes	
Mode	random
Count	3 – 6
6 · Output	
Ensure SAT	no
Suffixes	—

Figura 8.1: Configuración usada para verificar O1. Elaboración propia

Los modelos completos generados durante esta comprobación se incluyen en el Apéndice A.2. El Listado A.2 muestra un modelo con características tipadas, atributos de diferentes tipos, cardinalidades, restricciones aritméticas, funciones agregadas y expresiones sobre cadenas.

El Listado A.3 presenta un segundo modelo obtenido con la misma configuración, pero con una estructura jerárquica y una combinación de atributos y restricciones diferente. Esto permite comprobar que el generador produce modelos distintos manteniendo los niveles de expresividad seleccionados.

Además de estas comprobaciones visuales, la prueba `test_o1_level_combinations_roundtrip_to_uvl_and_boolean_sat` recorre las 26 combinaciones válidas de niveles principales y secundarios. Para cada combinación se generan tres modelos, alcanzando un total de 78 modelos evaluados. Cada modelo se serializa a UVL y se vuelve a cargar mediante `UVLReader`. En las configuraciones booleanas, los modelos también se transforman a PySAT y se ejecutan mediante la operación de satisfacibilidad. Todas las combinaciones superaron correctamente este proceso, proporcionando evidencia automatizada de la compatibilidad de los modelos generados con las herramientas del ecosistema. Los resultados obtenidos permiten verificar el cumplimiento del criterio de éxito asociado al objetivo O1.

8.2.2 Resultados sobre el objetivo O2

El objetivo O2 consistía en desarrollar un sistema de configuración flexible que permitiera modificar los parámetros principales del proceso de generación y adaptar los modelos obtenidos a distintos escenarios.

Para verificar este objetivo se analizaron pruebas que comprueban tanto la propagación de los parámetros a través del asistente de configuración como su efecto sobre la estructura y el contenido de los modelos UVL generados. La Tabla 8.1 resume las comprobaciones realizadas.

Funcionalidad	Prueba automatizada	Comprobación realizada
Propagación de la configuración	<code>test_step2_configuration_isCorrect</code>	Comprobación de que las opciones seleccionadas en el segundo paso se trasladan correctamente al siguiente.
Cardinalidad de grupo	<code>test_group_cardinality_off</code> y <code>test_group_cardinality_on</code>	Verifica si se genera o no la generación de relaciones de cardinalidad y produce <code>group_groups</code>
Cardinalidad de características	<code>test_feature_cardinality_off</code> y <code>test_feature_cardinality_on</code>	Comprobación de que la opción modifica la salida UVL produce o elimina cardinalidades.
Niveles aritmético y de tipos	<code>test_arithmetic_level_produces</code> y <code>test_string_level_produces</code>	Verifica si se construyen expresiones aritméticas y restricciones mediante <code>len</code> .
Distribución de atributos	<code>test_dist_boolean_only_produces</code> y <code>test_dist_integer_only_produces</code> y <code>test_real_attribute_distribution</code>	Comprobación de que se generan los tipos de atributos configurados determinan los tipos de atributos generados.
Cantidad de restricciones y variables	<code>test_min_max_constraints_respected</code> y <code>test_min_vars_per_constraint_respected</code>	Verifica que la cantidad de restricciones y variables respetados los límites definidos.
Semillas	<code>test_determinism_same_seed</code> y <code>test_different_seed_different_output</code>	Verifica la reproducibilidad con la misma semilla y la variación con semillas diferentes.

Tabla 8.1: Pruebas técnicas del efecto de los parámetros de configuración.

Las pruebas anteriores no se limitan a comprobar que el asistente acepta los valores introducidos, sino que verifican su efecto sobre los modelos generados. Por ejemplo, al activar la cardinalidad de grupo se generan las relaciones correspondientes, mientras que al activar el nivel aritmético aparecen expresiones numéricas en las restricciones. De esta forma, existe una relación verificable entre la configuración seleccionada, el comportamiento del generador y el contenido del modelo UVL obtenido.

El criterio de éxito asociado al objetivo O2 se considera cumplido, ya que las pruebas confirman tanto la correcta propagación de los parámetros por el flujo de configuración como su influencia efectiva en los modelos generados.

8.2.3 Resultados sobre el objetivo O3

El objetivo O3 consistía en integrar el generador dentro del ecosistema Flamapy y permitir su utilización desde UVLHub como herramienta de generación de modelos de características.

La integración se verifica mediante el flujo real de ejecución utilizado por la aplicación. Los parámetros almacenados por UVLHub se convierten en una instancia de `FmgeneratorModel` mediante `from_flat_dict`. Posteriormente, el *wrapper* importa directamente la operación `GenerateFeatureModel`, crea una instancia y la ejecuta con el modelo de configuración recibido. Como resultado se obtiene un `FeatureModel` compatible con el metamodelo de Flamapy.

De forma independiente, la prueba

`test_generate_feature_model_is_discoverable_by_flamapy` verifica que Flamapy también localiza esta operación mediante `DiscoverMetamodels`. Así, se comprueba tanto su uso efectivo desde UVLHub como su registro como operación descubrible dentro del ecosistema Flamapy.

Esta integración se comprobó mediante las pruebas recogidas en la Tabla 8.2.

Prueba	Evidencia obtenida
<code>test_wrapper_generates_feature_model_with_received_arguments</code>	Comprueba que el <i>wrapper</i> recibe los parámetros y genera un modelo de características.
<code>test_wrapper_build_one_model_delegates_to_feature_model_generator</code>	Verifica que la generación se delega al componente GenerateFeatureModel .
<code>test_wrapper_serializes_uvl_and_reads_includes</code>	Comprueba la serialización del modelo mediante Flamapy y la conservación de las declaraciones include .
<code>test_o1_level_combinations_roundtrip_to_uvl_and_boolean_sat</code>	Genera 78 modelos, los serializa, los vuelve a cargar mediante UVLReader y transforma los modelos booleanos a PySAT.
Suite de UVLHub	220 pruebas superadas, incluyendo los flujos del <i>wizard</i> y la generación desde la aplicación.
<code>test_generate_feature_model_is_discoverable_by_flamapy</code>	Comprueba que DiscoverMetamodels localiza la operación GenerateFeatureModel , que esta puede ejecutarse y que devuelve un FeatureModel válido.
Suite de <code>fm_generator</code>	44 pruebas superadas sobre el paquete independiente, incluyendo la comprobación de descubrimiento de la operación por Flamapy.

Tabla 8.2: Pruebas técnicas de integración con Flamapy y UVLHub.

La evidencia anterior demuestra que el componente no se limita a estar presente en la estructura del proyecto, sino que es utilizado durante la ejecución, produce un **FeatureModel** de Flamapy y genera modelos UVL que pueden ser consumidos posteriormente por otras herramientas del ecosistema.

Por tanto, el criterio de éxito asociado al objetivo O3 se considera cumplido.

8.2.4 Resultados sobre el objetivo O4

El objetivo O4 consistía en evaluar el comportamiento temporal del generador en un escenario con un volumen elevado de modelos de características.

Para ello se utilizó una configuración fija de 1000 modelos UVL, manteniendo constantes los niveles de expresividad, el número de características, los atributos y las restricciones. La configuración utilizada se muestra en la Figura 8.2.

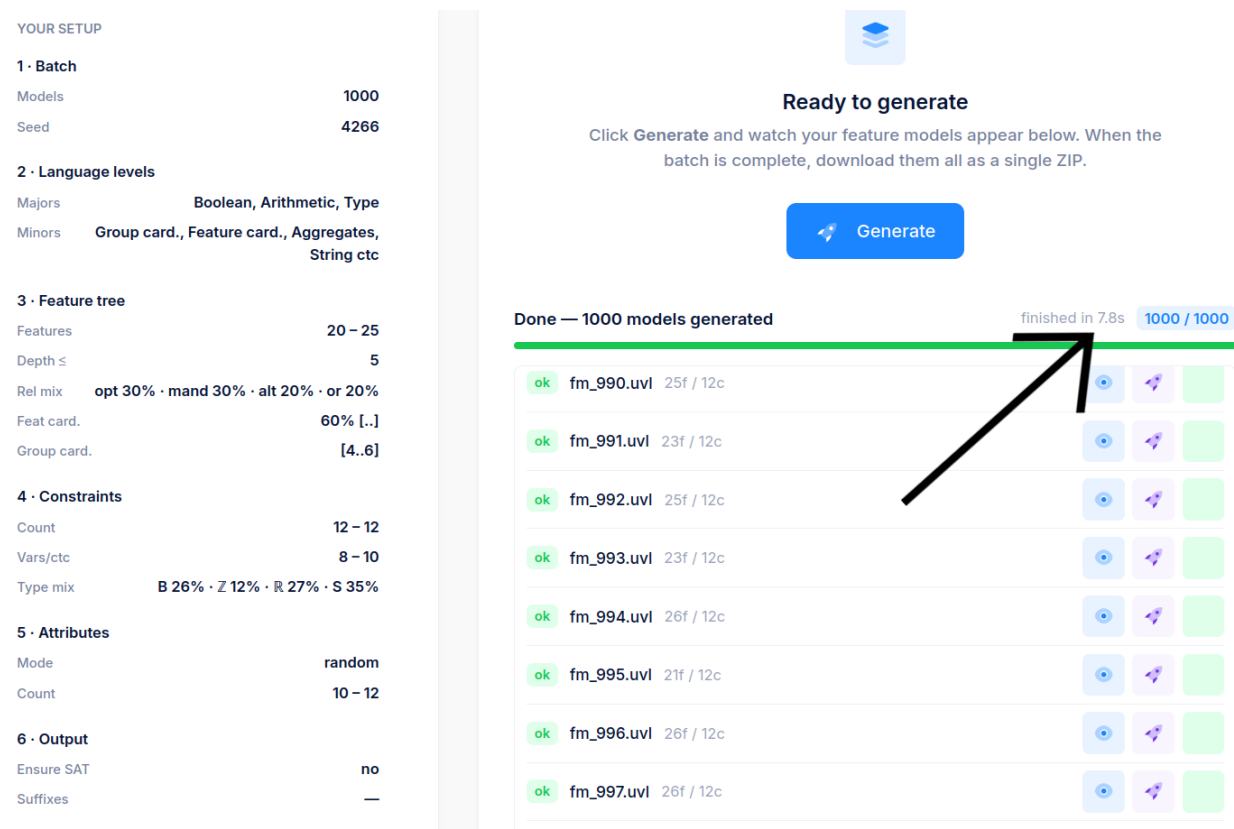


Figura 8.2: Configuración utilizada para la evaluación temporal. Elaboración propia.

La evaluación se repitió diez veces con la misma configuración. En cada repetición se registraron por separado el tiempo de inicialización del entorno, incluyendo la carga de Pyodide y sus dependencias, y el tiempo posterior de generación de los 1000 modelos UVL.

Los tiempos se midieron manualmente utilizando el cronómetro de un teléfono móvil, cuya resolución permite registrar valores con dos decimales. No obstante, la toma manual introduce un margen de error asociado al tiempo de reacción, por lo que se realizaron diez repeticiones y se analizaron medidas de tendencia central y dispersión. La Tabla 8.3 recoge la media, la mediana, la desviación típica muestral y el rango observado en cada fase. El tiempo total se ha calculado sumando, en cada repetición, los tiempos de inicialización y generación correspondientes.

Fase	N	Media	Mediana	Desv. típica	Rango
Inicialización del entorno	10	4,28 s	4,32 s	0,24 s	3,95–4,65 s
Generación de 1000 modelos UVL	10	9,16 s	9,12 s	0,19 s	8,85–9,46 s
Tiempo total	10	13,44 s	13,48 s	0,34 s	12,80– 14,10 s

Tabla 8.3: Resultados de las diez repeticiones de la evaluación temporal.

La inicialización del entorno presentó un tiempo medio de 4,28 segundos y una desviación típica de 0,24 segundos. Una vez cargadas las dependencias, la generación de los 1000 modelos UVL presentó un tiempo medio de 9,16 segundos, con una desviación típica de 0,19 segundos. El tiempo total medio, considerando ambas fases, fue de 13,44 segundos.

La generación mostró una variabilidad reducida, ya que los tiempos observados se situaron entre 8,85 y 9,46 segundos. La inicialización presentó una dispersión ligeramente superior, debido principalmente a la carga de Pyodide y de sus dependencias. Estos resultados permiten distinguir el coste inicial de preparación del entorno del coste propio de generar los modelos.

La ejecución se realizó en Firefox mediante Pyodide 0.27.7, utilizando Python 3.12.7 sobre WebAssembly. La generación se ejecuta en un único hilo de WebAssembly, por lo que los resultados caracterizan el comportamiento de esta configuración concreta y no deben interpretarse como un límite general del generador. El tiempo puede aumentar al utilizar modelos con un mayor número de características, atributos o restricciones.

8.2.5 Resultados sobre el objetivo O5

El objetivo O5 consistía en validar el correcto funcionamiento del generador mediante una batería de pruebas automatizadas que cubriese tanto la lógica interna del motor de generación como el flujo de configuración integrado en UVLHub.

Para verificar este objetivo se ha ejecutado la suite de pruebas del módulo *generator* dentro del entorno Docker de la aplicación. Estas pruebas validan diferentes componentes del sistema, incluyendo la generación de modelos UVL a partir de distintas configuraciones, la aplicación de los parámetros del asistente de generación, la persistencia del estado entre etapas del flujo, las opciones de salida generadas y el comportamiento del generador ante diferentes niveles de expresividad del lenguaje UVL, verificando que las configuraciones relativas a atributos, restricciones, cardinalidades y operadores avanzados producen los resultados esperados.

La ejecución completa de la suite mediante *rosemary* finalizó correctamente, obteniéndose 220 pruebas superadas en UVLHub. A estas se añaden las 44 pruebas del paquete *fm_generator*, por lo que la suite completa reúne 264 pruebas automatizadas, todas superadas. En la Figura 8.3 se muestra el resultado de la ejecución de las pruebas de UVLHub, mientras que la Figura 8.4 muestra la ejecución de las pruebas del *fm_generator*.

```
(base) antonio@antonio:~/Escritorio/TF6/uvlhub$ docker compose -f docker/docker-compose.dev.yml exec web rosemary test generator --all
app/features/generator/tests/test_wizard_outputs.py::test_generate_sat_models_raises_after_twenty_failed_attempts PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step2_type_enables_arithmetic PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step3_normalizes_relation_distribution PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step4_disables_arithmetic_probs_when_arithmetic_off PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step5_random_attrs_normalizes_distribution PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step3_normalizes_relation_distribution_with_group_cardinality PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step5_masks_unavailable_attribute_types_when_levels_are_off PASSED
app/features/generator/tests/test_wizard_persisters.py::test_safe_float_accepts_comma_and_invalid_values_fall_back PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step1_persists_batch_fields PASSED
app/features/generator/tests/test_wizard_persisters.py::test_add_level_flags_copies_all_level_values PASSED
app/features/generator/tests/test_wizard_persisters.py::test_collect_manual_attributes_keeps_supported_types_only PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step4_falls_back_when_extra_repr_and_ctc_dist_are_invalid PASSED
app/features/generator/tests/test_wizard_persisters.py::test_apply_step6_output_persists_checkboxes PASSED
app/features/generator/tests/test_wizard_state.py::test_save_step_state_preserves_checkbox_false_values PASSED
app/features/generator/tests/test_wizard_state.py::test_load_step_state_merges_saved_values_over_defaults PASSED
app/features/generator/tests/test_wizard_state.py::test_save_step_state_does_not_overwrite_other_steps PASSED
app/features/generator/tests/test_wizard_state.py::test_step_state_is_preserved_after_returning_to_previous_step PASSED
===== 220 passed in 114.41s (0:01:54) =====
```

Figura 8.3: Resultado de la ejecución de la suite de pruebas del módulo *generator*.
Elaboración propia

```
tests/test_engine.py::test_generated_model_declares_correct_uvl_includes[overrides3-expected3] PASSED
tests/test_engine.py::test_generated_model_declares_correct_uvl_includes[overrides4-expected4] PASSED
tests/test_engine.py::test_generated_model_declares_correct_uvl_includes[overrides5-expected5] PASSED
tests/test_engine.py::test_feature_distribution_controls_feature_type[FEATURE_DIST_INTEGER-FeatureType.INTEGER] PASSED
tests/test_engine.py::test_feature_distribution_controls_feature_type[FEATURE_DIST_REAL-FeatureType.REAL] PASSED
tests/test_engine.py::test_feature_distribution_controls_feature_type[FEATURE_DIST_STRING-FeatureType.STRING] PASSED
tests/test_engine.py::test_manual_mode_generates_all_supported_attribute_types PASSED
tests/test_engine.py::test_real_attribute_distribution_generates_real_attributes PASSED
tests/test_engine.py::test_zero_len_probability_generates_string_equality_constraints PASSED
tests/test_engine.py::test_relation_kinds_create_expected_cardinalities PASSED
tests/test_engine.py::test_generate_feature_model_is_discoverable_by_flamapy PASSED
tests/test_params_contract.py::test_wizard_referenced_params_keys_are_mapped_to_generator_model PASSED
tests/test_params_contract.py::test_extra_constraint_representativeness_matches_integer_form_contract PASSED
tests/test_params_contract.py::test_generator_model_defaults_are_valid_for_generation_contract PASSED
tests/test_params_contract.py::test_generator_model_rejects_relation_distribution_that_does_not_sum_to_one PASSED
tests/test_params_contract.py::test_generator_model_rejects_boolean_operator_distribution_that_does_not_sum_to_one PASSED
===== 44 passed in 0.27s =====
```

Figura 8.4: Resultado de la ejecución de la suite de pruebas del *fm_generator*.
Elaboración propia

Los resultados obtenidos permiten verificar el cumplimiento del criterio de éxito asociado al objetivo O5, demostrando la estabilidad de las funcionalidades desarrolladas mediante pruebas automatizadas.

8.2.6 Resultados sobre los objetivos formativos O6–O10

Los objetivos O6–O10 se refieren a competencias adquiridas durante el desarrollo del proyecto. A diferencia de los objetivos funcionales, no se evalúan mediante una métrica de rendimiento, sino mediante evidencias documentales y técnicas generadas durante el trabajo.

Obj.	Evidencia	Resultado
O6	Estudio del enfoque de generación propuesto por Sundermann et al. y adaptación de sus bases conceptuales al componente implementado.	El conocimiento científico revisado se ha aplicado en el diseño y desarrollo del generador.
O7	Integración de un componente nuevo en los proyectos Flamapy y UVLHub, utilización de Docker y adaptación de pruebas existentes.	El desarrollo se ha realizado sobre un ecosistema software real, con código previo y componentes reutilizables.
O8	Implementación en Python, adaptación de rutas y asistentes de Flask, ejecución mediante Docker y desarrollo de pruebas automatizadas.	Las tecnologías principales del proyecto se han utilizado de forma práctica durante la construcción e integración de la solución.
O9	Consulta de bibliografía científica, documentación técnica de Flamapy, UVL y Pyodide, y justificación de decisiones de diseño.	Las decisiones técnicas se han apoyado en información seleccionada y analizada durante el desarrollo.
O10	Planificación del trabajo, gestión de riesgos, documentación de diseño, implementación, pruebas y resultados.	El proyecto se ha gestionado y documentado de forma continuada durante sus distintas fases.

Tabla 8.4: Evidencias asociadas a los objetivos formativos. Elaboración propia.

La Tabla 8.4 permite considerar alcanzados los objetivos formativos planteados. Estas evidencias no pretenden medir de forma cuantitativa el nivel individual de

competencia adquirido, sino documentar las actividades, decisiones y artefactos mediante los que dichas competencias se han aplicado durante el desarrollo del proyecto.

8.3 Comparación con trabajos previos

La Tabla 2.2 del Capítulo 2 recoge una comparación cualitativa entre diferentes herramientas relacionadas con la generación automática de modelos de características. En ella se consideran tres aspectos principales: la capacidad de generación automática, el grado de personalización ofrecido y la incorporación de construcciones de expresividad avanzada.

Los resultados obtenidos durante la evaluación del generador desarrollado permiten observar que la solución propuesta cubre los tres criterios analizados. A diferencia de S.P.L.O.T. y FeatureIDE, cuyas capacidades de generación están principalmente orientadas a modelos basados en construcciones booleanas, el generador desarrollado permite producir modelos UVL incorporando elementos adicionales como atributos tipados, restricciones aritméticas, restricciones sobre cadenas, funciones agregadas y diferentes mecanismos de cardinalidad.

En comparación con BeTTY, la solución desarrollada mantiene la capacidad de configuración flexible de los parámetros de generación, pero amplía el rango de expresividad de los modelos generados. Mientras que BeTTY permite definir escenarios experimentales con diferentes niveles de complejidad estructural, el generador desarrollado incorpora además construcciones propias de lenguajes de variabilidad más recientes como UVL.

Además, la integración realizada dentro del ecosistema Flamapy y UVLHub supone una diferencia respecto a las herramientas analizadas, ya que permite utilizar el generador directamente dentro de una plataforma web orientada a la gestión y análisis de modelos de características, manteniendo la compatibilidad con las herramientas existentes del ecosistema.

No se ha realizado una comparación cuantitativa directa en términos de tiempo de generación o volumen máximo de modelos, debido a que las herramientas estudiadas no proporcionan escenarios de evaluación homogéneos que permitan establecer una comparación justa, por lo que la comparación se ha centrado en las capacidades funcionales y de expresividad.

En consecuencia, los resultados obtenidos muestran que el trabajo desarrollado amplía las capacidades de los generadores existentes, especialmente en cuanto a la generación de modelos con mayor expresividad y su integración dentro del ecosistema UVLHub, lo cual permite afirmar que efectivamente el trabajo realizado en este TFG no queda

por detrás del resto a pesar de que herramientas como BeTTy cuenten con una mayor madurez histórica y una validación más extensa en escenarios experimentales.

8.4 Discusión

Los resultados obtenidos muestran que la solución desarrollada responde al problema planteado inicialmente: la necesidad de disponer de un generador capaz de producir modelos de características con una mayor expresividad que las aproximaciones tradicionales basadas principalmente en construcciones booleanas. La integración del generador dentro del ecosistema Flamapy y UVLHub permite disponer de una herramienta flexible para la generación de modelos UVL destinados a tareas posteriores de análisis y experimentación.

Durante el desarrollo se encontraron diferentes dificultades relacionadas con la integración y adaptación de los componentes existentes. Sin embargo, la evaluación final confirmó que el generador soporta correctamente las funcionalidades planteadas inicialmente, incluyendo los diferentes niveles de expresividad y las distintas configuraciones disponibles.

La incorporación de construcciones avanzadas implica ciertos compromisos. Por un lado, aumentar la expresividad mediante atributos tipados, restricciones aritméticas, funciones agregadas o restricciones sobre cadenas incrementa la complejidad interna del generador. Por otro lado, ofrecer una configuración más flexible permite adaptar la generación a distintos escenarios, aunque aumenta el número de parámetros que el usuario debe comprender. Asimismo, una mayor riqueza en los modelos generados puede suponer un incremento del coste computacional en configuraciones más complejas.

Finalmente, existen algunas amenazas a la validez de los resultados. La evaluación confirma la compatibilidad sintáctica de los modelos generados con UVL y con las transformaciones empleadas, pero no determina si dichos modelos son representativos de dominios reales ni si resultan adecuados como conjunto de evaluación para motores de razonamiento. Asimismo, los resultados temporales se han obtenido con una única configuración de generación y en un entorno concreto, por lo que podrían variar al modificar el tamaño o la complejidad de los modelos. Tampoco se ha realizado una comparación cuantitativa directa con otras herramientas bajo condiciones experimentales equivalentes.

Los resultados obtenidos y la discusión realizada permiten establecer el grado de cumplimiento de los objetivos planteados y validar la solución desarrollada frente a los criterios de éxito definidos. A partir de este análisis, el Capítulo 9 aborda las conclusiones finales del trabajo, las limitaciones identificadas y las posibles líneas de trabajo futuro derivadas de los resultados obtenidos.

9 CONCLUSIONES Y TRABAJO FUTURO

9.1 Conclusiones

El objetivo principal de este trabajo era desarrollar un generador automático de modelos de características capaz de producir modelos UVL con diferentes niveles de complejidad y expresividad, integrándose dentro del ecosistema Flamapy y UVLHub. Para ello, se ha desarrollado una solución configurable que permite generar modelos con construcciones avanzadas del lenguaje UVL y validar su correcto funcionamiento mediante pruebas automatizadas.

Los resultados obtenidos en el Capítulo 8 permiten concluir que los objetivos funcionales y formativos planteados inicialmente han sido alcanzados.

El objetivo O1, relacionado con la generación automática de modelos UVL con distintos niveles de expresividad, se ha cumplido mediante la generación y validación de modelos con diferentes configuraciones. La prueba asociada recorre las combinaciones válidas de niveles de expresividad y verifica la serialización y recarga de los modelos generados (Sección 8.2.1).

El objetivo O2 se ha cumplido mediante la incorporación de un sistema de configuración que permite modificar los parámetros principales del proceso de generación. Las pruebas técnicas verifican que las configuraciones seleccionadas producen efectos observables sobre la jerarquía, las cardinalidades, los atributos y las restricciones generadas (Sección 8.2.2).

El objetivo O3 se ha alcanzado mediante la integración del generador con Flamapy y UVLHub. La evidencia técnica verifica tanto la generación desde el *wrapper* ejecutado en UVLHub como el descubrimiento de la operación `GenerateFeatureModel` mediante `DiscoverMetamodels` (Sección 8.2.3).

El objetivo O4 se ha verificado mediante diez repeticiones de una evaluación temporal con 1000 modelos UVL. La evaluación distingue el coste de inicialización del entorno del tiempo propio de generación (Sección 8.2.4).

El objetivo O5 se ha cumplido mediante la ejecución satisfactoria de las suites automatizadas de UVLHub y del paquete `fm_generator`, que reúnen 264 pruebas superadas (Sección 8.2.5).

Respecto a los objetivos formativos, O6 se ha alcanzado mediante la aplicación del enfoque científico revisado en el diseño del generador. O7 y O8 se han cumplido a través de la participación en la evolución de un proyecto software real, utilizando Python, Flask, Docker y pruebas automatizadas. Por su parte, O9 se ha abordado mediante la consulta y análisis de bibliografía y documentación técnica, mientras

que O10 se ha materializado en la planificación, documentación y seguimiento del proyecto. Las evidencias correspondientes se recogen en la Sección 8.2.6.

La Tabla 9.1 resume el grado de cumplimiento de los objetivos definidos.

Obj.	Objetivo	Grado	Evidencia
O1	Generación automática de modelos UVL con expresividad avanzada.	Completo	Sección 8.2.1.
O2	Configuración flexible del proceso de generación.	Completo	Sección 8.2.2.
O3	Integración con Flamapy y UVLHub.	Completo	Sección 8.2.3.
O4	Evaluación temporal del generador.	Completo	Sección 8.2.4.
O5	Validación mediante pruebas automatizadas.	Completo	Sección 8.2.5.
O6	Reutilización de conocimiento científico en el desarrollo software.	Completo	Sección 8.2.6.
O7	Experiencia en un proyecto software real y de código abierto.	Completo	Sección 8.2.6.
O8	Aplicación práctica de Python, Flask, Docker y pruebas automatizadas.	Completo	Sección 8.2.6.
O9	Búsqueda, análisis y selección de información técnica relevante.	Completo	Sección 8.2.6.
O10	Planificación, gestión y documentación del proyecto.	Completo	Sección 8.2.6.

Tabla 9.1: Grado de cumplimiento de los objetivos y evidencia asociada. Elaboración propia.

Como aportación principal, este trabajo proporciona una solución integrada para la generación automática de modelos UVL con mayor expresividad que los generadores tradicionales centrados en construcciones booleanas. Además, la integración dentro del ecosistema Flamapy y UVLHub permite utilizar esta funcionalidad en escenarios de experimentación y análisis de variabilidad.

A nivel personal y técnico, el desarrollo ha permitido identificar la importancia de realizar una planificación temporal con margen para imprevistos, así como la necesidad de utilizar desde el inicio herramientas que faciliten un entorno reproducible, como Docker. También se ha comprobado la relevancia de realizar refactorizaciones progresivas y acompañar los cambios funcionales con pruebas automatizadas para evitar la acumulación de errores durante la evolución del software.

En conjunto, los resultados obtenidos permiten concluir que la solución desarrollada cumple los objetivos establecidos y proporciona una base extensible para futuras mejoras relacionadas con la generación y análisis de modelos de características.

9.2 Limitaciones

A pesar de los resultados obtenidos, existen ciertas limitaciones que deben tenerse en cuenta al interpretar el alcance del trabajo realizado. Estas limitaciones delimitan el alcance de las conclusiones obtenidas y los escenarios en los que se ha validado la solución.

Desde un punto de vista técnico, la evaluación del generador se ha realizado sobre configuraciones representativas, pero no se ha estudiado su comportamiento en escenarios extremos con modelos de características de gran tamaño o con una complejidad significativamente superior. Esto puede afectar al coste computacional en configuraciones con un número elevado de características, atributos o restricciones. No obstante, la arquitectura desarrollada permite continuar optimizando y ampliando el generador para este tipo de escenarios.

Respecto a la validación, las pruebas realizadas permiten comprobar que los modelos generados se serializan correctamente en UVL, pueden volver a cargarse mediante `UVLReader` y, en las configuraciones booleanas, pueden transformarse a PySAT. Estas comprobaciones aportan evidencia sobre la validez sintáctica y la compatibilidad de los modelos con las herramientas del ecosistema. Sin embargo, no se ha realizado una evaluación específica de la calidad de los modelos generados en términos de consistencia semántica, representatividad respecto a dominios reales o utilidad práctica para evaluar motores de razonamiento. Tampoco se ha comparado el comportamiento de distintos razonadores utilizando los modelos producidos como conjunto de evaluación. Finalmente, en cuanto al alcance del trabajo, la solución desarrollada se ha centrado en la generación automática de modelos UVL y su integración dentro del ecosistema Flamapy y UVLHub. No se han abordado otras líneas relacionadas, como la optimización avanzada del proceso de generación, nuevas estrategias de búsqueda o análisis posterior de los modelos generados, ya que quedan fuera de los objetivos definidos inicialmente.

9.3 Trabajo futuro

A partir de las limitaciones identificadas y de las posibilidades de extensión del sistema desarrollado, se plantean diferentes líneas de trabajo futuro orientadas a mejorar la reutilización de configuraciones, la integración con UVLHub y la capacidad del generador para adaptarse a nuevos escenarios de uso.

Exportación e importación de configuraciones

Una primera línea de trabajo consistiría en desarrollar una transformación de Flamapy que permita exportar la configuración seleccionada del generador en un formato estándar como JSON, permitiendo posteriormente importar dicha configuración desde UVLHub para reproducir una generación concreta.

Esta extensión mejoraría la reproducibilidad de los experimentos y facilitaría el intercambio de configuraciones entre usuarios, evitando la necesidad de definir manualmente conjuntos complejos de parámetros. Además, permitiría compartir escenarios de generación de forma sencilla dentro del ecosistema de herramientas de variabilidad.

La implementación de esta funcionalidad presenta una viabilidad elevada, ya que los parámetros del generador ya se encuentran estructurados internamente. Sería necesario únicamente definir un esquema estable de serialización y añadir los mecanismos de exportación e importación correspondientes.

Publicación directa de modelos en UVLHub

Otra posible extensión consistiría en ampliar la integración con UVLHub para permitir subir directamente los modelos generados a la plataforma, pudiendo establecer su visibilidad y compartirlos con otros usuarios.

Actualmente, el generador permite producir modelos UVL, pero la gestión posterior de estos modelos requiere pasos adicionales. La incorporación de esta funcionalidad permitiría centralizar el flujo completo, desde la generación hasta el almacenamiento y distribución de modelos de características.

Esta línea presenta una viabilidad media-alta, ya que requeriría ampliar la comunicación entre el generador y los servicios existentes de UVLHub, además de definir aspectos relacionados con permisos, almacenamiento y gestión de modelos públicos.

Validación de representatividad y evaluación con razonadores

Una línea de trabajo futuro consistiría en evaluar la calidad de los modelos generados tomando como referencia colecciones de modelos de características procedentes de dominios reales. El objetivo no sería utilizar estos modelos como entrada del generador, sino comparar sus propiedades estructurales y de expresividad con las de los modelos producidos, por ejemplo en número de características, profundidad, relaciones, atributos y restricciones.

Esta evaluación permitiría determinar hasta qué punto las configuraciones del generador producen modelos representativos de escenarios reales. Asimismo, permitiría definir perfiles de generación basados en características observadas en conjuntos de referencia, aumentando la utilidad de la herramienta para la experimentación.

Como paso posterior, los modelos generados podrían utilizarse para evaluar motores de razonamiento bajo condiciones controladas. Esto permitiría analizar el impacto de diferentes niveles de expresividad sobre operaciones como la comprobación de satisfacibilidad, el cálculo de configuraciones o las transformaciones entre metamodelos.

Optimización del proceso de generación

Una posible línea de trabajo futuro consistiría en optimizar el proceso interno de generación para mejorar el rendimiento en configuraciones con una mayor complejidad estructural o un volumen elevado de modelos generados.

Aunque los resultados obtenidos muestran un comportamiento adecuado en los escenarios evaluados, el coste computacional puede incrementarse al aumentar el número de características, atributos o restricciones. Por ello, sería interesante analizar los puntos críticos del proceso de generación y aplicar técnicas de optimización, como mejoras en la gestión de estructuras internas, reducción de operaciones redundantes o paralelización de determinadas etapas.

Esta extensión presenta una viabilidad media-alta, ya que no requiere modificar la arquitectura general del sistema, sino realizar mejoras sobre los componentes existentes del motor de generación. La principal dificultad reside en identificar aquellas operaciones que tienen mayor impacto sobre el tiempo de ejecución y mantener la corrección de los modelos generados tras las optimizaciones.

Generación guiada mediante aprendizaje automático

Como línea de investigación más ambiciosa, se plantea la incorporación de técnicas de aprendizaje automático para aprender patrones presentes en modelos de características reales y utilizarlos durante el proceso de generación.

Esta aproximación permitiría evolucionar desde una generación basada únicamente en parámetros definidos manualmente hacia una generación capaz de producir modelos

sintéticos adaptados a las características de dominios concretos. De esta forma, se podrían obtener modelos más realistas para tareas de evaluación y experimentación. Su viabilidad es media, ya que requeriría disponer de colecciones de referencia adecuadas, definir métricas de comparación y establecer una metodología experimental para evaluar los razonadores en condiciones equivalentes. No obstante, constituye una posible evolución innovadora del generador desarrollado.

BIBLIOGRAFÍA

- [1] J. A. Zamudio. *Líneas de Producto Software para acelerar el desarrollo de aplicaciones relacionadas*. 2023. URL: <https://ingenieriadesoftware.es/lineas-producto-software/> (visitado 05-08-2026).
- [2] A. Felfernig, A. Falkner y D. Benavides. *Feature Models. AI-Driven Design, Analysis and Applications*. 1.^a ed. Springer, 2024. ISBN: 978-3-031-61874-1.
- [3] D. Benavides, C. Sundermann, K. Feichtinger, J. A. Galindo, R. Rabiser y T. Thüm. «UVL: Feature modelling with the Universal Variability Language». En: *ELSEVIER* 225 (2025). DOI: 10.1016/j.jss.2024.112326.
- [4] U. de Sevilla, U. de Málaga, U. of Graz y UNED. *Flamapy: The cutting-edge Python-based tool for Automated Analysis of Feature Models using UVL and more*. URL: <https://www.flamapy.org/> (visitado 05-08-2026).
- [5] *TASOVA Plus: Red en nuevas Tendencias en Arquitectura Software y Variabilidad*. 2023. URL: <https://tasovaplus.github.io/> (visitado 05-08-2026).
- [6] *UVLHub, a repository of feature models in UVL (Universal Variability Language) format, integrated with Zenodo and flamapy, and developed following Open Science*. 2023. URL: <https://docs.uvlhub.io/> (visitado 05-08-2026).
- [7] C. Sundermann, T. Heß, R. Sundermann, E. Kuitert, S. Krieter y T. Thüm. «Generating Feature Models with UVL's Full Expressiveness». En: *Proceedings of the 28th ACM International Systems and Software Product Line Conference. SPLC '24*. New York, NY, USA: Association for Computing Machinery, 2024, págs. 61-65. DOI: 10.1145/3646548.3676602.
- [8] P. Clements y L. Northrop. *Software Product Lines: Practices and Patterns*. 2002. ISBN: 9780201703320.
- [9] I. de Software. *Líneas de Producto Software*. 2025. URL: <https://ingenieriadesoftware.es/lineas-producto-software/> (visitado 15-08-2026).
- [10] J. van Gurp, J. Bosch y M. Svahnberg. «On the Notion of Variability in Software Product Lines». En: *Proceedings Working IEEE/IFIP Conference on Software Architecture*. IEEE Computer Society, 2001, págs. 45-54. DOI: 10.1109/WICSA.2001.948406.
- [11] K. C. Kang, S. G. Cohen, J. A. Hess, W. E. Novak y A. S. Peterson. «Feature-Oriented Domain Analysis (FODA) Feasibility Study». En: Software Engineering Institute, Carnegie Mellon University, 1990.

- [12] D. Benavides, S. Segura y A. Ruiz-Cortés. «Automated Analysis of Feature Models 20 Years Later: A Literature Review». En: *Information Systems* 35.6 (2010), págs. 615-636.
- [13] D. Benavides, C. Sundermann, K. Feichtinger, J. A. Galindo, R. Rabiser y T. Thüm. «UVL: Feature modelling with the Universal Variability Language». En: *The Journal of Systems and Software* 225 (2025), pág. 112326. DOI: 10.1016/j.jss.2025.112326.
- [14] R. E. Bryant. «Graph-Based Algorithms for Boolean Function Manipulation». En: *IEEE Transactions on Computers* C-35.8 (1986), págs. 677-691. DOI: 10.1109/TC.1986.1676819.
- [15] U. de Sevilla. *Proyecto - 24/25: Documentación general de UVLHub*. Wiki de la asignatura Evolución y Gestión de la Configuración. URL: https://1984.1si.us.es/wiki-egc/index.php/Proyecto_-_24/25#Documentaci.C3.B3n_general.
- [16] S. Segura, R. M. Hierons, D. Benavides y A. Ruiz-Cortés. «Automated Metamorphic Testing on the Analyses of Feature Models». En: *Information and Software Technology* 53.3 (2011), págs. 245-258. DOI: 10.1016/j.infsof.2010.11.002.
- [17] J. A. Galindo, A. J. Dominguez, J. White y D. Benavides. «Large Language Models to Generate Meaningful Feature Model Instances». En: *Proceedings of the 27th ACM International Systems and Software Product Line Conference*. ACM, 2023, págs. 15-26. DOI: 10.1145/3579027.3608973.
- [18] M. Mendonça, M. Branco y D. Cowan. «S.P.L.O.T. - Software Product Lines Online Tools». En: *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications*. 2009, págs. 761-762.
- [19] T. Thüm, C. Kästner, F. Benduhn, J. Meinicke, G. Saake y T. Leich. «FeatureIDE: An Extensible Framework for Feature-Oriented Software Development». En: *Science of Computer Programming* 79 (2014), págs. 70-85.
- [20] S. Segura, J. Galindo, D. Benavides, J. A. Parejo y A. Ruiz-Cortés. «BeTTy: Benchmarking and Testing on the Automated Analysis of Feature Models». En: *Proceedings of the 6th International Workshop on Variability Modelling of Software-intensive Systems (VaMoS)*. 2012, págs. 63-70.
- [21] S. Segura, J. A. Parejo, R. M. Hierons, D. Benavides y A. Ruiz-Cortés. «Automated Generation of Computationally Hard Feature Models using Evolutionary Algorithms». En: *Expert Systems with Applications* 40.17 (2013), págs. 6935-6946.

-
- [22] UVLHub. *Rosemary: UVLHub Generation Framework Documentation*. 2026.
URL: <https://docs.uvlhub.io/rosemary> (visitado 05-09-2026).

A APÉNDICE

A.1 Ejemplo de modelo de características en UVL

```
1 include
2     Boolean.group-cardinality
3     Arithmetic.*
4     Type.string-constraints
5
6 features
7     Smartwatch
8         mandatory
9             Display
10                alternative
11                    LCD
12                    OLED
13                    "E-Ink"
14            Connectivity
15                or
16                    Bluetooth
17                    "Wi-Fi"
18                    LTE
19            Sensors
20                mandatory
21                    "Heart Rate Monitor"
22                optional
23                    GPS
24                    "Blood Oxygen Sensor" {Accuracy 95}
25                    Accelerometer
26                    "Fall Detection"
27                    Gyroscope
28            Battery
29                alternative
30                    Standard {Capacity 300}
31                    "Long Life" {Capacity 450}
32            String "Serial Number"
33        optional
34            "Music Player" {Storage 2}
35            NFC
```

```

36         "Voice Assistant"
37         "Water Resistance"
38         alternative
39             IP67
40             IP68
41             IPX8
42         "Watch Faces" cardinality [5..50]
43         Accessories
44             [0..2]
45             "Leather Strap"
46             "Metal Strap"
47             "Silicone Strap"
48         String Color
49         Real Weight
50
51
52     constraints
53         LTE => (GPS & Bluetooth)
54         IPX8 => (!LCD)
55         "Fall Detection" => (Accelerometer & Gyroscope)
56         "Voice Assistant" <=> "Wi-Fi"
57         sum(Capacity) >= 400
58         avg(Accuracy) >= 90
59         len("Serial Number") == 10
60         Weight > 30.0
61         Color != "Pink"

```

Código A.1: Ejemplo de modelo de características en UVL. Usado en la Sección 2.2

A.2 Modelos UVL generados para la evaluación de O1

Los siguientes listados muestran dos de los modelos UVL generados durante la evaluación del objetivo O1. Ambos proceden de la configuración mostrada en la Figura 8.1.

```

1  include
2      Boolean.group-cardinality
3      Arithmetic.*
4      Type.string-constraints
5  features
6      F0
7          mandatory
8              F4
9          alternative
10             F3

```

```

11      String F2 cardinality [2..2]
12      optional
13      F5
14      alternative
15      F13
16      F12
17  mandatory
18      F1
19      optional
20      Real F8 cardinality [2..4]
21      optional
22      F7 cardinality [3..4] {Attr0 79}
23      mandatory
24      String F9 cardinality [5..5] {Attr3 70.19, Attr11 false}
25      or
26      Real F18
27      F17 {Attr2 83}
28      F16 {Attr1 60.73, Attr6 'dxNLfjcFKqMHg691YECu'}
29      Integer F15 cardinality [2..3] {Attr5 true}
30      mandatory
31      Integer F19
32      optional
33      F6 cardinality [2..3]
34      or
35      Integer F11 cardinality [3..5]
36      Real F10 cardinality [3..5] {Attr4 '
37      ekkWGoALOnp5MvFrFXGpbeeWBdKe1ACkpHn3EhF2i', Attr10 true}
38      mandatory
39      Integer F14 {Attr7 true, Attr9 'OLQDutbeDT33hQtmvHrMsC46Jw3nx9'}
40      optional
41      Real F23
42      optional
43      F22 cardinality [5..5]
44      optional
45      Real F21 cardinality [3..3]
46      optional
47      Integer F20 cardinality [2..5] {Attr8 54.87}
48 constraints
49      (((len(F10.Attr4) <= len(F9)) <=> ((F3 => F12) & ! F22)) & F4) | F14.Attr7
50      (((F6 & F13) <=> (F4 | F22)) & (F10 <= sum(F15, F20.Attr8))) | (len(F9) >= len(F10.Attr4))
51      (((F9 == F2) => (F14.Attr9 == F2)) & ((len(F10.Attr4) / F15) >= F21)) & (F3 & F17)
52      ((F1 & ((F16.Attr6 == F10.Attr4) | (F2 == F10.Attr4))) & ((F18 - F8) > F9.Attr3)) => (len(F16.
53      Attr6) >= len(F9))
54      (((F15 < F11) & (F7.Attr0 >= (F18 / F21))) | F7) | (F19 == F14)) | F13
55      ((len(F9) < len(F16.Attr6)) <=> (len(F2) >= ((F15 / F11) - F7.Attr0))) & ((F19 - len(F2)) <= F8)
56      (((F7.Attr0 - F8) + len(F9)) >= F21) & (len(F10.Attr4) >= (len(F2) * len(F14.Attr9))) & (len(
57      F9) == len(F14.Attr9))
58      ((F23 < (len(F9) / F21)) | (((F8 - F18) / len(F16.Attr6)) >= F23)) | F9.Attr11
59      (((F11 - len(F2)) <= F23) => ((len(F10.Attr4) - len(F16.Attr6)) > len(F14.Attr9))) & ! F4
60      (((F16.Attr1 - F23) <= F19) | (F15 <= F9.Attr3)) <=> (F16.Attr1 == avg(F11, F9.Attr3))) & F7
61      (((len(F16.Attr6) >= len(F2)) & (F17 & F6)) & (len(F2) < len(F14.Attr9))) & (F21 <= F23)
62      (((F16.Attr1 <= ((F21 - F10) + F8)) => F22) | (F7.Attr0 > len(F2))) => (len(F2) < (len(F16.Attr6)
63      - len(F10.Attr4)))

```

Código A.2: Primer modelo UVL generado para la evaluación de O1.

```

1 include
2   Boolean.group-cardinality
3   Arithmetic.*

```

```

4      Type.string-constraints
5  features
6      F0
7          optional
8              F4 cardinality [4..5]
9          optional
10             Integer F3 {Attr5 25.09}
11                 mandatory
12                     F5
13                         mandatory
14                             F13 cardinality [5..5]
15             optional
16                 F2 cardinality [3..3] {Attr2 85}
17                     mandatory
18                         F6 {Attr8 true}
19                             mandatory
20                                 F12 cardinality [2..4]
21             mandatory
22                 F1
23                     alternative
24                         Real F8
25                             optional
26                                 Real F9 cardinality [2..2]
27                                     optional
28                                         F18
29                                             optional
30                                                 F17
31                                                     optional
32                                                         F16 cardinality [3..4] {Attr7 true}
33                                                         optional
34                                                             F15 cardinality [4..5] {Attr9 true}
35                                                             optional
36                                                                 Integer F21
37                                                                 optional
38                                                                    F20 cardinality [3..3]
39                                                                    optional
40                                                                        F19 cardinality [2..4] {Attr3 45.98, Attr4 true}
41                                                                optional
42                                                                    F14 cardinality [4..4]
43                                                                    alternative
44                                                                        Integer F23 cardinality [3..5]
45                                                                        F22 {Attr0 50.27}
46                                                                Integer F7
47                                                                    optional
48                                                                        F11 cardinality [5..5] {Attr6 true}
49                                                                    optional
50                                                                        Integer F10 {Attr1 29.88}
51  constraints
52      ((F9 >= (F19.Attr3 - F7)) => ((F10 + F3) >= F10.Attr1)) <=> ((! F16.Attr7 & ! F15) | F1)
53      ((F6.Attr8 & (F10 == ((F22.Attr0 / F3) - F7))) <=> F6) | (! F16.Attr7 <=> F1)
54      (((sum(F10, F7) <= F8) <=> (F10 == F19.Attr3)) & (F3 <= F23)) | ! F19.Attr4
55      (((F21 > F23) => (F5 <=> F2)) | ! F12) & (F8 <= F10)
56      (((F8 <= F7) | (F23 <= (F21 - F10))) & ! F14) <=> F12
57      ((((! F22 <=> F14) & F16.Attr7) => F18) & (sum(F22.Attr0, F3.Attr5) >= (F2.Attr2 * F9))) => (F16.
          Attr7 <=> F14)
58      (((F21 == (F2.Attr2 + F7)) <=> (F5 & F18)) & (F23 <= F9)) & ! F22
59      (((F4 | ! F14) <=> (F23 >= F9)) & (F3 == F19.Attr3)) & F15 => F4
60      ((F10 <= (F21 / F7)) | ((! F5 => F22) => F13)) | ! F1
61      (((F2.Attr2 + F7) > sum(F9, F21)) & (F7 <= sum(F9, F21))) <=> F15

```

```
62 ((sum(F22.Attr0, F23) <= F9) | (sum(F7, F3) == F2.Attr2)) & (F9 < F3)
63 (((F12 & F13) => ! F11) => (F23 <= avg(F3, F7))) | (F21 < F10)
```

Código A.3: Segundo modelo UVL generado para la evaluación de O1.

B MANUAL DE INSTALACIÓN Y DESPLIEGUE

B.1 Requisitos previos

- **Sistema operativo:** el entorno utilizado durante el desarrollo y validación del sistema ha sido Ubuntu 24.04 LTS. También es posible utilizar otros sistemas compatibles con Docker.
- **Lenguaje y runtime:** no es necesario instalar manualmente Python ni sus dependencias en el sistema anfitrión, ya que el entorno de ejecución se encuentra definido dentro de los contenedores del proyecto.
- **Gestor de paquetes:** las dependencias necesarias para ejecutar la aplicación y el generador se instalan automáticamente durante la construcción de las imágenes Docker.
- **Servicios externos:** no es necesario instalar manualmente servicios adicionales. La configuración Docker del proyecto incluye los contenedores necesarios para los servicios utilizados por UVLHub, entre ellos la base de datos MariaDB, Redis, Elasticsearch, MailHog y Selenium.
- **Otras herramientas:** es necesario disponer de Git para obtener el código fuente y Docker junto con Docker Compose para construir y ejecutar el entorno de desarrollo.

B.2 Obtención del código fuente

El código fuente de referencia se encuentra en el repositorio oficial de UVLHub:

`https://github.com/diverso-lab/uvlhub`

La versión documentada en esta memoria puede encontrarse en `https://github.com/antjimort/uvlhub`, que incorpora el componente `fm_generator` y su integración en el módulo *generator* de UVLHub. Para reproducir la versión evaluada, deben ejecutarse los siguientes comandos:

```
1 git clone git@github.com:antjimort/uvlhub.git
2 cd uvlhub
```

B.3 Instalación de dependencias y ejecución

Las dependencias del proyecto se gestionan mediante Docker, por lo que no es necesaria una instalación manual de paquetes. La construcción de los contenedores instala automáticamente las dependencias definidas en la configuración del proyecto. El siguiente comando construye las imágenes necesarias y levanta el entorno de desarrollo definido para UVLHub. Durante este proceso se instala también el paquete `fm_generator` y se construyen los recursos necesarios para su ejecución desde el navegador mediante Pyodide.

```
1 docker compose -f docker/docker-compose.dev.yml up --build
```

Para detener y eliminar los contenedores se usa el siguiente comando:

```
1 docker compose -f docker/docker-compose.dev.yml down
```

B.4 Despliegue en producción

La herramienta utilizada para el despliegue de la app principal ha sido Render, pero ha sido usada únicamente para las revisiones del tutor. También se ha usado Redis, utilizado como almacenamiento externo para la gestión de las sesiones de Flask para conservar el estado del proceso de configuración del generador.

No obstante, la plataforma utilizada no constituye un requisito del sistema, ya que la aplicación puede ser desplegada mediante otras alternativas compatibles con contenedores.

El despliegue oficial de UVLHub es gestionado por DiversoLab.

B.5 Verificación de la instalación

Una vez levantados los contenedores mediante el comando indicado en la Sección B.3, se puede comprobar que la instalación se ha realizado correctamente accediendo a la dirección local proporcionada por la aplicación durante el arranque.

La dirección de acceso local depende de los puertos publicados por la configuración Docker. Puede comprobarse mediante el siguiente comando:

```
1 docker compose -f docker/docker-compose.dev.yml ps
```

El servicio debe abrirse en el navegador utilizando el puerto expuesto en el equipo anfitrión, normalmente mediante una dirección de la forma

`http://localhost:<PUERTO>`. La salida del arranque permite comprobar que el contenedor web se ha iniciado correctamente.

```
web_app_container | [DEBUG] Indexing document with ID 'hubfile-12' in 'search_index'
web_app_container | [SUCCESS] Document 'hubfile-12' indexed successfully.
web_app_container | [REINDEX] Reindexing completed.
web_app_container | [✓] Reindexing completed successfully!
mariadb_container | 2026-09-07 11:15:13 9 [Warning] Aborted connection 9 to db: 'uvl
web_app_container | 🕒 Looking for hubfiles without FactLabel...
web_app_container | [✓] All hubfiles already have FactLabels!
mariadb_container | 2026-09-07 11:15:14 10 [Warning] Aborted connection 10 to db: 'u
web_app_container | * Serving Flask app 'app'
web_app_container | * Debug mode: on
web_app_container | WARNING: This is a development server. Do not use it in a product
web_app_container | * Running on all addresses (0.0.0.0)
web_app_container | * Running on http://127.0.0.1:5000
web_app_container | * Running on http://172.18.0.10:5000
web_app_container | Press CTRL+C to quit
web_app_container | * Restarting with stat
web_app_container | * Debugger is active!
web_app_container | * Debugger PIN: 114-430-192

w Enable Watch d Detach
```

Figura B.1: Salida de consola tras el arranque correcto del contenedor web.
Elaboración propia

B.5.1 Ejecución de las pruebas automatizadas

Una vez iniciado el entorno Docker, la suite de pruebas integrada del módulo *generator* puede ejecutarse mediante el siguiente comando:

```
1 docker compose -f docker/docker-compose.dev.yml exec web \
2   rosemary test generator --all
```

La ejecución correcta debe finalizar sin errores. Esta suite valida el flujo del asistente, la generación de modelos UVL, la persistencia de parámetros y las opciones de salida integradas en UVLHub.

Las pruebas del paquete independiente *fm_generator* se ejecutan desde la raíz del repositorio mediante:

```
1 pytest fm_generator/tests -v
```

B.5.2 Medición de cobertura

La cobertura del código del módulo *generator* integrado en UVLHub puede obtenerse ejecutando la suite mediante Rosemary dentro del contenedor web:

```
1 docker compose -f docker/docker-compose.dev.yml exec web \
2   rosemary test generator --all --coverage
```

El informe generado incluye tanto el código de producción como los ficheros de prueba. Para los resultados recogidos en esta memoria se considera únicamente la cobertura de los ficheros de producción, tal y como se explica en la Sección 7.5.

La cobertura del paquete independiente `fm_generator` puede obtenerse mediante el siguiente comando:

```
1  pytest fm_generator/tests -v --cov=fm_generator \  
2  --cov-report=term-missing --cov-report=html
```

El informe textual muestra las líneas no cubiertas, mientras que el informe HTML se genera en el directorio `htmlcov`. Los resultados de cobertura utilizados en esta memoria se describen en la Sección 7.5.

C MANUAL DE USUARIO

C.1 Acceso al sistema

La versión del generador descrita en este manual puede utilizarse ejecutando la aplicación en un entorno local, siguiendo los pasos indicados en la Sección ?? del manual de instalación y despliegue.

UVLHub dispone de una instancia pública en <https://www.uvlhub.io/>. No obstante, esta instancia puede evolucionar de forma independiente al código descrito en esta memoria. Por este motivo, la ejecución local mediante Docker constituye la forma de acceso reproducible a la versión evaluada en el presente trabajo.

La generación de modelos no requiere que el usuario disponga de una cuenta ni que realice un proceso de autenticación previo.

La pantalla inicial de UVLHub se muestra en la Figura C.1.

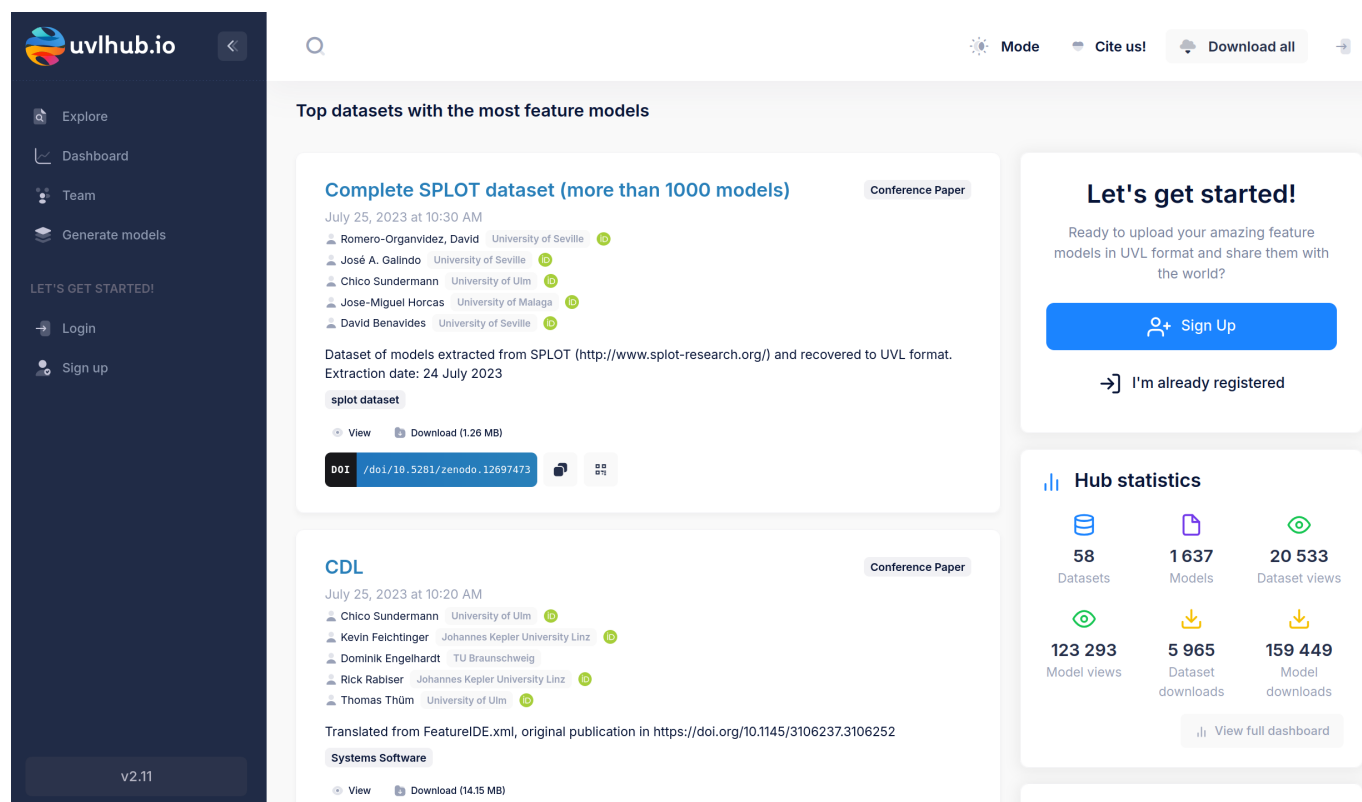


Figura C.1: Pantalla inicial de UVLHub. Elaboración propia

C.2 Funcionalidad principal

La funcionalidad principal del sistema consiste en la generación automática de modelos de características mediante un asistente de configuración (*wizard*). La estructura completa del flujo de generación y las pantallas correspondientes a cada etapa pueden consultarse en la Sección 5.4.1.

El funcionamiento del asistente consiste en configurar los diferentes parámetros de generación según las necesidades del usuario. En cada etapa del flujo se pueden modificar las opciones disponibles y avanzar al siguiente paso mediante el botón *Next*, situado en la parte inferior de la pantalla. En caso de querer modificar una configuración anterior, es posible volver a los pasos previos utilizando el botón *Previous*.

Una vez completada la configuración, en el sexto y último paso del asistente se generan los modelos de características solicitados. Desde esta pantalla el usuario dispone de diferentes opciones sobre los modelos generados:

- Descargar el conjunto de modelos generado mediante el botón *Download*.
- Visualizar un modelo directamente desde la aplicación mediante el botón con el icono del ojo.
- Descargar individualmente cada modelo mediante el botón verde disponible junto a cada resultado.
- Abrir un modelo mediante el botón morado con el icono del cohete en FlamapyIDE, un entorno externo que permite visualizar, editar y realizar tareas de análisis automático sobre modelos de características (<https://github.com/flamapy/flamapy-ide>).

C.3 Mensajes de error y resolución de incidencias

Durante la configuración, el asistente valida los valores introducidos en cada etapa antes de permitir que el usuario continúe al siguiente paso. Cuando una opción no cumple las reglas de negocio definidas en la Sección 4.5, se muestra un mensaje asociado al campo correspondiente indicando la condición que debe corregirse.

Estas validaciones incluyen, entre otros casos, límites numéricos incorrectos, distribuciones de probabilidades que no suman uno, dependencias entre niveles de expresividad y rangos inválidos para cardinalidades, restricciones o atributos. Una vez corregidos los valores señalados, el usuario puede continuar con el proceso de configuración y generar los modelos solicitados.

C.4 Contacto y soporte

Para cualquier duda relacionada con el funcionamiento del sistema o para reportar errores no contemplados en este manual, se puede contactar con el desarrollador mediante las siguientes direcciones de correo electrónico:

- antjimort@alum.us.es
- ajimenez34@us.es